Padrões de Arquitetura de Aplicações Corporativas PDF

Martin Fowler

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Padrões de Arquitetura de Aplicações Corporativas

Padrões Essenciais para uma Arquitetura de Aplicação Corporativa Bem-Sucedida

Escrito por Bookey

Saiba mais sobre o resumo de Padrões de Arquitetura de Aplicações Corporativas

Ouvir Padrões de Arquitetura de Aplicações Corporativas Audiolivro

Mais Livros Grátis

Escaneie para baixar

Ouça-o

Sobre o livro

Em "Padrões de Arquitetura de Aplicações Corporativas", o renomado especialista em engenharia de software Martin Fowler aborda os desafios únicos enfrentados pelos desenvolvedores de aplicações empresariais, como sistemas de reservas, soluções de cadeia de suprimentos e software financeiro. Reconhecendo que esses desenvolvedores se deparam com obstáculos distintos em comparação com aqueles que trabalham em sistemas de desktop ou embarcados, Fowler explora as complexidades da arquitetura corporativa, que é fundamental para o sucesso de projetos complexos. O livro apresenta uma coleção de padrões de design estabelecidos — soluções eficazes para problemas comuns — que servem como recursos valiosos para arquitetos e desenvolvedores. Ao oferecer um contexto e orientações perspicazes, Fowler capacita os leitores a tomarem decisões críticas sobre desempenho e acesso multiusuário, garantindo um desenvolvimento de aplicações robusto e eficiente.

Mais Livros Grátis

Escaneie para baixar

Ouça-o

Sobre o autor

Martin Fowler, nascido em 1963, é um proeminente engenheiro de software e o Cientista Chefe da ThoughtWorks. Reconhecido por sua expertise em design de software corporativo, ele é um fervoroso defensor das melhores práticas, especialmente em metodologias ágeis como a programação extrema.

Mais Livros Grátis

Escaneie para baixar

Ouça-o

Ad

Escanear para baixar

Experimente o aplicativo Bookey para ler mais de 1000 resumos dos melhores livros do mundo

Desbloqueie 1000+ títulos, 80+ tópicos

Novos títulos adicionados toda semana

duct & Brand | Liderança & Colaboração | Gerenciamento de Tempo | Relacionamento & Comunica

Estratégia de Negócios | Criatividade | Memórias | Conheça a Si Mesmo | Psicologia Po

mpreendedorismo | História Mundial | Comunicação entre Pais e Filhos | Autocuidado | Mi

Visões dos melhores livros do mundo

mento pos

Os 7 Hábitos das Pessoas Altamente Eficazes

Mini Hábitos

Hábitos Atômicos

O Clube das 5 da Manhã

Como Fazer Amigos e Influenciar Pessoas

Com Não

Teste gratuito com Bookey

Lista de conteúdo do resumo

Capítulo 1 : Para Quem Este Livro É

Capítulo 2 : Agradecimentos

Capítulo 3 : Colofão

Capítulo 4 : Arquitetura

Capítulo 5 : Aplicações Corporativas

Capítulo 6 : Tipos de Aplicações Corporativas

Capítulo 7 : Pensando em Performance

Capítulo 8 : Padrões

Capítulo 9 : Camadas

Capítulo 10 : Organizando a Lógica de Domínio

Capítulo 11 : Mapeamento para Bancos de Dados Relacionais

Capítulo 12 : Apresentação Web

Capítulo 13 : Concorrência

Capítulo 14 : Estado da Sessão

Mais Livros Grátis

Escaneie para baixar

Ouça-o

Capítulo 15 : Estratégias de Distribuição

Capítulo 16 : Unindo Tudo

Capítulo 17 : Padrões de Lógica de Domínio

Capítulo 18 : Padrões de Arquitetura de Fontes de Dados

Capítulo 19 : Padrões Comportamentais de Objeto-Relacional

Capítulo 20 : Padrões Estruturais Objetos-Relacionais

Capítulo 21 : Padrões de Mapeamento de Metadados Objetos-Relacionais

Capítulo 22 : Padrões de Apresentação Web

Capítulo 23 : Padrões de Distribuição

Capítulo 24 : Padrões de Concurrency Offline

Capítulo 25 : Padrões de Estado de Sessão

Capítulo 26 : Padrões Básicos

Mais Livros Grátis

Escaneie para baixar

Ouça-o

Capítulo 1 Resumo : Para Quem Este Livro É

Para Quem Este Livro É

Este livro é destinado a programadores, designers e arquitetos envolvidos na construção de aplicações corporativas, com foco em aprimorar seu entendimento sobre questões arquitetônicas e comunicação.

Grupos Alvo

1. **Necessidades Modestas**: Leitores que buscam criar seu próprio software encontrarão

Mais Livros Grátis
Escaneie para baixar
Ouça-o

padrões que fornecem uma compreensão fundamental, embora possam precisar de recursos adicionais mais tarde.

2.

Usuários de Ferramentas

: Aqueles que usam ferramentas corporativas obterão insights sobre os processos subjacentes e orientação na seleção de padrões suportados por ferramentas para lidar com situações específicas de mapeamento.

3.

Necessidades Exigentes

: Para leitores que aspiram a construir seu próprio software com requisitos complexos, recomenda-se cautela em relação à construção de frameworks, pois isso pode desviar o foco dos objetivos do projeto. Exemplos de código simplificados são fornecidos para compreensão, com a ressalva de que a personalização será necessária para atender a demandas mais altas.

Compreendendo Padrões

Padrões são soluções estabelecidas para problemas recorrentes e podem já ser familiares para profissionais experientes em aplicações corporativas. Este livro não introduz novas ideias, mas revisita conceitos estabelecidos

Mais Livros Grátis
Escaneie para baixar
Ouça-o

dentro da indústria. Seu objetivo é facilitar o aprendizado para novatos e melhorar a comunicação e o ensino entre aqueles familiarizados com as técnicas. O livro enfatiza a importância de um vocabulário comum entre designers para melhorar a clareza nas discussões sobre padrões de design, como "Facade Remota."

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Capítulo 2 Resumo : Agradecimentos

Agradecimentos

Contribuidores

Martin Fowler expressa gratidão a inúmeras pessoas que contribuíram para a escrita de seu livro. Entre os destaques estão David Rice, que teve um papel significativo no desenvolvimento do livro, e Matt Foemmel, que forneceu exemplos de código e feedback crítico. A documentação de Randy Stafford sobre a Camada de Serviço também é mencionada. Agradecimentos adicionais são direcionados a Edward Hieatt, Rob Mee e a um grupo de revisores oficiais cujas percepções moldaram o texto.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Esforços Colaborativos

Fowler credita colegas da ThoughtWorks por suas discussões e revisões de apoio, particularmente Kyle Brown, Rachel Reinitz e Bobby Woolf. Ele reconhece as conversas iniciais que moldaram o material, graças a contribuições de Alan Knight e Kai Yu, juntamente com feedback de várias pessoas que apoiaram sua exploração do .NET.

Feedback e Discussões

O autor aprecia o feedback valioso de colegas da indústria sobre o sistema de exemplo Foodsmart e reconhece as conversas enriquecedoras em workshops realizados por Bruce Eckel. Uma menção especial é feita à consultoria de Joshua Kerievsky sobre padrões, assim como as contribuições do grupo de leitura da UIUC.

Gratidão Pessoal

Fowler expressa imensa gratidão à sua esposa, Cindy, observando seu apoio inabalável durante todo o processo de escrita. Ele também reconhece muitos outros que forneceram

Mais Livros Grátis
Escaneie para baixar
Ouça-o

input, mas cujos nomes podem não ser lembrados, enfatizando o esforço coletivo na criação desta obra.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Capítulo 3 Resumo : Colofão

Colofão

Este livro marca o primeiro que Martin Fowler escreveu utilizando XML e tecnologias relacionadas. O texto principal foi desenvolvido como uma série de documentos XML com TextPad e um DTD personalizado. O XSLT foi empregado para gerar páginas da web para o site em HTML, enquanto o Visio foi usado para diagramas, utilizando os templates UML de Pavel Hruby. Um pequeno programa facilitou a importação automática de exemplos de código no resultado para agilizar o processo de escrita.
No rascunho inicial, tentou-se usar XSL-FO com Apache FOP, mas devido a limitações, scripts em XSLT e Ruby foram posteriormente escritos para importar texto no FrameMaker. Várias ferramentas de código aberto foram utilizadas, incluindo JUnit, NUnit, ant, Xerces, Xalan, Tomcat, Jboss, Ruby e Hsql, além de ferramentas comerciais como Visual Studio para .NET e IntelliJ's Idea para Java.
O livro foi adquirido pela Addison Wesley, com contribuições significativas de Mike Hendrickson, Ross Venables e Sarah Weaver, que supervisionaram sua

Mais Livros Grátis
Escaneie para baixar
Ouça-o

publicação. O manuscrito começou em novembro de 2000 e foi enviado para produção em junho de 2002, com um lançamento planejado para novembro de 2002 na OOPSLA. Dianne Wood foi a editora de texto, Kim Arney Mulcahy cuidou da composição do texto, e Cheryl Ferguson revisou as páginas. Irv Hershman preparou o índice.

Sobre a Imagem da Capa

Durante o período de escrita do livro, uma construção significativa da Leonard P. Zakim Bunker Hill Bridge estava em andamento em Boston. A ponte substituirá a atual de dois andares que leva a Interstate 93 sobre o Rio Charles. Notavelmente, é a ponte estaiada mais larga do mundo e a primeira nos EUA a apresentar um design assimétrico, refletindo um estilo popular na Europa. Embora esteticamente agradável, ela evoca lembretes bem-humorados sobre potenciais falhas de engenharia associadas a tais estruturas.
Martin Fowler, Melrose, Massachusetts, agosto de 2002, martinfowler.com

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Ad

Escanear para baixar

Por que o Bookey é um aplicativo indispensável para amantes de livros

Conteúdo de 30min
Quanto mais profunda e clara for a interpretação que fornecemos, melhor será sua compreensão de cada título.

Clipes de Ideias de 3min
Impulsione seu progresso.

Questionário
Verifique se você dominou o que acabou de aprender.

E mais
Várias fontes, Caminhos em andamento, Coleções...

Teste gratuito com Bookey

Capítulo 4 Resumo : Arquitetura

Seção	Resumo
Arquitetura	O termo 'arquitetura' pode ser confuso e se refere a aspectos significativos de design de um sistema, focando na divisão em alto nível e em decisões mais resistentes a mudanças.
Subjetividade da Arquitetura	A arquitetura é vista como subjetiva, dependendo de um entendimento compartilhado entre desenvolvedores. Decisões iniciais são vitais, pois são percebidas como mais difíceis de mudar depois.
Arquitetura em Camadas	A arquitetura em camadas é enfatizada como crucial para aplicações corporativas, com foco na decomposição e nas interações entre as camadas. Arquiteturas alternativas existem, mas são menos enfatizadas.
Padrões na Arquitetura	O livro discute vários padrões, misturando elementos arquitetônicos e de design sem uma distinção rigorosa, reconhecendo a natureza fluida da importância arquitetônica.

Arquitetura

O termo "arquitetura" na indústria de software é frequentemente estendido para várias interpretações e pode levar à confusão. Geralmente, denota aspectos significativos do design de um sistema, focando em dois elementos principais: a decomposição de mais alto nível de um sistema e decisões que são difíceis de mudar.

Subjetividade da Arquitetura

A arquitetura é cada vez mais reconhecida como subjetiva, refletindo uma compreensão compartilhada entre os desenvolvedores sobre o design de um sistema e seus principais componentes. As decisões tomadas no início do

Mais Livros Grátis
Escaneie para baixar
Ouça-o

processo de desenvolvimento são cruciais, pois são percebidas como mais difíceis de alterar posteriormente. O discurso em andamento destaca que, se alguns aspectos da arquitetura podem ser alterados mais facilmente do que se esperava, eles não são mais considerados arquitetônicos.

Arquitetura em Camadas

O autor enfatiza uma preferência pela arquitetura em camadas como um componente central das aplicações corporativas, que será detalhado no Capítulo 1. Este livro foca em como decompor uma aplicação corporativa em camadas e suas interações. Embora a arquitetura em camadas seja o foco principal devido à sua ampla aplicabilidade, arquiteturas alternativas como tubos e filtros são mencionadas brevemente como potencialmente valiosas em cenários específicos.

Padrões na Arquitetura

Alguns padrões discutidos no livro são decisões arquitetônicas significativas, enquanto outros se referem mais a elementos de design. A distinção entre padrões arquitetônicos e de design é subjetiva, e o autor não a

Mais Livros Grátis
Escaneie para baixar
Ouça-o

diferencia estritamente, reconhecendo a natureza fluida do que constitui uma importância arquitetônica.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Pensamento crítico

Ponto chave:Subjetividade nas Definições Arquitetônicas

Interpretação crítica:O autor sugere que a arquitetura é inerentemente subjetiva, o que provoca uma avaliação crítica sobre se essa perspectiva exagera a importância do consenso entre desenvolvedores em detrimento da rigorosidade técnica. Levanta questões sobre se definições flexíveis de arquitetura podem levar a inconsistências na prática, como visto nas discussões sobre normas de engenharia de software por fontes como Grady Booch e seu trabalho em 'Análise e Design Orientado a Objetos'. Assim, enquanto Fowler enfatiza a compreensão compartilhada, os leitores podem considerar o risco de consolidar princípios arquitetônicos essenciais sob interpretações subjetivas, potencialmente comprometendo a base de um design de sistema robusto.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Capítulo 5 Resumo : Aplicações Corporativas

Seção	Resumo
Visão Geral de Aplicações Corporativas	As aplicações corporativas apresentam desafios únicos, como gestão de dados complexa e regras de negócios não convencionais, embora não enfrentem alguns problemas vistos em outros setores, como telecomunicações.
Definição e Exemplos	Essas aplicações incluem sistemas como folha de pagamento e gestão da cadeia de suprimentos, mas excluem sistemas operacionais e videogames.
Gestão de Dados Persistentes	Envolve a gestão de dados persistentes acessíveis em múltiplas execuções, exigindo um design de banco de dados competente devido ao volume substancial de dados e à necessidade de migrações de dados.
Acesso Concorrente e Interfaces de Usuário	Many users access these applications simultaneously, leading to data integrity challenges. A variety of user interfaces is needed, with batch processing also being essential.
Desafios de Integração	As aplicações corporativas devem se integrar a outros sistemas, criando desafios na comunicação consistente e nos processos de negócios entre diferentes departamentos.
Complexidade da Lógica de Negócios	A lógica de negócios pode muitas vezes ser ilógica ou inconsistente, e os desenvolvedores devem endereçar e organizar essa lógica à medida que ela evolui ao longo do tempo.
Percepção do Tamanho das Aplicações Corporativas	O termo 'aplicação corporativa' geralmente sugere sistemas grandes, mas aplicações menores também podem agregar valor significativo, e simplificar a arquitetura pode aprimorar projetos maiores.

Visão Geral das Aplicações Corporativas

As aplicações corporativas são uma categoria distinta de software que apresenta desafios e complexidades únicos. Embora possam não ter os problemas intrincados de multithreading e integração de hardware encontrados em outros campos, como telecomunicações, elas frequentemente apresentam desafios significativos devido à gestão complexa

Mais Livros Grátis
Escaneie para baixar
Ouça-o

de dados e regras de negócios não convencionais.

Definição e Exemplos

Embora uma definição precisa seja desafiadora, as aplicações corporativas geralmente incluem sistemas como folha de pagamento, prontuários de pacientes, rastreamento de remessas e gestão da cadeia de suprimentos. Em contraste, não abrangem aplicações como sistemas operacionais, compiladores ou jogos eletrônicos.

Gestão de Dados Persistentes

As aplicações corporativas envolvem dados persistentes que devem ser acessíveis em várias execuções de programas, muitas vezes se estendendo por anos. Mudanças nos programas exigem migrações de dados, e o volume de dados pode ser substancial. Sistemas modernos geralmente dependem de bancos de dados relacionais, criando a necessidade de um design e gestão de banco de dados proficientes.

Acesso Concorrente e Interfaces de Usuário

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Muitos usuários acessam aplicações corporativas simultaneamente, o que introduz desafios na integridade dos dados e na prevenção de erros. Um número significativo de telas e interfaces de usuário é necessário para atender a diversas necessidades e capacidades dos usuários. Além disso, o processamento em lote é um aspecto vital frequentemente negligenciado em meio às considerações de interação do usuário.

Desafios de Integração

As aplicações corporativas raramente operam isoladamente; elas devem se integrar a várias outras aplicações desenvolvidas em diferentes momentos e com diferentes tecnologias. Essa interconectividade cria desafios na manutenção de uma tecnologia de comunicação consistente e processos de negócios entre departamentos ou divisões dentro de uma empresa.

Complexidade da Lógica de Negócios

O termo "lógica de negócios" muitas vezes reflete um conjunto de regras que são ilógicas ou inconsistentes. Os desenvolvedores enfrentam o desafio de organizar e

Mais Livros Grátis
Escaneie para baixar
Ouça-o

implementar essa lógica, que tende a evoluir ao longo do tempo, muitas vezes influenciada por casos de negócios únicos.

Percepção do Tamanho das Aplicações Corporativas

Embora o termo "aplicação corporativa" possa implicar em um sistema grande, muitas aplicações menores também podem oferecer um valor significativo. Negligenciar projetos pequenos pode minar o potencial impacto cumulativo que eles podem ter em uma organização. Simplificar a arquitetura e os processos pode efetivamente melhorar projetos maiores, dividindo-os em componentes gerenciáveis.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Exemplo

Ponto chave:Compreender a complexidade da gestão de dados persistentes em aplicações corporativas é crucial.

Exemplo:Imagine que você está desenvolvendo um sistema de folha de pagamento utilizado por milhares de funcionários em vários departamentos. A cada mês, as taxas de pagamento, deduções e status dos funcionários mudam, o que significa que sua aplicação precisa gerenciar esses dados persistentes de forma eficaz para garantir um processamento mensal preciso. Você percebe que, à medida que a empresa cresce, a migração de dados se torna uma necessidade frequente ao atualizar as versões de software, o que representa um sério risco de perda ou corrupção de dados se não for planejada com cuidado. Além disso, o volume de dados armazenados, desde horas trabalhadas pelos funcionários até declarações de impostos, deve ser organizado dentro de um banco de dados relacional para garantir acesso rápido e confiabilidade, ressaltando a importância de um design robusto de banco de dados para sustentar operações contínuas ao longo dos anos.

Mais Livros Grátis

Escaneie para baixar

Ouça-o

Pensamento crítico

Ponto chave:A complexidade da lógica de negócios em aplicações corporativas frequentemente leva à confusão e inconsistência.

Interpretação crítica:Fowler observa que o termo 'lógica de negócios' pode englobar regras que podem ser ilógicas, apresentando desafios para os desenvolvedores que precisam adaptar e implementar essas regras em evolução. Essa perspectiva convida à análise; enquanto as complexidades podem dificultar o design de software, pode-se argumentar que tais desafios também são oportunidades para soluções inventivas, como sugerem os defensores das metodologias ágeis (veja Beck e Cunningham para referências sobre os princípios ágeis). Além disso, se a lógica complexa é inerentemente prejudicial ou simplesmente um subproduto de ambientes de negócios em evolução é uma questão pronta para debate.

Mais Livros Grátis

Escaneie para baixar

Ouça-o

Capítulo 6 Resumo : Tipos de Aplicações Corporativas

Tipos de Aplicações Corporativas

Ao projetar aplicações corporativas, é crucial reconhecer sua diversidade e os problemas únicos que elas abordam. Em vez de aderir a uma solução universal, é essencial entender as alternativas e avaliar os trade-offs. Este capítulo destaca três tipos distintos de aplicações corporativas:

Exemplo 1: Varejista Online B2C

Um varejista online B2C deve gerenciar eficientemente um alto volume de usuários, exigindo escalabilidade e uma lógica de domínio simples para tarefas como captura de pedidos e notificação de envio. Utiliza uma apresentação web genérica para garantir acessibilidade em vários navegadores e conecta-se a um banco de dados para gerenciamento de pedidos e comunicação de inventário.

Exemplo 2: Sistema de Processamento de Contratos

Mais Livros Grátis
Escaneie para baixar
Ouça-o

de Locação

Em contraste, um sistema de processamento de contratos de locação envolve menos usuários, mas possui uma lógica de negócios mais complexa, incluindo cálculos para contas e manejo de diversos eventos de locação. Esses sistemas demandam uma interface de usuário sofisticada e interação devido à sua natureza complicada, o que resulta em um esquema de banco de dados mais intricado, com várias tabelas e integrações externas.

Exemplo 3: Sistema de Controle de Despesas

Um simples sistema de controle de despesas para uma pequena empresa é projetado para menos usuários com lógica básica e deve ser implementável rapidamente. Apesar de sua simplicidade, considerações para um possível crescimento devem ser levadas em conta, garantindo que ele possa se

Instalar o aplicativo Bookey para desbloquear texto completo e áudio

Mais Livros Grátis

Escaneie para baixar

Ouça-o

Ad

Escanear para baixar

App Store
Escolha dos Editores

22k avaliações de 5 estrelas

Feedback Positivo

Afonso Silva

s cada resumo de livro não só
o, mas também tornam o
m divertido e envolvente. O
tou a leitura para mim.

Fantástico! Brígida Santos

Estou maravilhado com a variedade de livros e idiomas que o Bookey suporta. Não é apenas um aplicativo, é um portal para o conhecimento global. Além disso, ganhar pontos para caridade é um grande bônus!

Fi

O
só
o
O

na Oliveira

correr as
ém me dá
omprar a
ar!

Adoro! Duarte Costa

Usar o Bookey ajudou-me a cultivar um hábito de leitura sem sobrecarregar minha agenda. O design do aplicativo e suas funcionalidades são amigáveis, tornando o crescimento intelectual acessível a todos.

Economiza tempo!

O Bookey é o meu apli
crescimento intelectua
perspicazes e lindame
um mundo de conheci

Aplicativo incrível! Estevão Pereira

Eu amo audiolivros, mas nem sempre tenho tempo para ouvir o livro inteiro! O Bookey permite-me obter um resumo dos destaques do livro que me interessa!!! Que ótimo conceito!!! Altamente recomendado!

Aplicativo lindo

Este aplicativo é um salva-vidas para
de livros com agendas lotadas. Os re
precisos, e os mapas mentais ajudar
o que aprendi. Altamente recomend

Teste gratuito com Bookey

Capítulo 7 Resumo : Pensando em Performance

Pensando em Performance

A performance é um aspecto crucial nas decisões arquiteturais, com muitas questões que são melhor abordadas por meio da medição inicial e da otimização disciplinada. Algumas decisões impactam a performance de maneiras que são difíceis de corrigir depois, levando a preocupações precoces dos membros do projeto. As orientações sobre performance não devem ser consideradas precisas até serem validadas em relação à configuração específica da aplicação.

Diretrizes e Verificação

Embora diretrizes como minimizar chamadas remotas sejam úteis, é essencial medir a performance na sua própria aplicação. Exemplos de código podem sacrificar a performance em prol de uma melhor compreensibilidade, e as otimizações não devem comprometer a clareza do código sem medição. Mudanças significativas na configuração

Mais Livros Grátis
Escaneie para baixar
Ouça-o

exigem uma reavaliação das otimizações de performance, pois melhorias anteriores podem não se manter em novos ambientes.

Termos Chave em Performance

1.
Tempo de Resposta
- Tempo levado para processar um pedido.
2.
Capacidade de Resposta
- Velocidade com que o sistema reconhece um pedido.
3.
Latência
- Tempo mínimo de resposta mesmo sem operações, muitas vezes uma preocupação em sistemas remotos.
4.
Taxa de Transferência
- A quantidade de trabalho realizada em um determinado tempo, medida em transações por segundo (TPS).
5.
Carga
- Quantidade de estresse em um sistema (número de usuários).

Mais Livros Grátis
Escaneie para baixar
Ouça-o

6.

Sensibilidade à Carga

- Como o tempo de resposta varia com a carga.

7.

Eficiência

- Performance relativa aos recursos utilizados.

8.

Capacidade

- Máxima taxa de transferência ou carga de trabalho efetiva.

9.

Escalabilidade

- Capacidade de adicionar recursos (geralmente hardware) para melhorar a performance.

Considerações sobre Escalabilidade

Sistemas escaláveis proporcionam melhorias de performance com a adição de hardware. A escalabilidade vertical adiciona potência a um servidor, enquanto a horizontal adiciona mais servidores. As decisões de design afetam múltiplos fatores de performance de maneiras diferentes, e entender esses fatores é crucial para avaliar escalabilidade e eficiência. Ao construir sistemas corporativos, focar em designs escaláveis pode ser mais benéfico do que apenas melhorar a capacidade ou

Mais Livros Grátis
Escaneie para baixar
Ouça-o

eficiência, pois isso permite opções de performance superiores no futuro. Muitas vezes, investir em hardware é mais rentável do que projetar software para funcionar efetivamente em sistemas menos poderosos.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Exemplo

Ponto chave:Importância da Medição de Desempenho Inicial

Exemplo:Imagine que você está desenvolvendo uma aplicação web para um cliente que espera milhares de usuários no lançamento. Se, durante a fase inicial de design, você decidir usar uma arquitetura de microserviços sem medir o impacto das chamadas a serviços remotos no tempo de resposta, poderá enfrentar um desempenho lento mais tarde, difícil de corrigir. Ao priorizar as medições de desempenho desde o início, você pode projetar um sistema que acomode a escalabilidade de forma suave à medida que a demanda dos usuários cresce, garantindo que as otimizações feitas ao longo do caminho se encaixem verdadeiramente na arquitetura e não comprometam o desempenho futuro.

Mais Livros Grátis

Escaneie para baixar

Ouça-o

Capítulo 8 Resumo : Padrões

Seção	Resumo
Introdução aos Padrões	Padrões oferecem soluções eficazes para desafios de design recorrentes, originados em aplicações do mundo real.
Valor dos Padrões	Padrões servem como referências práticas que ajudam a resolver problemas do mundo real sem a necessidade de um estudo abrangente.
Aplicação dos Padrões	Padrões devem ser adaptados a situações específicas, permitindo flexibilidade enquanto mantêm sua utilidade individual.
Comunicação através dos Padrões	Padrões promovem uma comunicação eficaz entre designers, fornecendo um vocabulário comum para discutir ideias complexas.
Estrutura dos Padrões	A estrutura inclui Nome, Intenção, Esboço, Problema Motivador, Como Funciona, Quando Usá-lo, Leituras Complementares e Exemplos.
Limitações dos Padrões	Os padrões não são exaustivos e devem ser vistos como pontos de partida para exploração e adaptação adicionais.

Padrões em Arquitetura de Aplicações Corporativas

Introdução aos Padrões

Padrões são um componente duradouro nas discussões de design. Christopher Alexander definiu um padrão como uma solução para problemas recorrentes aplicável em cenários diversos. Em software, um padrão representa soluções eficazes para desafios de design que se repetem, derivadas da observação de práticas bem-sucedidas.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Valor dos Padrões

A importância dos padrões reside em sua aplicação prática, servindo como uma referência que não requer uma leitura exaustiva. Compreender a essência dos padrões ajuda a abordar efetivamente problemas do mundo real e a adaptar soluções a contextos específicos.

Aplicação dos Padrões

Aplicar padrões requer adaptar soluções para se adequar a situações particulares. Os padrões não são estruturas rígidas, mas devem ser ajustados às necessidades de diferentes projetos. Embora cada padrão seja, em sua maioria, autossuficiente, eles também podem interagir e depender uns dos outros, criando uma rede de soluções.

Comunicação através dos Padrões

Os padrões aprimoram a comunicação entre os designers ao estabelecer um vocabulário compartilhado. O conhecimento de padrões específicos permite que os membros da equipe transmitam ideias complexas de forma rápida e eficiente, promovendo esforços colaborativos de design.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Estrutura dos Padrões

A estrutura de cada padrão inclui:

1. **Nome**: Essencial para criar um vocabulário comum.
2. **Intenção**: Uma breve descrição do propósito do padrão.
3. **Esboço**: Uma representação visual, muitas vezes usando diagramas UML.
4. **Problema Motivador**: Exemplos de problemas que o padrão aborda.
5. **Como Funciona**: Uma explicação da implementação e variações.
6. **Quando Usá-lo**: Contextos e trade-offs envolvidos no uso do padrão.
7.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Leitura Adicional

: Referências para informações adicionais.

8.

Exemplos

: Ilustrações de código simples do padrão em uso.

Limitações dos Padrões

Os padrões apresentados não são exaustivos; refletem o conhecimento e a experiência acumulados pelo autor. Os domínios do desenvolvimento de software são vastos demais para uma única perspectiva, e a compreensão continuará a evoluir ao longo do tempo. Os padrões devem ser vistos como pontos de partida para exploração e adaptação adicional, em vez de soluções definitivas.

Mais Livros Grátis

Escaneie para baixar

Ouça-o

Exemplo

Ponto chave:A Importância de Adaptar Padrões para Projetos Específicos

Exemplo:Imagine que você está liderando uma equipe encarregada de desenvolver uma plataforma de e-commerce complexa. Ao mergulhar no projeto, você percebe rapidamente que uma arquitetura padronizada não será suficiente. Você reúne sua equipe e se inspira em padrões estabelecidos para criar seu design, como a arquitetura de Microserviços e o padrão de Design Orientado a Domínio. No entanto, em vez de adotar esses frameworks de forma rígida, você avalia suas necessidades únicas—como a necessidade de alta escalabilidade durante vendas relâmpago ou uma experiência de usuário fluida durante o checkout. Ao adaptar esses padrões de forma cuidadosa, você não apenas aborda desafios específicos, mas também cria um sistema ágil que pode evoluir junto com as demandas em mudança do seu projeto. Essa abordagem adaptada não apenas aprimora a funcionalidade, mas também promove a coesão da equipe, pois vocês compartilham uma linguagem de design comum, ilustrando o verdadeiro poder dos padrões em se adaptar

Mais Livros Grátis
Escaneie para baixar
Ouça-o

às necessidades específicas de diversos projetos de software.

Pensamento crítico

Ponto chave:O conceito de padrões na arquitetura de software permite soluções de design escaláveis e flexíveis.

Interpretação crítica:Enquanto Martin Fowler enfatiza a utilidade dos padrões de aplicação como estruturas adaptáveis na arquitetura corporativa, é crucial reconhecer suas limitações também. Os padrões podem fornecer uma abordagem fundamental para a resolução de problemas, mas podem não abranger cada cenário único encontrado no desenvolvimento de software. Críticos argumentam que a adesão rígida aos padrões pode inibir a criatividade e a inovação ao promover uma mentalidade de tamanho único, sugerindo que os profissionais permaneçam críticos e abertos. Fontes como 'Design Patterns: Elements of Reusable Object-Oriented Software' de Gamma et al. e 'Domain-Driven Design' de Eric Evans destacam a importância do contexto na aplicação desses padrões, encorajando assim uma visão mais sutil em vez de aceitar a perspectiva de Fowler de forma acrítica.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Capítulo 9 Resumo : Camadas

Resumo do Capítulo 9: Camadas na Arquitetura de Aplicações Corporativas

Visão Geral das Camadas

A segmentação em camadas é uma técnica de design fundamental na arquitetura de software, onde sistemas complexos são divididos em subsistemas gerenciáveis, organizados em uma estrutura de "bolo de camadas". Cada camada depende da funcionalidade fornecida pelas camadas abaixo dela, enquanto permanece alheia às camadas superiores. Essa encapsulação facilita a compreensão, manutenção e substituição de componentes de camada sem impactar o sistema inteiro. No entanto, a segmentação pode trazer desvantagens, como mudanças em cascata e potencial degradação de desempenho devido à sobrecarga de transformação.

Contexto Histórico

Mais Livros Grátis
Escaneie para baixar
Ouça-o

A utilização de camadas tornou-se proeminente com o surgimento das arquiteturas cliente-servidor na década de 90. Sistemas iniciais não possuíam camadas, enquanto sistemas cliente-servidor tipicamente tinham duas camadas: a interface do usuário no cliente e o banco de dados no servidor. A crescente complexidade da lógica de domínio necessitou uma transição para sistemas de três camadas: apresentação, domínio e fonte de dados. A ascensão da web enfatizou ainda mais essa necessidade, permitindo mudanças modulares sem extensas reescritas.

Três Camadas Principais

1.
Camada de Apresentação

- Responsável pela interação do usuário e pela exibição de informações.

Instalar o aplicativo Bookey para desbloquear texto completo e áudio

Mais Livros Grátis

Escaneie para baixar

Ouça-o

Ad

Escanear para baixar

Ler, Compartilhar, Empoderar

Conclua Seu Desafio de Leitura, Doe Livros para Crianças Africanas.

O Conceito

BOOKS FOR AFRICA × ×

Esta atividade de doação de livros está sendo realizada em conjunto com a Books For Africa.Lançamos este projeto porque compartilhamos a mesma crença que a BFA: Para muitas crianças na África, o presente de livros é verdadeiramente um presente de esperança.

A Regra

Ganhe 100 pontos ----> **Resgate um livro** ----> **Doe para a África**

Seu aprendizado não traz apenas conhecimento, mas também permite que você ganhe pontos para causas beneficentes! Para cada 100 pontos ganhos, um livro será doado para a África.

Teste gratuito com Bookey

Capítulo 10 Resumo : Organizando a Lógica de Domínio

Padrão	Descrição	Vantagens	Desvantagens
Script de Transação	Um padrão baseado em procedimentos para lidar com ações do usuário, gerenciando entrada, validação e interações com o banco de dados.	Fácil de entender e implementar para desenvolvedores; simples de gerenciar as ações do usuário.	Pode levar a código duplicado e estrutura confusa à medida que a complexidade aumenta.
Modelo de Domínio	Uma abordagem orientada a objetos que organiza a lógica do domínio em torno de substantivos do domínio, com a lógica relacionada contida dentro de objetos.	Promove uma estrutura mais clara para lógica complexa; facilita a delegação e uma melhor organização dos comportamentos.	Desafiador para desenvolvedores não familiarizados com processos orientados a objetos; requer adaptação a um modelo mais rico.
Módulo de Tabela	Combina elementos de ambos os padrões; opera em um Conjunto de Registros com uma instância por conjunto de dados para cada tabela do banco de dados.	Oferece mais estrutura do que Scripts de Transação; melhor manejo das interações com o banco de dados.	Carece da flexibilidade de um Modelo de Domínio completo.
Escolhendo Entre Padrões	A escolha depende da complexidade da lógica do domínio; cenários mais simples podem não necessitar de um Modelo de Domínio, enquanto lógica complexa pode precisar.	Permite adaptação às necessidades do projeto com base na complexidade e na experiência do desenvolvedor.	Scripts de Transação podem ser inadequados para ambientes que favorecem interações complexas entre objetos.
Camada de Serviço	Age como uma ponte entre a lógica de apresentação e o modelo de domínio, simplificando a API e gerenciando transações e segurança.	Pode variar o envolvimento da lógica de negócios com base nas exigências do projeto.	Pode adicionar complexidade dependendo da extensão da lógica que lida.

Capítulo 10: Organizando a Lógica de Domínio

Neste capítulo, Martin Fowler explica três padrões principais para organizar a lógica de domínio: Script de Transação, Modelo de Domínio e Módulo de Tabela.

Script de Transação

Mais Livros Grátis
Escaneie para baixar
Ouça-o

O padrão mais simples é o Script de Transação, que atua como um procedimento para lidar com ações do usuário. Ele gerencia a entrada, realiza validações e cálculos, e interage com um banco de dados, oferecendo respostas de volta à camada de apresentação. Cada ação do usuário corresponde a um único procedimento, que pode incluir sub-rotinas para reduzir a duplicação de código. Embora seja fácil para os desenvolvedores entenderem e implementarem, Scripts de Transação podem levar a duplicação de código e a uma estrutura emaranhada à medida que a complexidade aumenta.

Modelo de Domínio

Em contrapartida, o Modelo de Domínio é uma abordagem orientada a objetos que organiza a lógica de domínio em torno dos substantivos do domínio. Cada classe representa diferentes conceitos de domínio, com a lógica relacionada contida dentro desses objetos. Esse modelo promove uma estrutura mais clara para lógica complexa, permitindo delegação e facilitando uma melhor organização dos comportamentos. No entanto, a transição para a utilização de um Modelo de Domínio pode ser desafiadora para desenvolvedores não familiarizados com processos

Mais Livros Grátis
Escaneie para baixar
Ouça-o

orientados a objetos, pois requer adaptação a um modelo mais rico.

Módulo de Tabela

O padrão Módulo de Tabela combina elementos tanto do Script de Transação quanto do Modelo de Domínio. Ele opera em um Conjunto de Registros, onde cada módulo corresponde a uma tabela de banco de dados, mas possui apenas uma instância por conjunto de dados, em vez de uma para cada registro. Esse padrão fornece mais estrutura do que Scripts de Transação e facilita o tratamento de interações com o banco de dados, mas carece de alguma da flexibilidade de um Modelo de Domínio completo.

Escolhendo Entre os Padrões

Escolher o padrão certo depende em grande medida da complexidade da lógica de domínio. Para cenários simples, um Modelo de Domínio pode ser excessivo, enquanto a lógica mais complexa pode exigir isso para melhor manutenção. Fatores como a experiência da equipe e o ambiente também influenciam a decisão. Scripts de Transação podem não ser adequados em ambientes que

Mais Livros Grátis
Escaneie para baixar
Ouça-o

suportam interações complexas de objetos, como o .NET.

Camada de Serviço

Uma Camada de Serviço pode atuar como uma ponte entre a lógica de apresentação e o modelo de domínio subjacente, fornecendo uma API simplificada e lidando com tarefas como gerenciamento de transações e segurança. O grau de lógica de negócios na Camada de Serviço pode variar de mínima a extensa, dependendo das necessidades do projeto. Em resumo, embora cada padrão tenha suas forças e fraquezas, Fowler incentiva uma consideração cuidadosa dos requisitos do projeto e das capacidades da equipe ao decidir sobre a estrutura apropriada para a lógica de domínio.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Pensamento crítico

Ponto chave:A escolha de padrões organizacionais para a lógica de domínio é dependente do contexto e subjetiva.

Interpretação crítica:Fowler enfatiza que selecionar a abordagem correta—seja um Script de Transação, Modelo de Domínio ou Módulo de Tabela—deve estar alinhado com as necessidades específicas de um projeto e a experiência da equipe. No entanto, é importante reconhecer que suas recomendações podem não ser aplicáveis de forma universal; diversas arquiteturas de projeto e dinâmicas de equipe podem levar a resultados diferentes. Por exemplo, enquanto uma organização pode ter sucesso com um Modelo de Domínio baseado em sua familiaridade com princípios de POO, outra pode achar isso excessivamente complexo e incômodo, preferindo um Script de Transação mais simples. Portanto, enquanto Fowler fornece uma estrutura, deve-se avaliar criticamente se seu ponto de vista se encaixa realmente em seu cenário único, conforme destacado em fontes como 'Code Complete' de Steve McConnell, que defende soluções adaptativas adaptadas a contextos específicos.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Capítulo 11 Resumo : Mapeamento para Bancos de Dados Relacionais

Resumo do Capítulo 11: Mapeamento para Banco de Dados Relacional

Introdução à Camada de Fonte de Dados

A camada de fonte de dados se comunica com diversos componentes de infraestrutura, focando principalmente em bancos de dados relacionais, que dominam os designs de aplicações modernas. SQL serve como uma linguagem padrão para comunicação com bancos de dados, apesar de complicações devido a adições específicas de fornecedores.

Padrões Arquiteturais

Os padrões arquiteturais definem como a lógica de domínio interage com os bancos de dados. Escolher o padrão certo impacta o design e pode ser difícil de refatorar. É essencial separar o acesso SQL da lógica do domínio, criando classes

Mais Livros Grátis
Escaneie para baixar
Ouça-o

distintas que correspondam à estrutura das tabelas do banco de dados.

Padrões de Gateway

Existem dois principais padrões de Gateway:

1. **Gateway de Dados de Linha**: Cada linha de uma consulta corresponde a uma instância de classe.
2. **Gateway de Dados de Tabela**: Uma instância por tabela do banco de dados, facilitando métodos para consultar o banco de dados que retornam dados.

Active Record e Modelo de Domínio

O Active Record combina a lógica de domínio com o acesso a dados, permitindo uma correlação mais próxima entre classes de domínio e tabelas do banco de dados. À medida que a complexidade do domínio cresce, torna-se necessário separar a lógica para manter uma arquitetura limpa, transitando para padrões como Data Mapper, que

Mais Livros Grátis
Escaneie para baixar
Ouça-o

proporciona isolamento entre modelos de domínio e bancos de dados.

Técnicas de Mapeamento

Diferentes estratégias de mapeamento se aplicam a relacionamentos, incluindo o manejo de coleções (um para muitos e muitos para muitos), e a aplicação de várias estratégias de herança. Decisões sobre como estruturar tabelas para hierarquias de objetos, incluindo Herança de Tabela Única, Tabela Concreta e Class Table Inheritance, afetam o desempenho e a manutenibilidade.

Tratando Lógica Complexa

Com lógica de domínio complexa, as camadas de persistência requerem um design cuidadoso. Padrões como Unidade de Trabalho mantêm a consistência nas interações do banco de dados, enquanto o Mapa de Identidade assegura uma representação única das linhas em memória.

Gerenciamento de Conexões

As conexões de banco de dados são cruciais e

Mais Livros Grátis
Escaneie para baixar
Ouça-o

frequentemente agrupadas para eficiência. As conexões devem ser gerenciadas para garantir que sejam fechadas prontamente, especialmente em contextos transacionais.

Pensamentos Finais sobre Mapeamento

Evite usar estruturas SQL excessivamente complexas e assegure clareza nas definições de métodos. Implementar padrões como Mapeamento de Metadados pode simplificar o código repetitivo envolvido em mapeamentos comuns.

Leitura Adicional

A exploração contínua do mapeamento objeto-relacional oferece diversas percepções de autores e recursos notáveis. Esse conhecimento aprimora a compreensão e a aplicação desses padrões arquiteturais.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Exemplo

Ponto chave:Importância de separar a lógica de domínio do acesso SQL

Exemplo:Imagine que você está construindo uma aplicação web onde os usuários podem gerenciar suas tarefas de forma ágil. Você decide implementar uma classe Task que incorpora a lógica de domínio essencial para sua aplicação. Contudo, se você permitir que essa classe trate diretamente das consultas SQL, torna-se difícil manter e atualizar tanto o acesso SQL quanto as regras de negócios incorporadas nessa classe. Em vez disso, ao adotar um padrão Gateway, como um Table Data Gateway, você cria uma fronteira clara onde uma classe TaskGateway gerencia todas as interações SQL. Essa separação não apenas melhora a clareza e a manutenibilidade, mas também facilita muito a atualização do esquema do seu banco de dados ou a troca para diferentes tecnologias de banco de dados mais tarde, sem perturbar a lógica central da sua aplicação.

Mais Livros Grátis

Escaneie para baixar

Ouça-o

Pensamento crítico

Ponto chave:Separação de Preocupações no Acesso a Dados

Interpretação crítica:A ênfase do autor em separar o acesso SQL da lógica de domínio representa uma decisão arquitetônica crítica que pode aprimorar a manutenibilidade e a escalabilidade. No entanto, é importante que os leitores considerem que esse ponto de vista pode não se aplicar de forma universal, pois certas aplicações podem se beneficiar de uma integração mais estreita entre o acesso a dados e a lógica de domínio por razões de simplicidade ou desempenho. Estruturas e metodologias alternativas sugerem que separações excessivamente rígidas podem levar a uma complexidade desnecessária (fonte: 'Domain-Driven Design: Tackling Complexity in the Heart of Software' de Eric Evans). Portanto, embora as estratégias de Fowler forneçam uma base valiosa, os desenvolvedores devem navegar o equilíbrio adaptado às necessidades específicas de suas aplicações.

Mais Livros Grátis

Escaneie para baixar

Ouça-o

Capítulo 12 Resumo : Apresentação Web

Seção	Resumo
Visão Geral das UIs baseadas na Web	As UIs baseadas em navegadores transformaram as aplicações corporativas ao oferecer acesso fácil e uma interface padrão sem a necessidade de instalação no cliente.
Estruturação de Programas em Servidores Web	Os programas de servidor web podem ser estruturados como scripts ou páginas de servidor; os scripts gerenciam as requisições HTTP, enquanto as páginas de servidor facilitam o manuseio de respostas em HTML.
Combinação de Abordagens de Script e Página de Servidor	Uma combinação de scripts para gerenciar requisições e páginas de servidor para renderização alinha-se ao padrão MVC, facilitando uma melhor separação entre entrada e saída.
Modelo-Visão-Controlador (MVC)	O MVC isola os modelos da apresentação web, melhorando as modificações e os testes, com o controlador gerenciando requisições e interações.
Controlador de Aplicação	Um Controlador de Aplicação gerencia o fluxo da aplicação e a exibição da tela, útil em aplicações complexas com extensa navegação.
Padrões de Visão	Três padrões de visão discutidos: Transformar Visão (transformação XSLT), Visão de Modelo (marcadores de conteúdo dinâmico) e Visão de Duas Etapas (renderização dividida para gerenciamento centralizado).
Padrões de Controlador de Entrada	O controlador de entrada pode ser específico de página ou usar uma arquitetura de Controlador Frontal para o manuseio centralizado de requisições, a fim de otimizar as operações.
Conclusão	O capítulo destaca a organização das apresentações web com MVC, controladores de aplicação e padrões de visão estruturados para flexibilidade e facilidade de desenvolvimento.
Leitura Adicional	Recursos recomendados focam no design para Java Web e aplicações mais amplas de padrões de design em diversos ambientes que não são Java.

Capítulo 12: Apresentação Web

Visão Geral das UIs Baseadas na Web

O surgimento de interfaces de usuário baseadas em navegadores web mudou significativamente as aplicações corporativas, oferecendo vantagens como facilidade de

Mais Livros Grátis
Escaneie para baixar
Ouça-o

acesso, ausência de instalação de software cliente e uma abordagem de UI uniforme. A configuração de aplicações web geralmente envolve a gestão de uma configuração de servidor que direciona URLs para programas específicos.

Estruturação de Programas em Servidores Web

Programas de servidores web podem ser estruturados como scripts ou páginas de servidor. Scripts (por exemplo, scripts CGI, servlets Java) gerenciam requisições HTTP e podem realizar operações complexas. No entanto, gerenciar respostas HTML diretamente através desses scripts é desafiador, levando ao desenvolvimento de páginas de servidor (por exemplo, PHP, ASP, JSP) onde HTML pode ser escrito juntamente com scriptlets para um manuseio mais fácil.

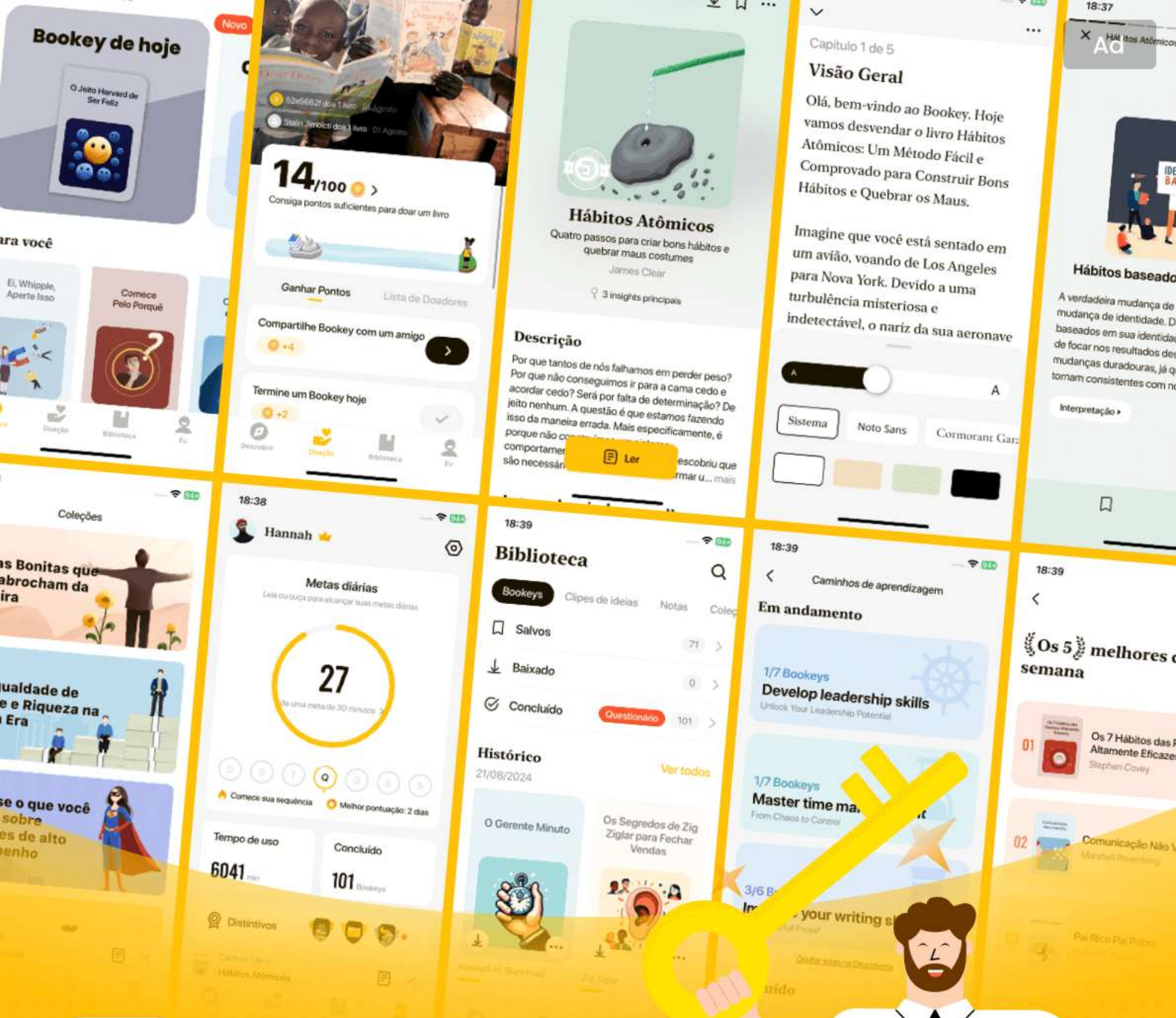
Combinação de Abordagens de Script e Página de

Instalar o aplicativo Bookey para desbloquear texto completo e áudio

Mais Livros Grátis

Escaneie para baixar

Ouça-o



As melhores ideias do mundo desbloqueiam seu potencial

Essai gratuit avec Bookey

Descarregar na App Store

OBTER NO Google Play

Escanear para baixar

Capítulo 13 Resumo : Concorrência

Capítulo 13: Concorrência

por Martin Fowler e David Rice

Introdução à Concorrência

A concorrência no desenvolvimento de software é complexa, especialmente quando múltiplas threads ou processos acessam os mesmos dados, levando a problemas potenciais. Apesar de sua prevalência em aplicações corporativas, a concorrência é frequentemente negligenciada devido às redes de segurança proporcionadas pelos gerenciadores de transações.

Concorrência Offline

A concorrência offline ocorre quando a manipulação de dados precisa abranger múltiplas transações de banco de dados, exigindo um gerenciamento cuidadoso para evitar problemas de concorrência. O capítulo enfatiza a necessidade de entender conceitos gerais de concorrência, especialmente

Mais Livros Grátis
Escaneie para baixar
Ouça-o

para aplicações corporativas.

Problemas Essenciais de Concorrência

O capítulo identifica problemas-chave de concorrência:

- **Atualizações Perdidas**: Alterações feitas por um usuário podem sobrescrever alterações de outro, levando à perda de dados.
- **Leituras Inconsistentes**: Ler diferentes pedaços de dados que estão corretos em momentos diferentes pode produzir resultados inválidos.

Contextos de Execução

A concorrência opera dentro de vários contextos de execução: solicitações, sessões, processos e threads. A adequada isolação dentro desses contextos é essencial para mitigar problemas de concorrência.

Isolamento e Imutabilidade

O isolamento e a imutabilidade são estratégias vitais para

Mais Livros Grátis
Escaneie para baixar
Ouça-o

gerenciar a concorrência. Isolar dados previne o acesso concorrente, enquanto dados imutáveis podem ser compartilhados sem risco de modificação.

Controle de Concorrência Otimista vs. Pessimista

Existem duas abordagens principais para lidar com dados mutáveis:

- **Controle de Concorrência Otimista**: Permite múltiplas edições e verifica conflitos ao atualizar.
- **Controle de Concorrência Pessimista**: Impede que outros editem os dados uma vez que um usuário esteja editando, reduzindo a concorrência, mas levando a potenciais impasses.

Impasses

Impasses surgem quando dois processos estão esperando um pelo outro para liberar um recurso. Técnicas para gerenciar impasses incluem detecção e timeouts, além de prevenir sua ocorrência controlando a ordem em que os bloqueios são adquiridos.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Transações em Aplicações Corporativas

Transações servem como a principal ferramenta de controle de concorrência em aplicações corporativas, definidas pelas propriedades ACID (Atomicidade, Consistência, Isolamento, Durabilidade). Recursos transacionais geralmente incluem bancos de dados, filas de mensagens e mais.

Gerenciamento de Transações de Negócios

Transações de negócios podem exigir múltiplas transações de sistema, complicando a necessidade de manter as propriedades ACID em chamadas separadas. O capítulo discute estratégias para reforçar a atomicidade e consistência ao longo dessas interações.

Padrões para Controle de Concorrência Offline

Embora aproveitar sistemas de transação seja preferível, os desenvolvedores podem precisar empregar padrões específicos para o gerenciamento de concorrência offline.
-

Bloqueio Offline Otimista

Mais Livros Grátis
Escaneie para baixar
Ouça-o

: Utiliza controle otimista durante transações de negócios para uma melhor experiência do usuário.

-

Bloqueio Offline Pessimista

: Proporciona detecção precoce de conflitos, mas pode prejudicar o desempenho do sistema.

-

Bloqueio de Granularidade Grossa e Bloqueio Implícito

: Simplifica o gerenciamento de concorrência ao reduzir o número de bloqueios individuais necessários.

Concorrência em Servidores de Aplicação

O capítulo muda o foco para a concorrência em servidores de aplicação, que não envolve transações, mas exige o manuseio eficiente de múltiplas solicitações. Ele discute o uso de processos por sessão e processos em pool como estratégias efetivas de concorrência.

Leitura Adicional

Para uma compreensão abrangente e técnicas avançadas sobre concorrência, os leitores são encorajados a explorar as obras de Bernstein e Newcomer, Lea, e Schmidt et al.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Pensamento crítico

Ponto chave:A complexidade da gestão de concorrência em aplicações corporativas.

Interpretação crítica:A análise de Fowler articula as complicações inerentes à concorrência no desenvolvimento de software, acentuando as nuances do controle de concorrência offline e as diversas abordagens, como o controle de concorrência otimista versus pessimista. No entanto, enquanto as estruturas de Fowler oferecem uma abordagem estruturada, pode-se argumentar que elas negligenciam a rápida evolução da tecnologia e as alternativas potenciais. A dependência de estratégias tradicionais pode dificultar a adaptação a novos paradigmas, como arquiteturas orientadas a eventos ou microserviços, que podem fornecer maneiras mais eficazes de gerenciar a concorrência. Por exemplo, a pesquisa de Lea (Java Concurrency in Practice) sugere a adaptação dos modelos de concorrência para coincidir com as técnicas modernas de processamento em múltiplos núcleos, significando que as percepções de Fowler, embora valiosas, devem ser consideradas juntamente com os avanços contemporâneos em design de software.

Mais Livros Grátis

Escaneie para baixar

Ouça-o

Capítulo 14 Resumo : Estado da Sessão

Capítulo 14: Estado da Sessão

Diferença Entre Transações de Negócio e Transações de Sistema

- Transações de negócio diferem de transações de sistema e afetam as estratégias de concorrência e armazenamento de dados.
- Esta distinção fundamenta o debate entre sessões sem estado e com estado.

Valor da Ausência de Estado

- Servidores sem estado não retêm informações entre as solicitações, o que ajuda na gestão de recursos.
- Um objeto sem estado ainda pode manter dados durante uma interação, mas se reinicia após cada solicitação.
- Exemplo: Um servidor responde a consultas de livros utilizando a ausência de estado, retornando novos dados a cada vez.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

- Servidores com estado retêm dados entre solicitações, mas podem levar a problemas de recursos.

Impacto de Com Estado vs. Sem Estado

- A ausência de estado permite o agrupamento de objetos, possibilitando uma melhor utilização de recursos para cenários de alto tráfego.
- Interações com estado, como carrinhos de compras, requerem memória de estados, complicando a gestão de recursos.
- O estado da sessão refere-se a dados específicos da sessão de um usuário que estão isolados de outras sessões.

Definição de Estado da Sessão

- O estado da sessão é temporário e deve ser comprometido para se tornar dado de registro permanente.
- Ele herda propriedades como ACID (Atomicidade, Consistência, Isolamento, Durabilidade), que são importantes para manter a integridade dos dados.

Armazenando o Estado da Sessão

Mais Livros Grátis
Escaneie para baixar
Ouça-o

1.
Estado da Sessão do Cliente

- Armazena dados da sessão no lado do cliente por meio de métodos como URLs, cookies ou campos ocultos.
- Desafios: Questões com largura de banda, segurança e isolamento.

2.
Estado da Sessão do Servidor

- Armazena dados na memória do servidor ou em formatos mais duráveis, como objetos serializados em fontes locais ou compartilhadas.
- As vantagens incluem uma gestão de recursos mais fácil, mas podem exigir identificadores de sessão para recuperação.

3.
Estado da Sessão em Banco de Dados

- Envolve a divisão de dados em tabelas para armazenamento em um banco de dados, semelhante aos dados de registro.
- Devem ser considerados os prós e contras referentes à complexidade, largura de banda, segurança e capacidade de resposta.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Desempenho e Capacidade de Resposta

- Cada opção de armazenamento afeta a capacidade de resposta do sistema e a eficiência de manuseio de dados de maneira diferente.
- Considerações incluem a complexidade dos dados, o número de usuários concorrentes e o cenário (ex: aplicações B2C).

Tratamento de Cancelamentos de Sessão

- O Estado da Sessão do Cliente permite fácil cancelamento de sessões, enquanto o Estado da Sessão do Servidor e do Banco de Dados requer mais gerenciamento.

Considerações de Desenvolvimento

- O Estado da Sessão do Servidor geralmente envolve menos esforço de desenvolvimento em comparação com os outros dois métodos.
- Uma abordagem híbrida pode ser eficaz, incorporando aspectos de diferentes métodos de armazenamento para várias partes do estado da sessão.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Conclusão

- A preferência por armazenar o estado da sessão pode variar com base no contexto, com uma inclinação para o Estado da Sessão do Servidor para armazenamento remoto e gerenciamento de IDs de sessão.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Capítulo 15 Resumo : Estratégias de Distribuição

Capítulo 15: Estratégias de Distribuição

Introdução aos Desafios de Distribuição

O capítulo discute as complexidades e desafios associados à distribuição de objetos em aplicações corporativas. Ele enfatiza que o apelo dos sistemas de objetos distribuídos muitas vezes oculta as armadilhas que muitos desenvolvedores encontram.

O Apelo dos Objetos Distribuídos

Arquitetos frequentemente projetam sistemas acreditando que a distribuição de objetos melhora o desempenho. No entanto, essa abordagem pode levar a uma degradação significativa de desempenho e complicar o desenvolvimento e a implantação do sistema.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Interfaces Remotas e Locais

Distinções importantes são feitas entre interfaces locais e remotas. Chamadas de procedimento locais rápidas contrastam fortemente com chamadas remotas significativamente mais lentas entre diferentes processos ou máquinas. Portanto, interfaces para objetos remotos devem ser de granularidade ampla para minimizar o número de chamadas, enquanto interfaces locais podem ser mais detalhadas.

Primeira Lei do Design de Objetos Distribuídos

Fowler adverte contra a distribuição de objetos. Em vez disso, ele defende o agrupamento, onde todas as classes são mantidas em um único processo, maximizando chamadas locais para melhor desempenho e programação mais simples.

Instalar o aplicativo Bookey para desbloquear texto completo e áudio

Mais Livros Grátis

Escaneie para baixar

Ouça-o

Ad

Escanear para baixar

Experimente o aplicativo Bookey para ler mais de 1000 resumos dos melhores livros do mundo

Desbloqueie 1000+ títulos, 80+ tópicos

Novos títulos adicionados toda semana

duct & Brand | Liderança & Colaboração | Gerenciamento de Tempo | Relacionamento & Comunica

Estratégia de Negócios | Criatividade | Memórias | Conheça a Si Mesmo | Psicologia Po

mpreendedorismo | História Mundial | Comunicação entre Pais e Filhos | Autocuidado | Mi

Visões dos melhores livros do mundo

mento pos

Os 7 Hábitos das Pessoas Altamente Eficazes

Mini Hábitos

Hábitos Atômicos

O Clube das 5 da Manhã

Como Fazer Amigos e Influenciar Pessoas

Com Não

Teste gratuito com Bookey

Capítulo 16 Resumo : Unindo Tudo

Resumo do Capítulo: Unindo Tudo

Visão Geral

Este capítulo sintetiza vários aspectos da arquitetura de aplicações corporativas, enfatizando a necessidade de considerar múltiplos padrões de design de forma holística. Embora revisite discussões anteriores, tem como objetivo fornecer um contexto abrangente para a seleção de padrões apropriados, com base nas necessidades e complexidades específicas do projeto.

Conselhos sobre Decisões Arquitetônicas

O autor reconhece as limitações de seus conselhos, incentivando os leitores a usar os insights fornecidos como catalisadores para seu próprio pensamento crítico, em vez de respostas definitivas. As decisões arquitetônicas devem, em última análise, alinhar-se com as realidades específicas de um projeto, reconhecendo que a refatoração é possível, mas

Mais Livros Grátis
Escaneie para baixar
Ouça-o

pode envolver desafios significativos.

Seleção da Camada de Domínio

Escolher a abordagem certa para a lógica de domínio é crucial, com três opções principais: Script de Transação, Módulo de Tabela e Modelo de Domínio. A complexidade da lógica de negócios é o principal fator que orienta essa decisão, juntamente com considerações de conectividade com o banco de dados.

-

Script de Transação

: Melhor para aplicações simples, mas tem dificuldades com lógica complexa.

-

Modelo de Domínio

: Melhor para aplicações complexas, oferecendo capacidades ricas, mas exigindo proficiência do desenvolvedor.

-

Módulo de Tabela

: Um compromisso que proporciona um tratamento de lógica de domínio melhor do que Scripts de Transação, ao mesmo tempo que se integra bem com bancos de dados relacionais.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Camada de Fonte de Dados

A seleção de padrões de fonte de dados deve alinhar-se com a camada de domínio escolhida.
- Para Script de Transação, as opções incluem Gateway de Dados de Linha e Gateway de Dados de Tabela.
- Para Modelo de Domínio, Recorde Ativo, Gateway de Dados de Tabela ou Gateway de Dados de Linha podem ser empregados, dependendo da complexidade e independência requerida do banco de dados.

Camada de Apresentação

As escolhas de apresentação dependem da implementação de um cliente rico ou uma interface de navegador HTML. A recomendação é utilizar HTML sempre que possível, com o padrão Modelo-Visão-Controlador sugerido para estruturar aplicações. Vários frameworks podem orientar decisões específicas sobre controladores e visões, dependendo da pilha tecnológica.

Orientação Específica de Tecnologia

O capítulo discute estratégias dentro de ambientes Java e

Mais Livros Grátis
Escaneie para baixar
Ouça-o

.NET, equilibrando entre padrões arquitetônicos e resultados tecnológicos específicos:

-

Java

: O conceito de Enterprise Java Beans (EJB) pode não ser necessário; POJOs com gateways apropriados podem ser suficientes.

-

.NET

: Enfatiza o padrão Módulo de Tabela, reconhecendo que Modelos de Domínio ainda são viáveis, mas costumam envolver mais complexidade.

Procedimentos Armazenados e Serviços Web

Procedimentos armazenados podem otimizar o desempenho, mas introduzem bloqueio de fornecedor, alertando os desenvolvedores a considerar seu uso com cautela. Serviços web são moldados mais como ferramentas para integração do que para construção de aplicações, alertando contra designs excessivamente complicados que envolvem múltiplos serviços web.

Esquemas de Camadas Alternativas

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Existem vários modelos de camadas alternativas, mostrando a flexibilidade no design arquitetônico. Cada alternativa tem suas forças e pode atender a diferentes requisitos de projetos, mas geralmente gira em torno das camadas básicas de apresentação, negócios e acesso a dados.

O capítulo conclui reiterando que a arquitetura deve se adaptar à complexidade de projetos e ambientes específicos, instando os leitores a explorarem abordagens personalizadas para aplicações corporativas robustas e escaláveis.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Capítulo 17 Resumo : Padrões de Lógica de Domínio

Padrão	Descrição	Quando Usar	Vantagens	Desvantagens
Script de Transação	Organiza a lógica de negócios através de procedimentos que lidam com solicitações únicas.	Aplicações pequenas e descomplicadas com uma série de transações.	Simples de implementar; gerenciamento direto de entrada, interações com o banco de dados e saída.	Pode levar à duplicação de código em transações.
Modelo de Domínio	Representa um modelo de objeto do domínio de negócios, encapsulando comportamento e dados.	Aplicações complexas onde as regras de lógica de negócios se entrelaçam.	Reduz a duplicação de código; melhora a legibilidade e oferece flexibilidade.	Mais complexo; requer compreensão de princípios avançados de design orientado a objetos.
Módulo de Tabela	Organiza a lógica de negócios em uma única instância para lidar com dados de uma tabela/visão do banco de dados.	Situações que envolvem dados tabulares.	Menos complexidade na recuperação e manipulação de dados; mantém os dados e o comportamento associados.	Carece de capacidades totalmente orientadas a objetos para lógica complexa.
Camada de Serviço	Define o limite da aplicação e encapsula a lógica de negócios enquanto coordena interações com o cliente.	Aplicações com múltiplos tipos de clientes e interações complexas.	Lógica de aplicação reutilizável e fácil de manter; gerencia transações de forma eficaz.	A implementação pode variar; pode incorrer em complexidade adicional, dependendo da tecnologia utilizada.

Resumo do Capítulo 17: Padrões de Arquitetura de Aplicações Corporativas

Script de Transação

O padrão Script de Transação organiza a lógica de negócios através de procedimentos, onde cada procedimento lida com

Mais Livros Grátis
Escaneie para baixar
Ouça-o

um único pedido da camada de apresentação. É benéfico para aplicações com uma série de transações, tornando fácil gerenciar entradas, interações com o banco de dados e saídas. Embora seja simples de implementar, pode levar à duplicação de código quando a lógica semelhante é necessária em diferentes transações. Um Script de Transação pode ser estruturado de várias maneiras, incluindo o uso de classes ou funções globais, e é mais adequado para aplicações pequenas e descomplicadas.

Modelo de Domínio

Um Modelo de Domínio representa um modelo de objeto do domínio de negócios, encapsulando tanto o comportamento quanto os dados. A complexidade da lógica de negócios muitas vezes requer esse padrão, especialmente quando regras e lógica começam a se entrelaçar. Um Modelo de Domínio rico oferece mais flexibilidade e opções para estruturar o código em comparação com um Script de Transação, reduzindo a duplicação de código e melhorando a legibilidade. No entanto, pode ser mais complexo e pode exigir a compreensão de princípios avançados de design orientado a objetos.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Módulo de Tabela

O padrão Módulo de Tabela organiza a lógica de negócios em uma única instância que lida com dados de uma tabela ou visualização do banco de dados. Ele mantém os dados e o comportamento intimamente associados, semelhante a abordagens tradicionais orientadas a objetos. Ao contrário dos Modelos de Domínio que gerenciam instâncias individuais, Módulos de Tabela representam uma coleção de itens, levando a menos complexidade na recuperação e manipulação de dados. Módulos de Tabela são mais bem utilizados em situações que envolvem dados tabulares, reconhecendo ainda que eles carecem de capacidades completas orientadas a objetos para gerenciar lógica complexa.

Camada de Serviço

A Camada de Serviço define a fronteira de uma aplicação, oferecendo um conjunto claro de operações enquanto encapsula a lógica de negócios. Ajuda a gerenciar transações e a coordenar respostas entre diferentes camadas de clientes que interagem. Existem duas implementações principais: a abordagem de fachada de domínio, onde fachadas finas sobre

Mais Livros Grátis
Escaneie para baixar
Ouça-o

o Modelo de Domínio lidam com operações, e a abordagem de script de operação, onde classes implementam a lógica da aplicação diretamente. Camadas de Serviço podem ser implementadas com várias tecnologias, incluindo POJOs ou session beans dentro do J2EE. Elas são mais úteis em aplicações com múltiplos tipos de clientes e interações complexas, garantindo que a lógica da aplicação seja reutilizável e manutenível.

Exemplo: Reconhecimento de Receita

O capítulo ilustra os diferentes padrões através de um cenário de reconhecimento de receita, explorando como cada um pode ser aplicado para gerenciar a lógica de negócios e as interações de dados de forma eficaz. Ao usar Script de Transação para casos simples, Modelos de Domínio para regras complexas, Módulo de Tabela para dados tabulares e Camada de Serviço para operações e coordenação, os padrões demonstram uma abordagem eficaz para estruturar aplicações corporativas. Cada método mostra forças em contextos específicos, enfatizando a importância de escolher o padrão certo com base nos requisitos da aplicação.

Conclusão

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Em resumo, padrões variados como Script de Transação, Modelo de Domínio, Módulo de Tabela e Camada de Serviço têm seu lugar na arquitetura corporativa. A escolha depende de vários fatores, incluindo complexidade, a necessidade de reutilização de código e interações de dados. Compreender esses padrões permite que arquitetos de software e desenvolvedores projetem aplicações que sejam robustas, eficientes e mais fáceis de manter.

Mais Livros Grátis

Escaneie para baixar

Ouça-o

Pensamento crítico

Ponto chave:Avaliação da Adequação dos Padrões

Interpretação crítica:Enquanto Fowler enfatiza a praticidade de diferentes padrões arquitetônicos para aplicações corporativas, é preciso considerar que tais categoriz ações podem simplificar em demasia as complexidades enfrentadas em aplicações do mundo real. O sucesso da implementação de Scripts de Transação ou Modelos de Domínio depende fortemente do contexto específico e das exigências em evolução de cada projeto, algo que a análise de Fowler pode não captar completamente. Críticos argumentam que a arquitetura de software não se trata apenas de seguir padrões estabelecidos, mas requer uma compreensão sutil de cenários únicos, como destacado em obras como "Padrões de Projeto: Elementos de Software Orientado a Objetos Reutilizáveis" de Gamma et al. A necessidade de adaptabilidade sugere que a adesão rígida à classificação de Fowler pode não resultar em resultados ótimos.

Mais Livros Grátis

Escaneie para baixar

Ouça-o

Capítulo 18 Resumo : Padrões de Arquitetura de Fontes de Dados

Capítulo 18: Padrões de Arquitetura de Fontes de Dados

Gateway de Dados da Tabela

-
Definição:
Um objeto gateway que interage com uma única tabela ou visualização do banco de dados.
-
Funcionalidade:
Manipula todas as operações CRUD. Cada instância, tipicamente, gerencia uma tabela, fornecendo métodos para selecionar, inserir, atualizar e excluir.
-
Implementação:

- Interfaces consistem em múltiplos métodos `find` para

Mais Livros Grátis
Escaneie para baixar
Ouça-o

acessar dados.
- Geralmente, sem estado, focando na recuperação e manipulação de dados.
- Retorna estruturas de dados como mapas ou Conjuntos de Registros para facilitar o mapeamento de objetos.
-

Casos de Uso:
Melhor para Scripts de Transação, funciona efetivamente com Módulos de Tabela. Pode ser utilizado para simplificar a lógica de acesso ao banco de dados e reduzir o acoplamento com o banco de dados.

Quando Usar Gateway de Dados da Tabela

-

Simples:
Ótimo para mapeamento direto de banco de dados.
-

Instalar o aplicativo Bookey para desbloquear texto completo e áudio

Mais Livros Grátis

Escaneie para baixar

Ouça-o

Ad

Escanear para baixar

Por que o Bookey é um aplicativo indispensável para amantes de livros

Conteúdo de 30min
Quanto mais profunda e clara for a interpretação que fornecemos, melhor será sua compreensão de cada título.

Clipes de Ideias de 3min
Impulsione seu progresso.

Questionário
Verifique se você dominou o que acabou de aprender.

E mais
Várias fontes, Caminhos em andamento, Coleções...

Teste gratuito com Bookey

Capítulo 19 Resumo : Padrões Comportamentais de Objeto-Relacional

Resumo do Capítulo 19: Unidade de Trabalho, Mapa de Identidade e Carregamento Sob Demanda

Unidade de Trabalho

O padrão Unidade de Trabalho gerencia uma lista de objetos afetados por uma transação de negócios, coordenando a gravação das alterações e resolvendo problemas de concorrência. Ele rastreia mudanças em objetos, incluindo criações, atualizações e exclusões, para garantir que essas alterações sejam refletidas com precisão no banco de dados. Realizar atualizações no banco de dados uma a uma pode ser ineficiente; assim, a Unidade de Trabalho permite agrupar alterações para reduzir o número de chamadas ao banco de dados.

Como Funciona

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Quando mudanças são feitas, uma instância da Unidade de Trabalho é criada para manter o estado. Ela rastreia objetos novos, alterados e excluídos, de modo que, quando uma transação é confirmada, pode atualizar o banco de dados de acordo, sem que o programador da aplicação precise gerenciar manualmente essas atualizações.
As opções para registrar objetos na Unidade de Trabalho incluem o registro pelo chamador, onde o usuário de um objeto registra as alterações, e o registro do objeto, onde o próprio objeto assume a responsabilidade pela atualização de seu status. O padrão também pode gerenciar a ordem de atualização e prevenir deadlocks controlando a sequência em que os objetos são modificados.

Quando Usar

A Unidade de Trabalho é particularmente útil em cenários que envolvem transações de negócios que abrangem múltiplas operações ou quando várias mudanças precisam ser coordenadas para manter a integridade dos dados em toda a sessão, mas não necessariamente escritas imediatamente.

Mapa de Identidade

Mais Livros Grátis
Escaneie para baixar
Ouça-o

O padrão Mapa de Identidade garante que os objetos sejam carregados apenas uma vez por sessão, mantendo um registro de todos os objetos carregados para evitar que o mesmo registro do banco de dados seja carregado em múltiplas instâncias de objeto. Isso ajuda a manter a consistência e otimiza o desempenho ao reduzir chamadas desnecessárias ao banco de dados.

Como Funciona

Um Mapa de Identidade mantém o controle dos objetos à medida que são carregados. Quando um objeto é solicitado, ele primeiro verifica o Mapa de Identidade. Se o objeto for encontrado, retorna essa instância; caso contrário, busca o objeto no banco de dados e o armazena no mapa.

Quando Usar

Use o Mapa de Identidade ao trabalhar com objetos que podem ser modificados durante uma única sessão, garantindo representações únicas em memória de cada registro do banco de dados. Ele atua como um cache para evitar leituras repetitivas do banco de dados.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Carregamento Sob Demanda

O padrão Carregamento Sob Demanda adia o carregamento de atributos ou objetos relacionados até que sejam especificamente acessados. Essa abordagem pode melhorar significativamente o desempenho ao evitar a sobrecarga de carregar dados desnecessários de uma só vez.

Como Funciona

O carregamento sob demanda pode ser implementado por meio de técnicas como inicialização preguiçosa, proxies virtuais, portadores de valor e objetos fantasmas. Cada método permite adiar a recuperação de dados até que o atributo específico seja necessário, otimizando assim o uso de recursos.

Quando Usar

O Carregamento Sob Demanda é vantajoso para campos que requerem chamadas separadas ao banco de dados para recuperação e que não são imediatamente necessários no

Mais Livros Grátis
Escaneie para baixar
Ouça-o

momento em que o objeto principal é carregado. Ele permite acessar dinamicamente dados relacionados sem incorrer no custo inicial de pré-carregar tudo.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Capítulo 20 Resumo : Padrões Estruturais Objetos-Relacionais

Resumo do Capítulo 20: Padrões de Arquitetura de Aplicações Corporativas

Introdução aos Padrões Estruturais Objeto-Relacional

Este capítulo discute vários padrões usados para mapear estruturas de programação orientada a objetos em bancos de dados relacionais, com foco em manter a integridade dos dados e a eficiência ao salvar e recuperar objetos.

Campo de Identidade

O padrão Campo de Identidade permite que um objeto mantenha um identificador exclusivo que vincula a representação na memória à linha correspondente no banco de dados. É crucial para garantir a consistência das identidades dos objetos entre os sistemas.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Como Funciona

-
Escolhendo sua Chave
: A escolha da chave primária pode ser significativa (como um Número de Seguro Social) ou insignificante (como um ID gerado). Chaves insignificantes são preferidas devido à sua simplicidade e ao risco reduzido de erro humano.
-
Tipos de Chave
: Chaves simples (campos únicos do banco de dados) são mais fáceis, mas têm limitações em comparação com chaves compostas (múltiplos campos), que capturam relacionamentos, mas podem complicar o código.
-
Tratamento de Chaves em Objetos
: O objeto deve representar a chave diretamente nos atributos, permitindo uma verificação de igualdade e recuperação eficientes.

Obter uma Nova Chave

Para criar novas entradas no banco de dados, você pode

Mais Livros Grátis
Escaneie para baixar
Ouça-o

escolher entre chaves geradas automaticamente, GUIDs (IDs Globalmente Únicos) ou métodos de geração manual, avaliando cada um com base na eficiência e praticidade em relação ao ambiente do banco de dados.

Mapeamento de Chave Estrangeira

O Mapeamento de Chave Estrangeira permite mapear relacionamentos entre objetos para chaves estrangeiras no banco de dados. Este método é essencial ao lidar com referências associativas em tabelas.

Mapeamento de Coleções

Lidar com coleções requer uma compreensão de como gerenciar atualizações e exclusões sem perder a integridade dos dados. As opções incluem deletar e reinserir, manter referências ou usar uma abordagem de diferença.

Mapeamento de Tabelas de Associação

Usado para relacionamentos de muitos-para-muitos, este padrão emprega uma tabela de vinculação para gerenciar associações. A tabela salva referências armazenando as

Mais Livros Grátis
Escaneie para baixar
Ouça-o

chaves primárias de ambos os objetos associados.

Mapeamento Dependente

Este padrão simplifica o mapeamento de objetos "filhos" que dependem exclusivamente de um objeto "pai" para sua existência no banco de dados. A classe pai gerencia a persistência e o ciclo de vida dos dependentes, facilitando as operações de atualização e exclusão.

Valor Embutido

O padrão Valor Embutido permite mapear um objeto em múltiplos campos de outro objeto, tornando-o adequado para objetos leves, como dinheiro ou intervalos de datas que não justificam sua própria tabela.

LOB Serializado

Este padrão lida com o armazenamento de um gráfico complexo de objetos como um único grande objeto (LOB) no banco de dados. Ele pode gerenciar efetivamente estruturas intrincadas que vão além do mapeamento simples de campos.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Herança de Tabela Única

Um método para representar hierarquias de classes em uma única tabela, incluindo todos os campos de todas as subclasses, tornando a recuperação de dados direta, mas potencialmente ineficiente devido a colunas vazias.

Herança de Tabela de Classe

Permite que várias tabelas representem diferentes classes na hierarquia. Mantém a estrutura de herança natural, mas pode levar a problemas de desempenho devido a joins ou consultas necessárias entre várias tabelas.

Herança de Tabela Concreta

Cada classe concreta tem sua própria tabela, permitindo a encapsulação completa dos atributos da classe. Este padrão ajuda a gerenciar relacionamentos e a integridade dos dados, mas complica o gerenciamento de chaves entre as tabelas.

Mapeadores de Herança

Organiza o mapeamento de hierarquias de herança para

Mais Livros Grátis
Escaneie para baixar
Ouça-o

garantir que tanto classes abstratas quanto concretas possam ser devidamente salvas e recuperadas, delegando responsabilidades de forma apropriada.

Conclusão

O capítulo enfatiza que um mapeamento objeto-relacional eficaz pode aprimorar significativamente a arquitetura de aplicações, garantindo um tratamento consistente dos dados, minimizando redundâncias e otimizando o desempenho de acordo com as limitações do modelo relacional.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Capítulo 21 Resumo : Padrões de Mapeamento de Metadados Objetos-Relacionais

Capítulo 21: Padrões de Mapeamento de Metadados Objetos-Relacionais

Introdução ao Mapeamento de Metadados

O Mapeamento de Metadados ajuda os desenvolvedores a gerenciar o mapeamento objeto-relacional (ORM) definindo mapeamentos em uma forma tabular, eliminando a codificação repetitiva frequentemente necessária para a correspondência entre o banco de dados e os objetos na memória.

Como Funciona

Existem dois métodos principais para implementar o Mapeamento de Metadados:
1.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Geração de Código
: Gera código-fonte com base nas entradas de metadados durante o tempo de compilação, resultando em código mais limpo e explícito que é mais fácil de depurar. As classes geradas não devem ser editadas manualmente e devem ser integradas ao processo de compilação.
2.
Programação Reflexiva
: Usa reflexão para acessar métodos de objetos dinamicamente. Isso pode levar a aplicações mais flexíveis, mas também pode introduzir problemas de desempenho e tornar a depuração desafiadora.

Escolhendo Entre Geração de Código e Reflexão

-
Desempenho
: A geração de código é geralmente mais rápida que a

Instalar o aplicativo Bookey para desbloquear texto completo e áudio

Mais Livros Grátis

Escaneie para baixar

Ouça-o

Ad

Escanear para baixar

App Store
Escolha dos Editores

22k avaliações de 5 estrelas

Feedback Positivo

Afonso Silva

s cada resumo de livro não só
o, mas também tornam o
m divertido e envolvente. O
tou a leitura para mim.

Fantástico! Brígida Santos

Estou maravilhado com a variedade de livros e idiomas que o Bookey suporta. Não é apenas um aplicativo, é um portal para o conhecimento global. Além disso, ganhar pontos para caridade é um grande bônus!

Fi

O
só
o
O

na Oliveira

correr as
ém me dá
omprar a
ar!

Adoro! Duarte Costa

Usar o Bookey ajudou-me a cultivar um hábito de leitura sem sobrecarregar minha agenda. O design do aplicativo e suas funcionalidades são amigáveis, tornando o crescimento intelectual acessível a todos.

Economiza tempo!

O Bookey é o meu apli
crescimento intelectua
perspicazes e lindame
um mundo de conheci

Aplicativo incrível! Estevão Pereira

Eu amo audiolivros, mas nem sempre tenho tempo para ouvir o livro inteiro! O Bookey permite-me obter um resumo dos destaques do livro que me interessa!!! Que ótimo conceito!!! Altamente recomendado!

Aplicativo lindo

Este aplicativo é um salva-vidas para
de livros com agendas lotadas. Os re
precisos, e os mapas mentais ajudar
o que aprendi. Altamente recomend

Teste gratuito com Bookey

Capítulo 22 Resumo : Padrões de Apresentação Web

Capítulo 22: Padrões de Apresentação Web

Modelo-Visão-Controlador (MVC)

O MVC divide a interação da interface do usuário em três papéis distintos: Modelo, Visão e Controlador. O Modelo representa as informações do domínio, a Visão lida com a exibição dessas informações e o Controlador gerencia a entrada do usuário, atualizando o Modelo e a Visão conforme necessário. Os princípios-chave do MVC incluem separar a apresentação do modelo, permitindo múltiplas apresentações da mesma informação, e separar a Visão do Controlador, embora este último seja menos crítico.

Controlador de Página

Um Controlador de Página gerencia páginas individuais em uma aplicação web, lidando com solicitações para páginas ou

Mais Livros Grátis
Escaneie para baixar
Ouça-o

ações específicas. Cada página lógica tem seu próprio controlador, estruturando a aplicação para vincular ações a um módulo específico. O Controlador de Página decodifica dados de URL, invoca objetos de modelo e encaminha informações relevantes para determinar qual visão exibir.

Quando Usar Controlador de Página

Os Controladores de Página funcionam bem com uma lógica de controlador simples, permitindo uma estrutura natural. Podem ser utilizados ao lado de Controladores Frontais, especialmente durante a refatoração do sistema.

Controlador Frontal

O padrão do Controlador Frontal consolida o tratamento de solicitações através de um único objeto manipulador, que pode realizar comportamentos comuns e direcionar solicitações para objetos de comando específicos. Este design visa reduzir a lógica duplicada entre os controladores, oferecendo uma abordagem simplificada para o gerenciamento de solicitações.

Quando Usar Controlador Frontal

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Ideal para aplicações complexas com comportamentos compartilhados, o Controlador Frontal simplifica as configurações do servidor e permite o gerenciamento dinâmico de comandos. Sua estrutura apoia melhor organização e extensibilidade do que um Controlador de Página.

Visão de Modelo

A Visão de Modelo utiliza marcadores embutidos em páginas HTML para gerar conteúdo de forma dinâmica. Esse enfoque minimiza complicações de manipulação de strings e é adequado para não-programadores, facilitando a edição de páginas. A chave para o sucesso é separar a lógica da camada de apresentação utilizando objetos auxiliares, reduzindo o uso de scriptlets dentro das visões.

Quando Usar Visão de Modelo

As Visões de Modelo são adequadas quando é necessário um manuseio fácil de layout e devem servir principalmente como visões sem se estender na lógica do controlador. Os modelos se beneficiam de estruturas mais simples, permitindo

Mais Livros Grátis
Escaneie para baixar
Ouça-o

modificações não técnicas enquanto permanecem testáveis e portáveis entre plataformas web.

Visão de Transformação

A Visão de Transformação transforma dados de domínio elemento por elemento em HTML, utilizando comumente XSLT. Ela se desvia da Visão de Modelo por se organizar em torno do processamento de dados de domínio em vez do design da apresentação. É particularmente útil para manipulação de dados sem estar entrelaçada com lógica de negócios.

Quando Usar Visão de Transformação

Aplicável onde múltiplos formatos ou dados XML detalhados requerem transformação consistente sem dificuldades do Wix, a Visão de Transformação suporta alterações na exibição enquanto mantém funcionalidade e facilidade de testes.

Exemplos

-

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Exibição Simples com um Controlador Servlet e Visão JSP (Java)
: Isso ilustra uma interação básica do Controlador de Página, exibindo informações de artistas através de um servlet que encaminha dados para uma JSP para apresentação.
-
Página de Servidor ASP.NET (C#)
: Isso demonstra como uma página web processa a entrada do usuário enquanto delega a lógica para uma classe de código por trás, organizando efetivamente as responsabilidades de controlador e visão.
-
Transformação Simples (Java)
: Este exemplo demonstra o uso de XSLT para transformar dados programaticamente, mantendo a separação entre a apresentação e a lógica de processamento.
Em resumo, os princípios e padrões MVC, como Controlador de Página, Controlador Frontal, Visão de Modelo e Visão de Transformação garantem uma segregação eficaz de responsabilidades, permitindo flexibilidade e manutenção no desenvolvimento de aplicações web.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Exemplo

Ponto chave:O MVC garante uma clara separação entre as camadas da interface do usuário

Exemplo:Imagine que você está desenvolvendo uma livraria online. Ao utilizar o padrão MVC, você pode garantir que os dados sobre os livros (Modelo) sejam mantidos distintos de como eles são exibidos no site (Visão), enquanto as interações do usuário, como adicionar um livro ao carrinho, são gerenciadas pelo Controlador. Essa separação não apenas torna sua aplicação modular, permitindo que outros desenvolvedores trabalhem no Modelo sem afetar a Visão, mas também simplifica o processo quando você deseja mudar a aparência do site ou introduzir novos recursos. Consequentemente, se você decidir reformular a apresentação para aumentar o engajamento do usuário, a lógica subjacente sobre como os livros são gerenciados permanece intacta, destacando o poder do MVC na criação de aplicações web adaptáveis e manuteníveis.

Mais Livros Grátis

Escaneie para baixar

Ouça-o

Capítulo 23 Resumo : Padrões de Distribuição

Capítulo 23: Padrões de Distribuição

Fachada Remota

- O padrão Fachada Remota fornece uma interface de alto nível que melhora a eficiência da rede ao acessar objetos de baixo nível, reduzindo o número de chamadas de método necessárias em interações remotas potencialmente caras.
- Objetos de baixo nível são eficientes dentro de um único espaço de endereço, mas se tornam custosos quando exigem comunicação entre processos devido ao empacotamento de dados, segurança e latência de rede.

Como Funciona

- A Fachada Remota atua como um intermediário entre o cliente e os objetos de baixo nível, garantindo que a lógica complexa permaneça na estrutura de baixo nível, enquanto

Mais Livros Grátis
Escaneie para baixar
Ouça-o

oferece uma interface simplificada para os chamadores remotos.
- Em vez de invocar múltiplos métodos para cada parte de um pedido, uma única chamada à fachada recupera ou atualiza todo o pedido, juntamente com as linhas ou dados relacionados, em uma única operação.

Considerações de Design

- Fachadas Remotas devem evitar conter qualquer lógica de domínio; seu papel principal é retransmitir chamadas para os objetos de baixo nível subjacentes.
- Elas podem ser com estado ou sem estado, onde fachadas sem estado podem oferecer benefícios de desempenho ao agrupar recursos. Designs com estado podem armazenar o estado da sessão, mas podem enfrentar desafios de escalabilidade.

Segurança e Controle de Transações

- Métodos na Fachada Remota fornecem pontos naturais para implementar verificações de segurança e gerenciar transações, garantindo eficiência e integridade dos dados.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Distinguir Fachada Remota de Fachada de Sessão

- A Fachada Remota fornece estritamente uma camada fina para acesso remoto, isenta de lógica de domínio, enquanto Fachadas de Sessão frequentemente misturam lógica de domínio, o que pode levar à confusão.

Comparação de Camada de Serviço

- A camada de serviço pode fornecer operações de alto nível sem a necessidade de acesso remoto, cumprindo diferentes propósitos arquiteturais ao projetar aplicações.

Cenários de Uso

- Utilize a Fachada Remota quando um cliente precisar de acesso remoto a um modelo de objeto de baixo nível, especificamente em casos onde um modelo de apresentação e um modelo de domínio operam em processos diferentes.
- Não é necessário para comunicação local entre processos; a Fachada Remota é otimizada para ambientes distribuídos ou situações B2C.

Objeto de Transferência de Dados (DTO)

Mais Livros Grátis
Escaneie para baixar
Ouça-o

- DTOs são estruturas projetadas para transportar dados entre processos de maneira eficiente, permitindo que múltiplos itens sejam enviados em uma única chamada de método, minimizando assim o número de chamadas necessárias em uma rede potencialmente cara.
- Normalmente representações simplificadas de dados de domínio, os DTOs devem ser serializáveis e estruturados para garantir compatibilidade de ambos os lados de qualquer barreira de distribuição.

Implementações Práticas

- Em aplicações Java distribuídas, usar beans de sessão como Fachadas Remotas ou implementar serviços Web demonstra a versatilidade desses padrões.

Desafios de Serialização

- Ao projetar DTOs, deve-se ter cuidado em relação à serialização, pois alterar a estrutura pode levar a problemas de compatibilidade significativos. A serialização baseada em texto (como XML) pode oferecer resiliência contra mudanças menores.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Conclusão

- O equilíbrio entre o uso de DTOs e Fachadas Remotas é essencial para uma gestão eficaz de dados em sistemas distribuídos, garantindo que a arquitetura permaneça modular e manutenível.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Exemplo

Ponto chave:Uso de Fachada Remota

Exemplo:Imagine que você está desenvolvendo uma plataforma de compras online onde os clientes fazem pedidos. Em vez de fazer o cliente enviar várias solicitações para obter os detalhes de cada item do banco de dados — o que poderia desacelerar o sistema — você cria uma Fachada Remota. Essa única interface recupera todos os detalhes do pedido, incluindo itens e preços, com apenas uma chamada, acelerando a experiência do usuário e minimizando o tráfego de rede. Ao implementar a Fachada Remota, você otimizou as comunicações remotas, tornando as interações mais eficientes e amigáveis.

Mais Livros Grátis

Escaneie para baixar

Ouça-o

Capítulo 24 Resumo : Padrões de Concurrency Offline

Resumo do Capítulo 24: Padrões de Concurrency Offline

Visão Geral

O Capítulo 24 discute vários padrões para gerenciar a concorrência na arquitetura de aplicações corporativas, focando especificamente em métodos de concorrência offline—Bloqueio Otimista Offline, Bloqueio Pessimista Offline, Bloqueio de Grão Grosso e Bloqueio Implícito.

Bloqueio Otimista Offline

- Previne conflitos em transações concorrentes validando as mudanças antes de confirmar, garantindo a consistência dos dados.
- Permite que múltiplos usuários trabalhem nos mesmos registros simultaneamente, assumindo que conflitos são

Mais Livros Grátis
Escaneie para baixar
Ouça-o

raros.
- A implementação geralmente envolve números de versão associados a registros, garantindo que as mudanças possam ser validadas antes de prosseguir com as atualizações.

Como Funciona

- As mudanças são validadas em relação à versão atual dos dados. Se a versão corresponder, a mudança é aplicada; caso contrário, ocorre um rollback.
- Pode envolver a verificação de múltiplos registros relacionados para evitar leituras inconsistentes, especialmente em cenários como o cálculo de impostos com base em dados de endereço.
- Estratégias adicionais incluem a detecção de leituras inconsistentes e a gestão de problemas de concorrência através de verificações na lógica da aplicação.

Instalar o aplicativo Bookey para desbloquear texto completo e áudio

Mais Livros Grátis

Escaneie para baixar

Ouça-o

Ad

Escanear para baixar

Ler, Compartilhar, Empoderar

Conclua Seu Desafio de Leitura, Doe Livros para Crianças Africanas.

O Conceito

BOOKS FOR AFRICA × ×

Esta atividade de doação de livros está sendo realizada em conjunto com a Books For Africa.Lançamos este projeto porque compartilhamos a mesma crença que a BFA: Para muitas crianças na África, o presente de livros é verdadeiramente um presente de esperança.

A Regra

Ganhe 100 pontos ----> **Resgate um livro** ----> **Doe para a África**

Seu aprendizado não traz apenas conhecimento, mas também permite que você ganhe pontos para causas beneficentes! Para cada 100 pontos ganhos, um livro será doado para a África.

Teste gratuito com Bookey

Capítulo 25 Resumo : Padrões de Estado de Sessão

Resumo do Capítulo 25: Padrões de Estado de Sessão

Estado de Sessão do Cliente

- O Estado de Sessão do Cliente envolve armazenar dados de sessão no lado do cliente, exigindo que o cliente envie todos os dados da sessão com cada solicitação.
- Essa abordagem permite que o servidor permaneça sem estado.
- As técnicas para gerenciar dados de sessão do cliente incluem:
  - **Parâmetros de URL**: Simples para pequenos dados, mas limitados pelo tamanho da URL e problemas de favoritos.
  - **Campos Ocultos**

Mais Livros Grátis
Escaneie para baixar
Ouça-o

: Dados serializados enviados com formulários; no entanto, visíveis no código-fonte da página.
-

Cookies

: Trocados automaticamente, mas os usuários podem desativá-los, e são limitados a um domínio.

Quando Usar o Estado de Sessão do Cliente

- Vantagens: Suporta objetos de servidor sem estado, adequado para clustering e failover.
- Desvantagens: Tamanhos de dados maiores complicam o armazenamento e a transferência, riscos potenciais de segurança e o desafio de gerenciar IDs de sessão.

Estado de Sessão do Servidor

- Neste padrão, o estado da sessão é armazenado no servidor, seja na memória ou serializado para armazenamento persistente.
- O servidor recupera a sessão usando um ID de sessão.
- Questões comuns incluem recursos mantidos na memória, formatos de serialização e a necessidade de armazenamento persistente.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Implementação em Java

- As técnicas incluem o uso de sessões HTTP ou beans de sessão com estado.
- Beans com estado simplificam a persistência, mas podem introduzir riscos de afinidade ao servidor.

Implementação em .NET

- Recursos de estado de sessão integrados permitem armazenamento em processos de servidor ou ajustes usando um serviço de estado para armazenamento baseado em rede.

Quando Usar o Estado de Sessão do Servidor

- Atrai devido à simplicidade; geralmente requer programação mínima.
- O desempenho depende da estratégia de implementação—se em memória é viável ou se requer configurações mais complexas.

Estado de Sessão do Banco de Dados

Mais Livros Grátis
Escaneie para baixar
Ouça-o

- Este padrão envolve salvar a sessão como dados comprometidos em um banco de dados.
- Fluxo de trabalho típico: recuperar dados de sessão do banco de dados, processar e salvar todos os dados relevantes de volta.

Gerenciamento de Dados de Sessão

- As estratégias incluem o uso de IDs de sessão para rastrear estados pendentes e criar tabelas separadas.
- Desafios surgem na gestão de rollbacks e na garantia da integridade dos dados de sessão.

Quando Usar o Estado de Sessão do Banco de Dados

- Oferece benefícios de desempenho ao utilizar objetos sem estado para pool de servidores, enquanto equilibra a sobrecarga das interações com o banco de dados.
- A complexidade da programação gira em torno da manutenção da integridade da sessão e facilidade de clustering/disponibilidade.

Em conclusão, a escolha entre estado de sessão do cliente, do servidor ou do banco de dados depende de vários fatores, incluindo desempenho, esforço de programação e requisitos específicos de arquitetura de aplicações.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Capítulo 26 Resumo : Padrões Básicos

Resumo do Capítulo 26

Gateway

Um Gateway é um objeto que encapsula o acesso a um sistema ou recurso externo. Ele simplifica APIs complexas de recursos externos, como bancos de dados ou XML, ao envolvê-los em uma classe que imita uma interface de objeto comum. Esse padrão ajuda nos testes ao fornecer um ponto claro para implementar Service Stubs. As principais utilizações de um Gateway incluem facilitar a testagem de sistemas, trocar recursos sem alterar extensivamente o sistema e lidar com integrações de forma eficiente.

Mapper

Um Mapper estabelece a comunicação entre subsistemas independentes, permitindo que eles permaneçam alheios uns aos outros, o que é crucial para manter um acoplamento frouxo. É uma camada isolante que traduz dados entre

Mais Livros Grátis
Escaneie para baixar
Ouça-o

sistemas, sem que nenhum dos sistemas precise conhecer os detalhes do outro. Mappers são tipicamente escolhidos quando as interações entre subsistemas são complicadas.

Supertipo de Camada

Esse padrão se concentra em criar uma superclasse que consolida métodos compartilhados por todos os objetos dentro de uma camada, evitando código duplicado. Ele simplifica a manutenção e melhora a legibilidade ao fornecer um comportamento comum para todos os objetos em uma camada específica.

Interface Separada

O padrão de Interface Separada envolve a definição de uma interface em um pacote e sua implementação em outro. Isso ajuda a reduzir o acoplamento e permite uma arquitetura mais limpa, tornando possível que diferentes partes do sistema permaneçam alheias umas às outras enquanto ainda se comunicam. Esse padrão é benéfico quando é necessário quebrar dependências entre partes do sistema.

Registro

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Um Registro é um objeto centralizado e bem conhecido que permite que outros objetos encontrem recursos ou serviços comuns. Ele serve como um mecanismo de busca sem precisar criar dependências nos recursos reais. Embora os Registros possam facilitar a recuperação de dados, devem ser usados com cautela como dados globais, pois podem levar a um acoplamento forte se não forem gerenciados adequadamente.

Objeto de Valor

Um Objeto de Valor é um objeto simples cuja igualdade é baseada nos valores de seus campos, em vez da identidade. Objetos de Valor devem ser imutáveis para evitar bugs de aliasing, onde uma mudança em uma instância afeta outras. O dinheiro é um exemplo comum de um Objeto de Valor.

Dinheiro

A classe Dinheiro representa um valor monetário, encapsulando as complexidades de lidar com moedas e operações aritméticas para evitar erros de arredondamento. Deve incluir funcionalidades para gerenciar operações como

Mais Livros Grátis
Escaneie para baixar
Ouça-o

adição, subtração e conversão entre moedas de forma eficaz.

Caso Especial

O padrão de Caso Especial aborda cenários onde valores nulos podem interromper o polimorfismo ou causar erros em tempo de execução. Em vez de retornar nulo, você pode criar uma subclasse que fornece uma implementação padrão inofensiva. Isso pode ajudar a evitar verificações de nulidade e código boilerplate ao longo do sistema.

Plugin

O padrão Plugin suporta configurações em tempo de execução, permitindo que os desenvolvedores definam comportamentos que podem variar entre ambientes sem exigir recompilação. Ele estabelece um ponto centralizado para gerenciar configurações, carregando implementações com base no contexto em tempo de execução.

Service Stub

Um Service Stub substitui serviços externos complexos durante os testes por uma versão local simplificada,

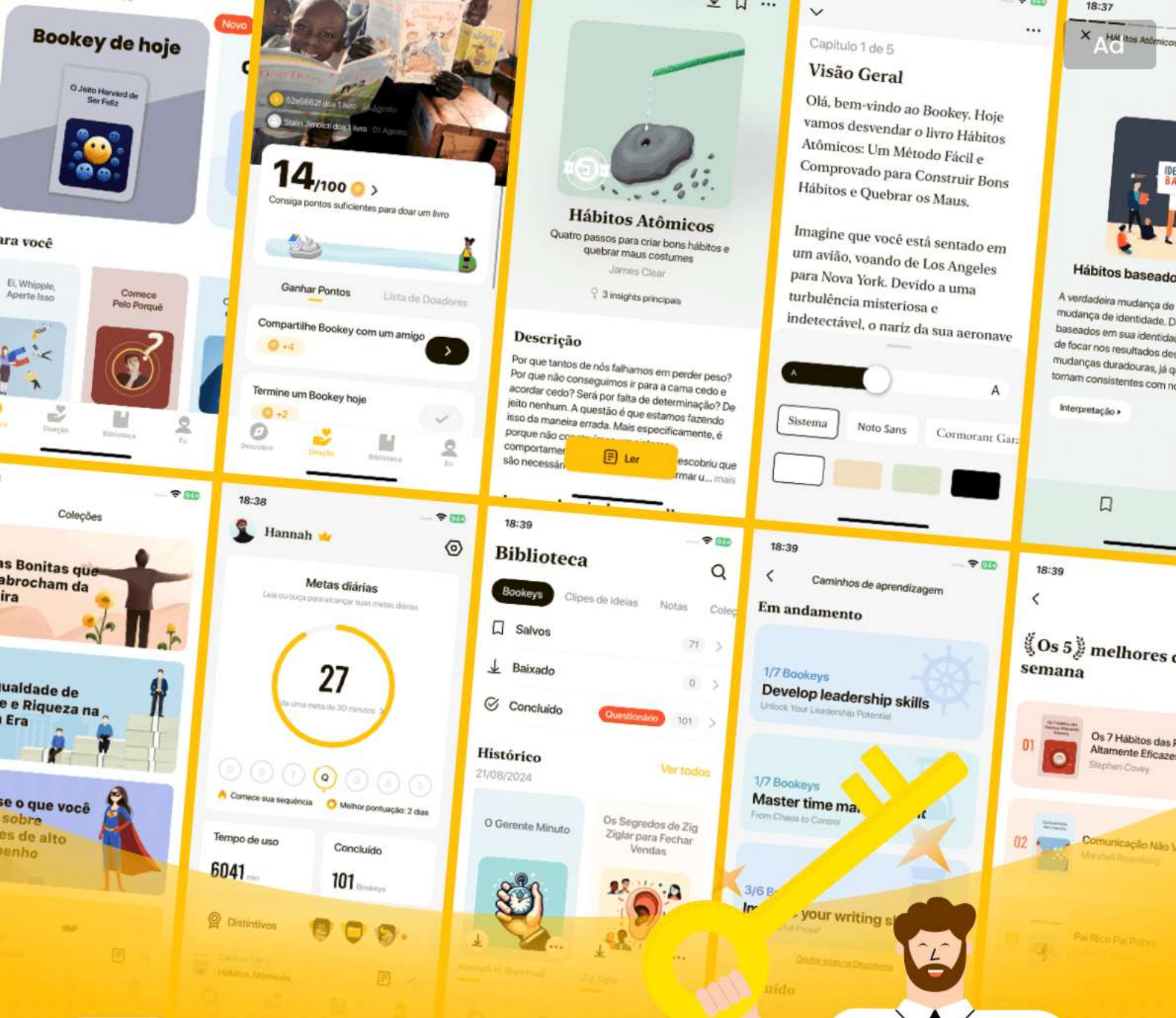
Mais Livros Grátis
Escaneie para baixar
Ouça-o

melhorando a experiência de desenvolvimento ao eliminar dependências de recursos externos. Esse padrão é particularmente valioso para garantir que os testes não sejam prejudicados pela indisponibilidade desses serviços.

Conjunto de Registros

O padrão Conjunto de Registros representa dados tabulares na memória, permitindo interação e manipulação semelhantes a um conjunto de resultados de banco de dados. Ele se desconecta de fontes de dados para facilitar o uso em diferentes contextos, enquanto também permite que as atualizações sejam devolvidas ao banco de dados, se necessário.
Este capítulo descreve vários padrões e suas aplicações na Padrões de Arquitetura de Aplicações Corporativas, enfatizando a necessidade de abstração, acoplamento frouxo e gerenciamento eficaz de recursos em sistemas complexos.

Mais Livros Grátis
Escaneie para baixar
Ouça-o



As melhores ideias do mundo desbloqueiam seu potencial

Essai gratuit avec Bookey

Descarregar na App Store

OBTER NO Google Play

Escanear para baixar

Melhores frases do Padrões de Arquitetura de Aplicações Corporativas por Martin Fowler com números de página

Ver no site do Bookey e gerar imagens de citações bonitas

Capítulo 1 | Frases das páginas 31-33

1. Escrevi este livro para programadores, designers e arquitetos que estão construindo aplicações corporativas e que desejam melhorar tanto sua compreensão das questões arquitetônicas quanto sua comunicação sobre elas.
2. Se você é novo neste campo, espero que o livro o ajude a aprender sobre essas técnicas. Se você já está familiarizado com as técnicas, espero que o livro o ajude a se comunicar e ensiná-las a outros.
3. Uma parte importante dos padrões é tentar construir um vocabulário comum, para que você possa dizer que esta classe é uma Fachada Remota, e outros designers saberão o que você quer dizer.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Capítulo 2 | Frases das páginas 34-39

1. Devo mais do que posso expressar ao meu excelente painel de revisores oficiais: John Brewer, Rob Mee, Kyle Brown, Gerard Meszarios, Jens Coldewey, Dirk Riehle, John Crupi, Randy Stafford, Leonard Fenster, David Siegel, Alan Knight, Kai Yu.
2. Aprendi muito com as muitas pessoas que trabalham neste campo e com quem tive boas conversas e colaborações.
3. Enquanto eu escrevia este livro, subi rascunhos na Web. Durante esse tempo, muitas pessoas me enviaram e-mails apontando problemas, fazendo perguntas ou discutindo alternativas.

Capítulo 3 | Frases das páginas 40-43

1. Eu tenho um link no meu site se você quiser acessá-los.
2. Eu escrevi um pequeno programa que importava automaticamente os exemplos de código para a saída, o que me livrou do pesadelo habitual de cortar e colar código.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

3. A primeira IDE que me empolgou desde o Smalltalk.
4. O design do texto é baseado no formato que usamos para Refatoração.
5. A ponte Zakim não é particularmente longa, mas é a ponte estaiada mais larga do mundo e também a primeira ponte estaiada dos EUA a ter um design assimétrico.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Escanear para baixar

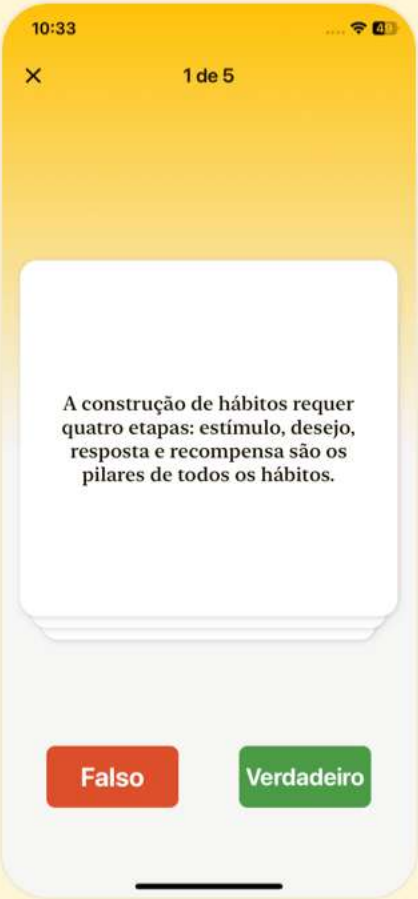
Baixe o app Bookey para desfrutar

Mais de 1000 resumos de livros com quizzes

Teste grátis disponível!

Descarregar na App Store

OBTER NO Google Play



Capítulo 4 | Frases das páginas 46-49

1. Arquitetura é um termo que muitas pessoas tentam definir, com pouco consenso.
2. Existem dois elementos comuns: um é a decomposição de mais alto nível de um sistema em suas partes; o outro, decisões que são difíceis de mudar.
3. Se você descobrir que algo é mais fácil de mudar do que você pensava, então não é mais arquitetônico.
4. No final, a arquitetura se resume ao que é importante—seja lá o que for.
5. O padrão arquitetônico que eu mais gosto é o de camadas... este livro trata, portanto, de como você decompõe uma aplicação corporativa em camadas e como essas camadas interagem.

Capítulo 5 | Frases das páginas 50-56

1. Isso surge quando eu falo com alguns amigos da área de telecomunicações. De certa forma, as aplicações corporativas são muito mais fáceis do que o software de telecomunicações – não lidamos

Mais Livros Grátis
Escaneie para baixar
Ouça-o

com problemas graves de multiprocessamento, e não temos integração de hardware e software. Mas, em outros aspectos, é muito mais difícil.

2. As aplicações corporativas geralmente envolvem dados persistentes. Os dados são persistentes porque precisam estar disponíveis entre várias execuções do programa – na verdade, geralmente precisam persistir por vários anos.
3. Geralmente, há muitas telas de interface do usuário para lidar com isso. Não é incomum ter centenas de telas distintas.
4. Mesmo que haja uma mudança fundamental e a empresa instale uma nova aplicação para lidar com uma tarefa, os dados precisam ser migrados para a nova aplicação.
5. Quando você constrói um sistema operacional, se esforça para manter tudo lógico. Mas as regras de negócios são simplesmente dadas a você, e sem um grande esforço político, não há nada que você possa fazer para mudá-las.
6. Para algumas pessoas, o termo ‘aplicação corporativa’ implica um sistema grande. No entanto, é importante

Mais Livros Grátis
Escaneie para baixar
Ouça-o

lembrar que nem todas as aplicações corporativas são grandes, embora possam proporcionar muito valor para a empresa.

7. Uma das melhores coisas que você pode fazer é transformar um grande projeto em um pequeno, simplificando sua arquitetura e processo.

Capítulo 6 | Frases das páginas 57-62

1. Há uma ampla gama de alternativas a escolher, mas aqui, eu vou selecionar três pontos neste plano muito grande.
2. Se um sistema traz benefícios para os negócios (como todas as aplicações corporativas deveriam), adiar esses benefícios custa dinheiro.
3. Escolher uma arquitetura significa que você precisa entender os problemas particulares do seu sistema e escolher um design apropriado com base nesse entendimento.
4. Você não pode construir software corporativo sem pensar, e tudo o que qualquer livro pode fazer é fornecer mais

Mais Livros Grátis
Escaneie para baixar
Ouça-o

informações para embasar suas decisões.

5. Cuidado ao usar uma ferramenta que realmente é adequada para um tipo diferente de aplicação - isso pode dificultar mais do que ajudar.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Escanear para baixar

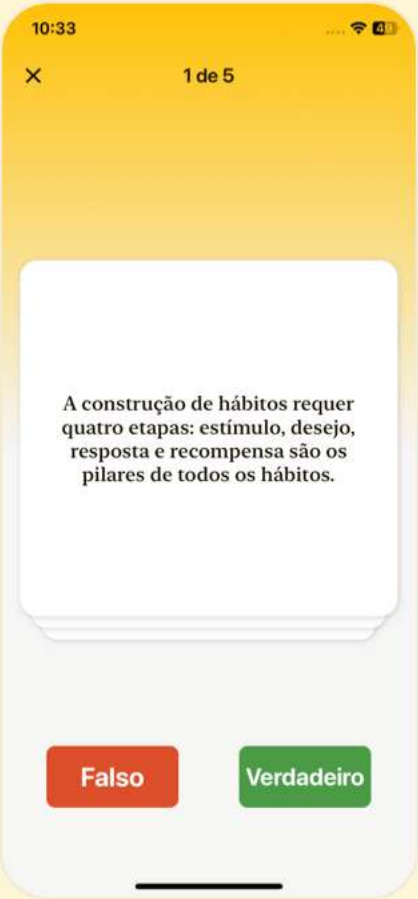
Baixe o app Bookey para desfrutar

Mais de 1000 resumos de livros com quizzes

Teste grátis disponível!

Descarregar na App Store

OBTER NO Google Play



Capítulo 7 | Frases das páginas 63-70

1. Para a maioria dos problemas de desempenho, prefiro colocar um sistema em funcionamento, instrumentá-lo e, em seguida, usar um processo de otimização disciplinado baseado em medições.
2. É sempre difícil falar sobre desempenho em um livro como este. A razão pela qual é tão difícil é que qualquer conselho sobre desempenho não deve ser tratado como fato até ser medido na sua configuração.
3. Sempre que você fizer uma otimização de desempenho, no entanto, deve medir tanto antes quanto depois, caso contrário, você pode apenas estar tornando seu código mais difícil de ler.
4. A escalabilidade é uma medida de como a adição de recursos (geralmente hardware) afeta o desempenho.
5. Se o Camel tem um custo maior que o Swordfish, e esse custo maior é equivalente a alguns servidores, então o Swordfish acaba sendo mais barato mesmo que você precise apenas de 40 tps.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Capítulo 8 | Frases das páginas 71-81

1. Cada padrão descreve um problema que ocorre repetidamente em nosso ambiente e, em seguida, descreve o cerne da solução para esse problema, de tal forma que você pode usar essa solução um milhão de vezes, sem nunca fazer da mesma maneira duas vezes." [Alexander, et al.]
2. Uma parte chave dos padrões é que eles estão enraizados na prática. Você encontra padrões observando o que as pessoas fazem, observando coisas que funcionam e, em seguida, procurando o 'cerne da solução.'
3. Uma vez que você precisa do padrão, deve descobrir como aplicá-lo às suas circunstâncias. Uma coisa importante sobre os padrões é que você nunca pode apenas aplicar a solução de forma cega... Costumo dizer que os padrões estão 'meio assados', o que significa que você sempre tem que terminá-los no forno do seu próprio projeto.
4. O resultado é que os padrões criam um vocabulário sobre design, e é por isso que a nomeação é uma questão tão

Mais Livros Grátis
Escaneie para baixar
Ouça-o

importante.

5. Este livro reflete meu entendimento atual, e esse entendimento se desenvolveu enquanto eu escrevia o livro. Espero que continue a evoluir muito depois que este livro se transforme em papel.
6. Lembre-se sempre de que cada padrão é incompleto e que você tem a responsabilidade, e a diversão, de completá-lo no contexto do seu próprio sistema.

Capítulo 9 | Frases das páginas 84-107

1. Dividir um sistema em camadas traz uma série de benefícios importantes.
2. Você pode substituir camadas por implementações alternativas dos mesmos serviços básicos.
3. Camadas são bons lugares para padronização.
4. A parte mais difícil de uma arquitetura em camadas é decidir quais camadas ter e qual deve ser a responsabilidade de cada camada.
5. Escolha a forma de separação mais apropriada para o seu problema, mas certifique-se de fazer algum tipo de

Mais Livros Grátis
Escaneie para baixar
Ouça-o

separação—pelo menos no nível de sub-rotinas.

6. O domínio e a fonte de dados nunca devem depender da apresentação.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Escanear para baixar

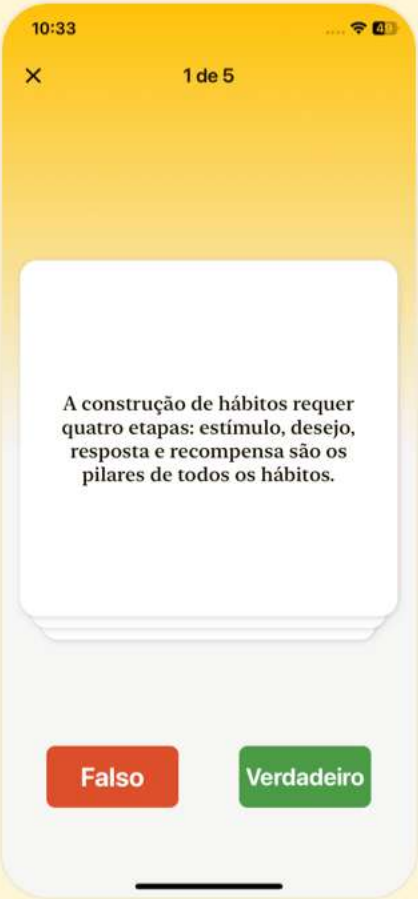
Baixe o app Bookey para desfrutar

Mais de 1000 resumos de livros com quizzes

Teste grátis disponível!

Descarregar na App Store

OBTER NO Google Play



Capítulo 10 | Frases das páginas 108-124

1. Um Script de Transação (110) é essencialmente um procedimento que recebe a entrada da apresentação, processa-a com validações e cálculos, armazena dados no banco de dados e invoca quaisquer operações de outros sistemas.
2. Com um Modelo de Domínio (116), construímos um modelo do nosso domínio que, pelo menos em uma primeira aproximação, está organizado principalmente em torno dos substantivos do domínio.
3. Usar um Modelo de Domínio (116) em vez de um Script de Transação (110) é a essência da mudança de paradigma da qual as pessoas orientadas a objetos falam tanto.
4. O valor de um Modelo de Domínio (116) reside no fato de que, uma vez que você se acostuma com as coisas, há muitas técnicas que permitem lidar com lógica cada vez mais complexa de uma maneira bem organizada.
5. A maior vantagem de um Módulo de Tabela (125) é como ele se encaixa no resto da arquitetura.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

6. Esses três padrões não são escolhas mutuamente exclusivas. De fato, é bastante comum usar Script de Transação (110) para parte da lógica de domínio e Módulo de Tabela (125) ou Modelo de Domínio (116) para o restante.

Capítulo 11 | Frases das páginas 125-179

1. A escolha que você faz aqui tem implicações profundas para seu design e, portanto, é difícil de refatorar, então é algo que você deve prestar atenção.
2. É prudente separar o acesso SQL da lógica de domínio e colocá-lo em classes separadas.
3. À medida que a lógica de domínio se torna mais complicada e você começa a se mover em direção a um Modelo de Domínio rico, a abordagem simples de um Active Record começa a falhar.
4. Uma Unidade de Trabalho mantém o controle de todos os objetos lidos do banco de dados, junto com todos os objetos modificados de alguma forma.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

5. Se você está usando um Modelo de Domínio, geralmente organiza as coisas de maneira que objetos vinculados sejam carregados juntos de forma que uma leitura de um objeto de pedido carregue seu objeto cliente associado.
6. O principal objetivo de um Mapa de Identidade é manter identidades corretas, e não aumentar o desempenho.
7. É geralmente melhor usar uma consulta que traga de volta linhas desnecessárias do que emitir 50 consultas individuais.
8. Um Mapeador de Dados lida com todo o carregamento e armazenamento entre o banco de dados e o Modelo de Domínio... permitindo que ambos variem independentemente.
9. Não presuma que uma ferramenta elimina todo o esforço. Ela faz uma grande diferença, mas você ainda descobrirá que usar e ajustar uma ferramenta O/R requer uma parte pequena, mas significativa, do trabalho.

Capítulo 12 | Frases das páginas 180-196

1. 'A primeira e mais importante razão para aplicar

Mais Livros Grátis
Escaneie para baixar
Ouça-o

o Padrão Model View Controller é garantir que os modelos estejam completamente separados da apresentação web.'

2.'Colocar o processamento em objetos separados de Script de Transação ou Modelo de Domínio tornará mais fácil testá-los também.'

3.'Se a máquina controla o fluxo da tela, você precisa de um Controlador de Aplicação; se o usuário controla, você não precisa.'

4.'O Template View permite que você escreva a apresentação na estrutura da página e insira marcadores na página para indicar onde o conteúdo dinâmico deve ser inserido.'

5.'A vantagem do Two Step View é que ela concentra em um único lugar a decisão sobre qual HTML usar.'

Mais Livros Grátis

Escaneie para baixar

Ouça-o

Escanear para baixar

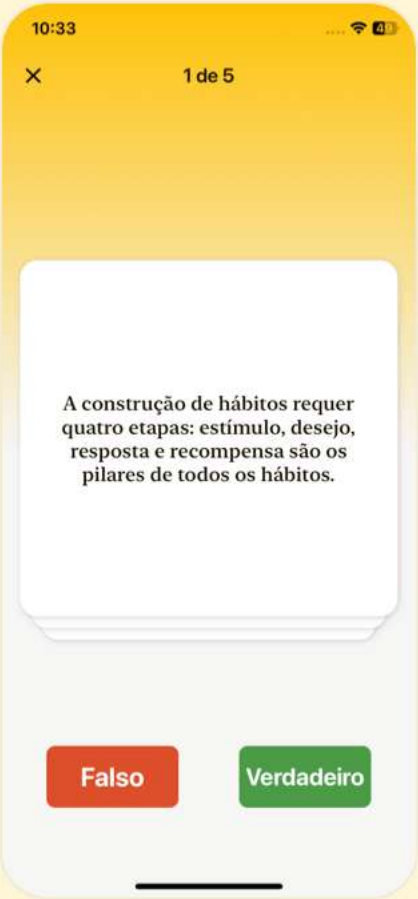
Baixe o app Bookey para desfrutar

Mais de 1000 resumos de livros com quizzes

Teste grátis disponível!

Descarregar na App Store

OBTER NO Google Play



Capítulo 13 | Frases das páginas 197-250

1. A concorrência é um dos aspectos mais complicados do desenvolvimento de software.
2. Independentemente do que você faça, sempre parece haver algo que você esqueceu.
3. O problema essencial de qualquer programação concorrente é que não é suficiente se preocupar com a correção; você também deve se preocupar com a vivacidade.
4. Se você puder tolerar os problemas, poderá evitar qualquer forma de controle de concorrência.
5. Um bom design de concorrência é, portanto, encontrar maneiras de criar tais zonas [de isolamento].
6. A principal ferramenta para lidar com a concorrência em aplicações corporativas é a transação.
7. A escolha entre controles otimistas ou pessimistas afeta toda a experiência do usuário do sistema.
8. Os bugs de concorrência são difíceis de reproduzir e muito difíceis de rastrear.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

9. Lidar com controle de concorrência que abrange transações de sistema coloca você firmemente nas águas turvas de lidar com concorrência sozinho.
10. Ao deixar o controle de concorrência nas mãos do seu software de transação, você evitará muitos problemas.

Capítulo 14 | Frases das páginas 251-267

1. A mudança de sem estado para com estado é muito mais do que três ou quatro letras no final da palavra.
2. Nem todos os dados mantidos pela sessão contam como estado de sessão.
3. Os dados da sessão precisam ser isolados. Na maioria dos casos, o que acontece em uma sessão não deve afetar o que acontece em outra.
4. A falta de estado nos permite agrupar nossos objetos, assim precisamos de menos objetos para lidar com mais usuários.
5. Precisamos decidir o que fazer com o estado.

Capítulo 15 | Frases das páginas 268-288

1. Não distribua seus objetos!

Mais Livros Grátis
Escaneie para baixar
Ouça-o

2. Venda sua avó favorita primeiro, se possível.
3. O tema preponderante... é ser econômico na distribuição de objetos.
4. Apenas os objetos que realmente precisam de um serviço remoto recebem o método de baixo nível, e é óbvio para os desenvolvedores que estão arcando com esse custo.
5. Toda chamada remota viaja no equivalente cibernético de um cavalo e uma carruagem.
6. Use serviços Web XML apenas quando uma abordagem mais direta não for possível.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Escanear para baixar

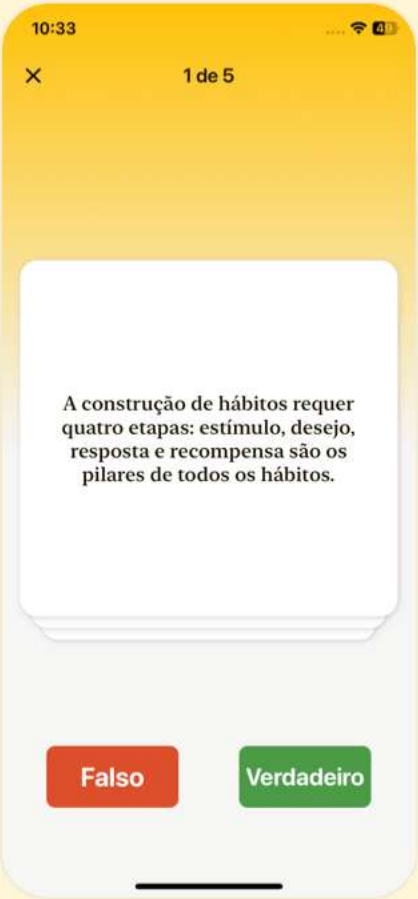
Baixe o app Bookey para desfrutar

Mais de 1000 resumos de livros com quizzes

Teste grátis disponível!

Descarregar na App Store

OBTER NO Google Play



Capítulo 16 | Frases das páginas 289-321

1....leia este capítulo no espírito em que foi apresentado. Eu não sei todas as respostas, e certamente não sei suas perguntas. Use este conselho para estimular seu pensamento, mas não o utilize como um substituto para seu raciocínio.

2.A refatoração arquitetural é difícil, e ainda ignoramos seus custos totais, mas não é impossível.

3....se você não gosta da história completa da programação extrema...deve considerar seriamente três práticas técnicas: integração contínua, desenvolvimento orientado a testes e refatoração.

4.No final, você precisa tomar e viver com as decisões por conta própria.

5.Os três principais concorrentes são Script de Transação, Módulo de Tabela e Modelo de Domínio.

Capítulo 17 | Frases das páginas 324-384

1.Como Funciona: Com o Script de Transação, a lógica de domínio é organizada principalmente

Mais Livros Grátis
Escaneie para baixar
Ouça-o

pelas transações que você realiza com o sistema.

2. A glória do Script de Transação é sua simplicidade.
3. É difícil quantificar o nível de transição, especialmente quando você está mais familiarizado com um padrão do que com o outro.
4. Um Modelo de Domínio cria uma rede de objetos interconectados, onde cada objeto representa algum indivíduo significativo, seja tão grande quanto uma corporação ou tão pequeno quanto uma única linha em um formulário de pedido.
5. O mais comum é ter vários Scripts de Transação em uma única classe, onde cada classe define uma área temática de Scripts de Transação relacionados.
6. A vantagem de usar um Modelo de Domínio rico é que ele permite que você manipule instâncias de scripts como objetos em tempo de execução.
7. A beleza dos bancos de dados orientados a objetos é que eles dão a impressão de fazer isso enquanto movem objetos entre a memória e o disco.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Capítulo 18 | Frases das páginas 385-447

1. Misturar SQL com a lógica da aplicação pode causar vários problemas.
2. Acho que usar mapas para passar dados é uma prática inadequada porque compromete a verificação em tempo de compilação e não é uma interface muito explícita.
3. O Padrão Table Data Gateway é provavelmente o padrão de interface de banco de dados mais simples de usar, pois se alinha tão bem a uma tabela ou tipo de registro do banco de dados.
4. O Table Data Gateway contém todo o SQL para acessar uma única tabela ou vista: seleções, inserções, atualizações e exclusões.
5. A principal ocasião para usar o Data Mapper é quando você deseja que o esquema do banco de dados e o modelo de objeto evoluam de forma independente.
6. Active Record é uma boa escolha para lógica de domínio que não é muito complexa, como criar, ler, atualizar e excluir.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

7. Com mapeamentos complicados, especialmente aqueles envolvendo bancos de dados existentes, isso é muito valioso.
8. Você pode modificar o Modelo de Domínio ou o banco de dados sem precisar alterar nenhum dos dois.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Escanear para baixar

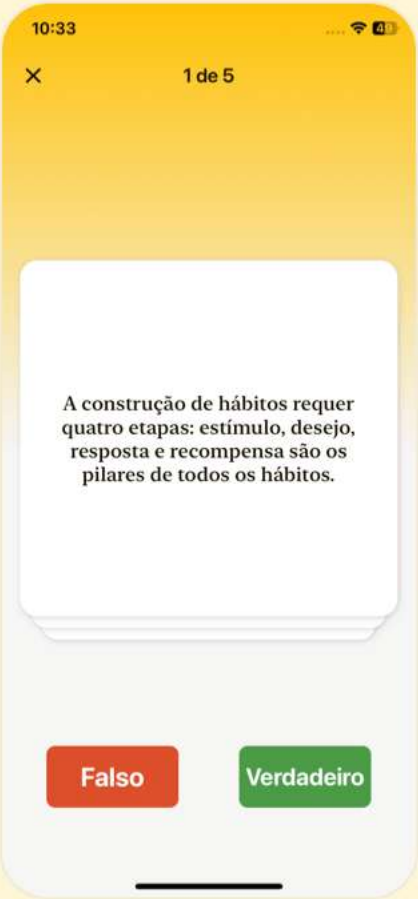
Baixe o app Bookey para desfrutar

Mais de 1000 resumos de livros com quizzes

Teste grátis disponível!

Descarregar na App Store

OBTER NO Google Play



Capítulo 19 | Frases das páginas 448-496

1. A Unidade de Trabalho mantém uma lista de objetos afetados por uma transação de negócios e coordena a gravação das mudanças e a resolução de problemas de concorrência.
2. A grande questão sobre a Unidade de Trabalho é que, quando chega a hora de confirmar, a Unidade de Trabalho decide o que fazer.
3. O Mapa de Identidade garante que cada objeto seja carregado apenas uma vez, mantendo cada objeto carregado em um mapa.
4. Carregamento Preguiçoso é um objeto que não contém todos os dados de que você precisa, mas sabe como obtê-los.
5. Decidir quando usar Carregamento Preguiçoso é tudo sobre decidir quanto você quer puxar do banco de dados ao carregar um objeto e quantas chamadas ao banco de dados isso exigirá.

Capítulo 20 | Frases das páginas 497-638

Mais Livros Grátis
Escaneie para baixar
Ouça-o

1. Quando Usar
2. Tudo o que você faz é armazenar a chave primária da tabela do banco de dados relacional nos campos do objeto.
3. Toda vez que você carrega um proprietário, você carrega os dependentes também.
4. Se você tem um campo de coleção sem ponteiro de retorno, deve considerar se o lado muitos deve ser um Mapeamento Dependente.
5. A principal diferença é que não há nada de especial sobre eles do ponto de vista puramente em memória.

Capítulo 21 | Frases das páginas 639-679

1. Muito do código que lida com mapeamento objeto-relacional descreve como os campos no banco de dados correspondem a campos em objetos na memória.
2. A geração de código é uma abordagem menos dinâmica, uma vez que quaisquer alterações no mapeamento exigem recompilação e reimplantação, pelo menos, daquela parte do software.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

3. No geral, vale a pena evoluir seu design à medida que suas necessidades crescem, pois não é difícil adicionar novas capacidades a softwares orientados a metadados.
4. O fato de que os objetos de domínio em si normalmente não são armazenados diretamente no Repositório não é exposto ao código do cliente.
5. Suas próprias medições revelarão se isso é um problema para você.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Escanear para baixar

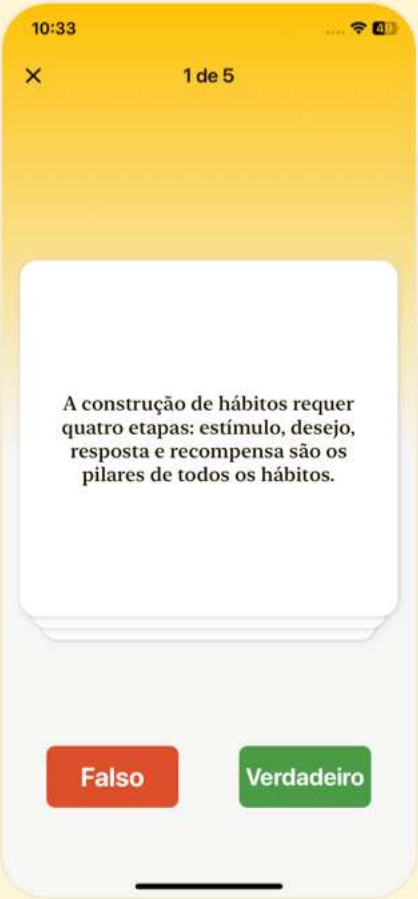
Baixe o app Bookey para desfrutar

Mais de 1000 resumos de livros com quizzes

Teste grátis disponível!

Descarregar na App Store

OBTER NO Google Play



Capítulo 22 | Frases das páginas 680-775

1. Dentre estes, a separação da apresentação do modelo é uma das heurísticas mais fundamentais do bom design de software.
2. Um ponto chave nessa separação é a direção das dependências: a apresentação depende do modelo, mas o modelo não depende da apresentação.
3. Objetos não visuais costumam ser mais fáceis de testar do que os visuais.
4. Quando você está desenvolvendo uma visão, você está pensando sobre os mecanismos da UI e como dispor uma boa interface de usuário. Quando você está trabalhando com um modelo, você está pensando sobre políticas de negócios, talvez interações com o banco de dados.
5. Separar a apresentação e o modelo permite testar toda a lógica de domínio facilmente sem recorrer a coisas como ferramentas de script GUI complicadas.
6. Para fazer isso sem criar uma dependência, você geralmente precisa de uma implementação do padrão

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Observer, como a propagação de eventos ou um listener.

Capítulo 23 | Frases das páginas 776-824

1. A Fachada Remota não possui lógica de domínio.
2. Você precisa reduzir o número de chamadas, e isso significa que você precisa transferir mais dados a cada chamada.
3. Se você precisar de acesso remoto a um modelo de objeto de granularidade fina, você ganha as vantagens de uma interface de granularidade grossa, enquanto ainda mantém a vantagem de objetos de granularidade fina, oferecendo o melhor dos dois mundos.
4. Projete uma aplicação sem distribuição e, em seguida, adicione a capacidade de distribuição por cima dela com Fachadas Remotas e Objetos de Transferência de Dados.
5. O uso mais comum deste padrão é entre uma apresentação e um Modelo de Domínio, onde os dois podem ser executados em processos diferentes.

Capítulo 24 | Frases das páginas 825-893

1. O bloqueio otimista offline resolve esse problema

Mais Livros Grátis
Escaneie para baixar
Ouça-o

ao validar que as alterações prestes a serem confirmadas por uma sessão não entram em conflito com as alterações de outra sessão.

2. A expectativa de que conflitos de sessão não são prováveis permite que vários usuários acessem os mesmos dados ao mesmo tempo.
3. Uma aplicação adequada informará quando o registro foi alterado e por quem.
4. Quanto melhor você entender seus problemas de concorrência, melhor você poderá gerenciá-los em seu código.
5. Detectar uma leitura inconsistente fica um pouco difícil quando sua transação depende dos resultados de uma consulta dinâmica.
6. Se os conflitos são prováveis, não é amigável para o usuário anunciar um apenas quando ele terminou seu trabalho e está pronto para confirmar.
7. O Bloqueio Pessimista Offline... previne conflitos ao evitá-los completamente.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

8. Compreender esses tipos de bloqueio pode certamente agregar muito valor à sua implementação do Bloqueio Pessimista Offline.
9. Não se esqueça desse passo!

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Escanear para baixar

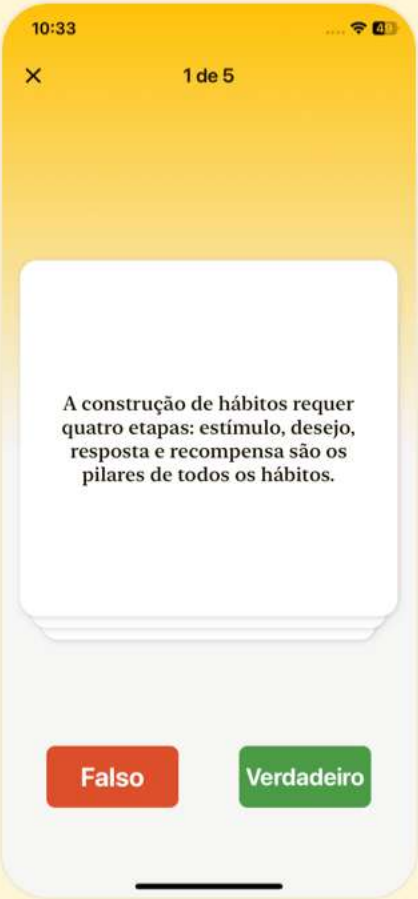
Baixe o app Bookey para desfrutar

Mais de 1000 resumos de livros com quizzes

Teste grátis disponível!

Descarregar na App Store

OBTER NO Google Play



Capítulo 25 | Frases das páginas 894-917

1. Mesmo os designs mais orientados a servidor precisam de pelo menos um pouco de Estado de Sessão do Cliente, se não apenas para manter um identificador de sessão.
2. O Objeto de Transferência de Dados pode se serializar ao longo da rede e, assim, permitir que até mesmo dados complexos sejam transmitidos.
3. O Estado de Sessão do Cliente contém uma série de vantagens. Em particular, ele reage bem no apoio a objetos de servidor sem estado com máxima agrupamento e resiliência a falhas.
4. O grande apelo do Estado de Sessão do Servidor é sua simplicidade. Em vários casos, você não precisa fazer nenhuma programação para que isso funcione.
5. O Estado de Sessão do Banco de Dados é uma alternativa para lidar com o estado da sessão; deve ser comparado com o Estado de Sessão do Servidor e o Estado de Sessão do Cliente.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Capítulo 26 | Frases das páginas 918-1001

1. Software interessante raramente vive em isolamento.
2. A resposta é tão comum que mal vale a pena ser dita. Envolva todo o código especial da API em uma classe cuja interface se pareça com um objeto regular.
3. Não tenha medo de fazer isso—stubs de serviço bem colocados podem tornar um sistema muito mais fácil de testar e, assim, muito mais fácil de escrever.
4. Gateway é uma forma simples e poderosa de variação protegida.
5. Use Interface Separada quando precisar quebrar uma dependência entre duas partes do sistema.
6. Um Registro é um objeto bem conhecido que outros objetos podem usar para encontrar objetos e serviços comuns.
7. Trate algo como um Objeto de Valor quando você estiver baseando a igualdade em algo além de uma identidade.
8. Use Caso Especial sempre que você tiver múltiplos lugares

Mais Livros Grátis
Escaneie para baixar
Ouça-o

no sistema que tenham o mesmo comportamento após uma verificação condicional para uma instância particular de classe.

9. Plugin resolve ambos os problemas ao fornecer configuração centralizada em tempo de execução.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Escanear para baixar

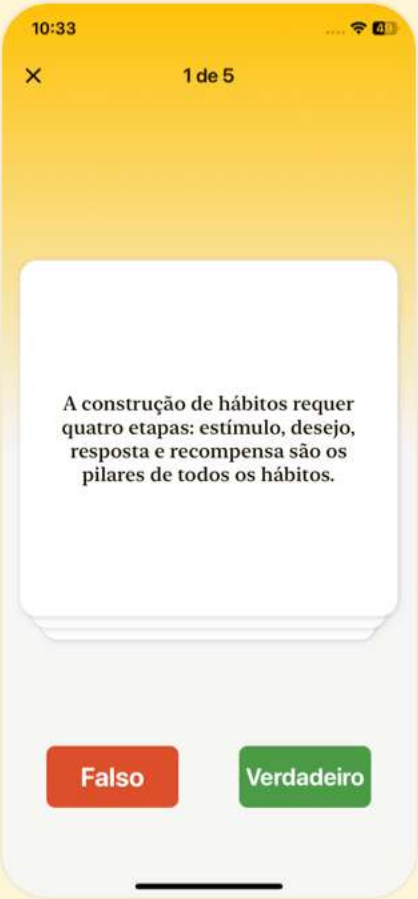
Baixe o app Bookey para desfrutar

Mais de 1000 resumos de livros com quizzes

Teste grátis disponível!

Descarregar na App Store

OBTER NO Google Play



Padrões de Arquitetura de Aplicações Corporativas Perguntas

Ver no site do Bookey

Capítulo 1 | Para Quem Este Livro É| Perguntas e respostas

1.Pergunta

Quem é o público-alvo do livro e o que espera obter com ele?

Resposta:O livro é voltado para programadores, designers e arquitetos envolvidos na construção de aplicações corporativas. Os principais objetivos dos leitores são aprimorar sua compreensão sobre questões arquitetônicas e melhorar a comunicação a respeito desses temas. Para aqueles com necessidades modestas, o livro oferece padrões fundamentais para auxiliá-los no desenvolvimento de software. Leitores mais exigentes, especialmente usuários de ferramentas, recebem insights sobre os conceitos subjacentes e orientações sobre como

Mais Livros Grátis
Escaneie para baixar
Ouça-o

escolher padrões adequados para seus projetos.

2.Pergunta

O que os leitores devem considerar antes de construir seus próprios frameworks, segundo o autor?

Resposta:Os leitores são aconselhados a avaliar cuidadosamente a necessidade de construir seus próprios frameworks, uma vez que muitos projetos sofreram ao se desviar para o desenvolvimento extensivo de frameworks, o que pode não estar alinhado com os objetivos originais do projeto. O autor sugere que contar com ferramentas existentes pode ser muitas vezes suficiente e que os leitores devem estar preparados para adaptar exemplos de código simplificados do livro para atender às demandas específicas do projeto.

3.Pergunta

O que o autor quer dizer ao afirmar que padrões são soluções comuns para problemas recorrentes?

Resposta:Padrões em arquitetura de software referem-se a abordagens estabelecidas que lidam com desafios

Mais Livros Grátis
Escaneie para baixar
Ouça-o

frequentemente encontrados no desenvolvimento. O autor enfatiza que esses padrões não são necessariamente ideias novas, mas sim soluções bem consolidadas que muitos que trabalharam no desenvolvimento corporativo podem já reconhecer. Isso reforça a noção de que aprender e usar esses padrões pode facilitar melhor comunicação e entendimento entre os desenvolvedores.

4.Pergunta

Qual é a importância de construir um vocabulário comum entre designers, segundo o autor?

Resposta:Construir um vocabulário comum é crucial para uma comunicação eficaz entre designers. O autor sugere que, quando os designers utilizam termos específicos como 'Facade Remota', isso fomenta a compreensão mútua e permite que as equipes discutam conceitos arquitetônicos de forma colaborativa, sem confusões, melhorando a eficácia do processo de design.

5.Pergunta

Como o autor pretende apoiar os leitores que são novos na arquitetura de aplicações corporativas?

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Resposta:Para os leitores que são novos na arquitetura de aplicações corporativas, o autor pretende fornecer insights sobre técnicas e padrões comumente utilizados. Esse conhecimento fundamental tem a intenção de ajudar os iniciantes a se familiarizarem com conceitos e práticas essenciais, facilitando assim seu crescimento na área.

Capítulo 2 | Agradecimentos| Perguntas e respostas

1.Pergunta

O que motivou o autor a escrever este livro?

Resposta:A motivação do autor surgiu de esforços colaborativos com muitas pessoas talentosas ao longo dos anos, cujas contribuições moldaram o material apresentado no livro.

2.Pergunta

Como a colaboração impactou o processo de escrita?

Resposta:A colaboração forneceu insights e críticas valiosos que enriqueceram o conteúdo, destacando a importância do diálogo com colegas no desenvolvimento de ideias complexas.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

3.Pergunta

Qual foi o papel das sessões de revisão na refinamento do livro?

Resposta:As sessões de revisão com colegas permitiram uma análise detalhada das ideias, resultando em insights mais profundos e uma narrativa melhor estruturada.

4.Pergunta

Por que é importante reconhecer os colaboradores em um livro?

Resposta:Reconhecer os colaboradores não apenas dá crédito, mas também enfatiza a natureza colaborativa da criação do conhecimento, mostrando que nenhum trabalho significativo é feito isoladamente.

5.Pergunta

Como o feedback dos leitores influenciou a versão final do livro?

Resposta:O feedback das versões iniciais compartilhadas online ajudou a identificar problemas, clarificar conceitos e explorar soluções alternativas, aprimorando a qualidade geral do livro.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

6.Pergunta

Quais influências pessoais moldaram a carreira do autor em consultoria e escrita?

Resposta:O autor atribui grande parte de seu crescimento em consultoria e escrita a mentores como Jim Odell, que facilitaram sua entrada nessas áreas.

7.Pergunta

O que foi único na abordagem do autor para coletar feedback durante o processo de escrita?

Resposta:O autor buscou ativamente feedback público ao disponibilizar rascunhos online, convidando percepções e desafios ao material de um público mais amplo.

8.Pergunta

De que maneira o autor expressa gratidão pelo apoio da esposa?

Resposta:O autor expressa profunda apreciação por sua esposa Cindy, reconhecendo seu apoio inabalável como um alicerce em sua vida pessoal e profissional.

9.Pergunta

O que esta seção de reconhecimento nos diz sobre a

Mais Livros Grátis
Escaneie para baixar
Ouça-o

natureza da escrita como empreendimento?
Resposta:A seção de reconhecimento reflete que a escrita é muitas vezes um esforço comunitário, onde o sucesso de um autor se baseia nas contribuições e no apoio de outros.

10.Pergunta

Como o autor comunica o valor emocional da colaboração em seus agradecimentos?
Resposta:Compartilhando anedotas específicas e contribuições de indivíduos, o autor ilustra que cada interação enriqueceu sua compreensão e levou a conexões significativas.

Capítulo 3 | Colofão| Perguntas e respostas

1.Pergunta

Quais ferramentas e tecnologias Martin Fowler usou para escrever este livro?
Resposta:Martin Fowler usou várias ferramentas e tecnologias, incluindo XML para o texto principal, TextPad para escrever documentos XML, XSLT para gerar páginas da web, Visio para diagramas e

Mais Livros Grátis
Escaneie para baixar
Ouça-o

uma variedade de ferramentas, tanto de código aberto quanto comerciais, como JUnit, NUnit, Apache FOP, Visual Studio para .NET e IntelliJ's IDEA para Java. Ele também criou scripts em XSLT e Ruby para auxiliar na formatação e importação de texto.

2.Pergunta

Como Martin Fowler abordou a publicação deste livro?

Resposta:Fowler começou a trabalhar no manuscrito em novembro de 2000 e finalizou-o em junho de 2002. Ele colaborou estreitamente com uma equipe, incluindo seu editor Mike Hendrickson da Addison Wesley, a revisora Dianne Wood e a designer Kim Arney Mulcahy, para garantir um produto final refinado. Esse processo destaca a importância do trabalho em equipe na publicação.

3.Pergunta

Qual foi a importância da Ponte Zakim durante a escrita deste livro?

Resposta:A Ponte Zakim, que estava em construção enquanto

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Fowler escrevia o livro, representa inovação e design moderno na engenharia. É notável por ser a ponte estaiada mais larga do mundo e apresenta um design assimétrico exclusivo dos EUA. Isso serve como uma metáfora para a 'ponte' de ideias e metodologias que Fowler busca alcançar ao discutir Padrões de Arquitetura de Aplicações Corporativas.

4.Pergunta

Qual papel a automação desempenhou no processo de escrita de Fowler?

Resposta:Fowler automatizou partes do seu processo de escrita desenvolvendo um programa personalizado para importar exemplos de código no texto, assim evitando a tarefa trabalhosa de inserção manual de código. Isso reflete uma abordagem moderna para a escrita e a integração da tecnologia para aumentar a produtividade e a precisão.

5.Pergunta

Qual é a importância do processo de escrita pelo qual Martin Fowler passou?

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Resposta:A jornada de escrita de Fowler mostra a evolução da escrita técnica através do uso de ferramentas variadas e esforços colaborativos, ilustrando a conexão entre tecnologia e literatura. Isso enfatiza a criatividade e a dedicação necessárias para produzir um guia abrangente sobre Padrões de Arquitetura de Aplicações Corporativas.

6.Pergunta

Como a dependência de Martin Fowler tanto de ferramentas de código aberto quanto comerciais ilustra uma filosofia em relação à tecnologia?

Resposta:Ao usar uma combinação de ferramentas de código aberto como JUnit e produtos comerciais como o Visual Studio, Fowler incorpora uma filosofia de pragmatismo, valorizando a ferramenta certa para o trabalho, independentemente de sua origem. Essa diversidade na escolha das ferramentas sugere uma mentalidade adaptável que abraça diferentes paisagens tecnológicas.

7.Pergunta

Como a imagem da capa do livro se relaciona com seu conteúdo?

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Resposta:A imagem da capa da Ponte Zakim simboliza estrutura e design, paralelamente aos temas de arquitetura de aplicações robustas discutidos no livro. Assim como a ponte é uma maravilha da engenharia, o trabalho de Fowler busca conectar aspectos teóricos e práticos de aplicações corporativas, oferecendo aos leitores uma estrutura para construir sistemas sólidos.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Ad

Escanear para baixar

Experimente o aplicativo Bookey para ler mais de 1000 resumos dos melhores livros do mundo

Desbloqueie 1000+ títulos, 80+ tópicos

Novos títulos adicionados toda semana

duct & Brand | Liderança & Colaboração | Gerenciamento de Tempo | Relacionamento & Comunica

Estratégia de Negócios | Criatividade | Memórias | Conheça a Si Mesmo | Psicologia Po

mpreendedorismo | História Mundial | Comunicação entre Pais e Filhos | Autocuidado | Mi

Visões dos melhores livros do mundo

mento pos

Os 7 Hábitos das Pessoas Altamente Eficazes

Mini Hábitos

Hábitos Atômicos

O Clube das 5 da Manhã

Como Fazer Amigos e Influenciar Pessoas

Com Não

Teste gratuito com Bookey

Capítulo 4 | Arquitetura| Perguntas e respostas

1.Pergunta

Qual é a perspectiva do autor sobre o termo 'arquitetura' no desenvolvimento de software?

Resposta:O autor vê 'arquitetura' como um termo que frequentemente possui significados variados e contraditórios dentro da indústria de software. Ele considera que é uma palavra que soa impressionante e é usada para indicar importância, ao mesmo tempo em que reconhece sua natureza subjetiva — não existe uma única definição de arquitetura, mas sim múltiplas interpretações dependendo do contexto e dos envolvidos.

2.Pergunta

Quais são os dois elementos comuns que definem a arquitetura de software segundo o autor?

Resposta:Os dois elementos comuns que definem a arquitetura de software são: 1) a decomposição de um sistema em suas partes mais altas e 2) decisões que são

Mais Livros Grátis
Escaneie para baixar
Ouça-o

percebidas como difíceis de mudar. Esses componentes contribuem para um entendimento compartilhado entre os desenvolvedores sobre o design do sistema.

3.Pergunta

Como o autor descreve a natureza subjetiva da arquitetura em projetos de software?

Resposta:O autor explica que a arquitetura é subjetiva porque representa um entendimento compartilhado entre os desenvolvedores especialistas. O que é considerado arquitetonicamente significativo pode evoluir ao longo do tempo à medida que as perspectivas dos desenvolvedores mudam, especialmente se eles descobrirem que certos aspectos do design são mais fáceis de modificar do que anteriormente pensado.

4.Pergunta

Qual padrão arquitetônico o autor prefere e por quê?

Resposta:O autor prefere o padrão de arquitetura em camadas porque é amplamente útil para decompor aplicações corporativas em camadas gerenciáveis. Ele pretende focar

Mais Livros Grátis
Escaneie para baixar
Ouça-o

neste modelo ao longo do livro, já que é comumente utilizado em aplicações não triviais.

5.Pergunta

Qual é a importância das decisões difíceis de mudar no design arquitetônico?

Resposta:No design arquitetônico, as decisões difíceis de mudar são significativas porque são percebidas pelos desenvolvedores como críticas nas etapas iniciais do desenvolvimento do projeto. Acertar essas decisões é crucial, pois elas podem ter efeitos duradouros na arquitetura geral do sistema.

6.Pergunta

Por que o autor não separa estritamente as decisões arquitetônicas das decisões de design?

Resposta:O autor não faz uma distinção rigorosa entre decisões arquitetônicas e decisões de design porque acredita que a classificação do que é arquitetônico ou não é altamente subjetiva. Muitas decisões podem influenciar tanto a arquitetura quanto o design, tornando desafiador

Mais Livros Grátis
Escaneie para baixar
Ouça-o

categorizá-las estritamente.

7.Pergunta

Como o autor relaciona as camadas na arquitetura às aplicações corporativas?

Resposta:O autor relaciona as camadas na arquitetura às aplicações corporativas ao enfatizar que a maioria dessas aplicações utiliza arquitetura em camadas para organizar os componentes logicamente. Ele discute como essas camadas interagem e a importância de entender seus papéis na construção de sistemas corporativos eficazes.

8.Pergunta

O que o autor sugere sobre a natureza da arquitetura ao longo do tempo?

Resposta:O autor sugere que a arquitetura não é estática; ela muda ao longo do tempo à medida que os sistemas evoluem e os desenvolvedores ganham novos insights. À medida que os projetos amadurecem, a visão do que é considerado arquitetonicamente significativo pode mudar, destacando a natureza fluida e adaptativa da arquitetura de software.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Capítulo 5 | Aplicações Corporativas| Perguntas e respostas

1.Pergunta

O que distingue as aplicações empresariais de outros tipos de software?

Resposta:As aplicações empresariais são caracterizadas pela necessidade de dados persistentes, regras de negócios complexas e a exigência de integração com vários outros sistemas na empresa. Ao contrário das aplicações típicas, elas lidam com grandes volumes de dados e frequentemente envolvem interfaces de usuário complexas e vários tipos de interação do usuário.

2.Pergunta

Por que gerenciar dados em aplicações empresariais é particularmente desafiador?

Resposta:As aplicações empresariais frequentemente gerenciam vastas quantidades de dados persistentes que precisam persistir através de múltuas atualizações da aplicação e mudanças de hardware e software. A gestão de

Mais Livros Grátis
Escaneie para baixar
Ouça-o

dados pode se tornar complicada devido à necessidade de migração de dados, mudanças de estruturas e a garantia da integridade dos dados quando acessados simultaneamente por muitos usuários.

3.Pergunta

Que tipos de problemas as aplicações empresariais enfrentam em relação à lógica de negócios?

Resposta:A lógica de negócios em aplicações empresariais pode ser inconsistente e ilógica, levando a desafios na implementação. As regras podem evoluir devido a negociações ou mudanças nos processos de negócios, tornando difícil estabelecer uma estrutura lógica estável. Os desenvolvedores precisam acomodar essas complexidades enquanto garantem uma organização eficaz e flexibilidade na aplicação das regras de negócios.

4.Pergunta

Como as demandas das aplicações empresariais afetam as interfaces de usuário?

Resposta:Dada a complexidade e variabilidade dos dados nas

Mais Livros Grátis
Escaneie para baixar
Ouça-o

aplicações empresariais, a interface de usuário deve atender a uma ampla gama de usuários, desde ocasionais a frequentes, muitas vezes com mínima expertise técnica. Isso resulta na necessidade de várias telas distintas para apresentar os dados de forma eficaz para diferentes propósitos.

5.Pergunta

Qual é o papel da integração na arquitetura das aplicações empresariais?

Resposta:A integração é crucial para as aplicações empresariais, pois elas precisam colaborar com vários outros sistemas, frequentemente construídos em diferentes tecnologias em momentos diferentes. O processo de integração é complexo devido às diferenças nos processos de negócios e na representação de dados entre os sistemas, tornando-se um desafio significativo para os desenvolvedores.

6.Pergunta

Como pequenas aplicações empresariais podem fornecer um valor significativo?

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Resposta:Mesmo pequenas aplicações empresariais podem ter um grande impacto em uma organização. Embora possam não chamar a atenção como projetos maiores, seus efeitos coletivos podem levar a melhorias substanciais em eficiência e eficácia dentro da empresa, especialmente porque muitas vezes atendem a necessidades específicas que poderiam ser negligenciadas em sistemas maiores.

7.Pergunta

Por que é importante simplificar grandes projetos em aplicações empresariais?

Resposta:Simplificar a arquitetura e os processos de grandes projetos pode transformá-los em projetos menores e mais gerenciáveis, o que pode levar a uma melhoria na eficiência e adaptabilidade. Esta abordagem muitas vezes permite uma melhor implementação de recursos e ajustes mais fáceis às necessidades de negócios em mudança.

Capítulo 6 | Tipos de Aplicações Corporativas| Perguntas e respostas

1.Pergunta

Quais são os fatores-chave que devem guiar o design de

Mais Livros Grátis
Escaneie para baixar
Ouça-o

uma aplicação corporativa?

Resposta:O design bem-sucedido de aplicações corporativas depende do reconhecimento dos problemas únicos que cada aplicação aborda e da compreensão dos trade-offs entre alternativas de design. As complexidades de vários domínios ditam que não pode haver uma arquitetura de 'tamanho único', destacando a importância de soluções personalizadas.

2.Pergunta

Como os diferentes tipos de aplicações afetam as decisões arquitetônicas?

Resposta:A arquitetura de uma aplicação deve refletir seu caso de uso específico, volume de usuários e complexidade. Por exemplo, um varejista online B2C precisa de uma interface escalável e amigável para lidar com alto tráfego, enquanto um sistema de automação de leasing requer lógica de negócios intrincada e robusta manipulação de transações devido às suas regras complexas.

Mais Livros Grátis

Escaneie para baixar

Ouça-o

3.Pergunta

Por que é importante entender a lógica de negócios por trás das aplicações corporativas?

Resposta:A lógica de negócios impulsiona a funcionalidade de uma aplicação e encapsula suas regras e processos. Uma compreensão aprofundada das especificidades do domínio de negócios garante que a aplicação atenda efetivamente às necessidades dos usuários e se adapte à pressão competitiva, como demonstrado pelas variações no processamento de leasing em comparação ao e-commerce.

4.Pergunta

Quais desafios surgem ao construir aplicações simples que podem se tornar complexas mais tarde?

Resposta:Um sistema simples de rastreamento de despesas pode parecer direto inicialmente; no entanto, se não for arquitetado com o crescimento futuro em mente, pode se tornar difícil de adaptar. Equilibrar entre os requisitos imediatos e a flexibilidade futura é crucial para evitar atrasos na implementação e na realização dos benefícios comerciais.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

5.Pergunta

O que os desenvolvedores devem ter em mente em relação às ferramentas para desenvolvimento de aplicações corporativas?

Resposta:Selecionar as ferramentas certas deve estar alinhado com o propósito específico da aplicação. Os desenvolvedores devem ter cuidado para não impor ferramentas que não estão alinhadas com as necessidades do projeto, pois isso pode levar a ineficiências, reforçando a necessidade de uma seleção cuidadosa adaptada a contextos específicos de aplicação.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Ad

Escanear para baixar

Por que o Bookey é um aplicativo indispensável para amantes de livros

Conteúdo de 30min
Quanto mais profunda e clara for a interpretação que fornecemos, melhor será sua compreensão de cada título.

Clipes de Ideias de 3min
Impulsione seu progresso.

Questionário
Verifique se você dominou o que acabou de aprender.

E mais
Várias fontes, Caminhos em andamento, Coleções...

Teste gratuito com Bookey

Capítulo 7 | Pensando em Performance| Perguntas e respostas

1.Pergunta

Qual é a abordagem recomendada para lidar com preocupações de desempenho em aplicações corporativas?

Resposta:A melhor abordagem é primeiro colocar o sistema em funcionamento, depois instrumentá-lo para medições e seguir um processo de otimização disciplinado com base nessas medições. Decisões arquitetônicas iniciais podem ter implicações de desempenho a longo prazo, por isso é crucial medir o desempenho na sua própria configuração, em vez de confiar apenas em conselhos teóricos.

2.Pergunta

Por que é importante medir tanto antes quanto depois de realizar otimizações de desempenho?

Resposta:Medir antes e depois é vital, pois garante que as otimizações realmente melhorem o desempenho sem tornar o código mais difícil de entender ou manter. Não medir pode

Mais Livros Grátis
Escaneie para baixar
Ouça-o

levar a suposições falsas sobre ganhos de desempenho e pode piorar a qualidade do código.

3.Pergunta

O que é escalabilidade no contexto de métricas de desempenho, e por que isso é significativo?

Resposta:Escalabilidade é a capacidade de um sistema de manter o desempenho à medida que a carga aumenta, geralmente adicionando hardware. É significativo porque um sistema escalável permite a expansão de recursos (como servidores adicionais) para melhorar o desempenho, o que muitas vezes é mais fácil e barato do que fazer otimizações de software.

4.Pergunta

Como diferentes fatores como tempo de resposta, capacidade de resposta e latência afetam a experiência do usuário?

Resposta:O tempo de resposta é o tempo necessário para processar uma solicitação, enquanto a capacidade de resposta é quão rapidamente o sistema reconhece uma solicitação. Uma alta capacidade de resposta pode aliviar a frustração do

Mais Livros Grátis
Escaneie para baixar
Ouça-o

usuário, mesmo que os tempos de resposta não sejam ideais. A latência, o atraso causado pela distância (especialmente em sistemas remotos), é crucial, pois pode impactar significativamente a experiência do usuário ao atrasar o desempenho percebido.

5.Pergunta

Quais são as implicações da sensibilidade à carga de um sistema?

Resposta:A sensibilidade à carga refere-se a como o tempo de resposta de um sistema varia com o aumento de usuários. Um sistema com menor sensibilidade à carga é preferido, pois sugere que seu desempenho permanece estável mesmo com a carga aumentada, indicando melhor escalabilidade e experiência do usuário.

6.Pergunta

O que os desenvolvedores devem considerar ao otimizar para desempenho, dado que as configurações podem mudar ao longo do tempo?

Resposta:Os desenvolvedores devem reconhecer que mudanças nas configurações (como upgrades de hardware ou

Mais Livros Grátis
Escaneie para baixar
Ouça-o

migrações de sistema) podem invalidar medições de desempenho anteriores. Portanto, é essencial reavaliar e potencialmente refazer otimizações sempre que houver uma mudança significativa para garantir benefícios de desempenho contínuos.

7.Pergunta

Por que pode ser preferível construir para escalabilidade de hardware em vez de otimizar para capacidade ou eficiência?

Resposta:Construir para escalabilidade de hardware é frequentemente mais rentável, pois oferece a flexibilidade de aumentar o desempenho adicionando recursos, em vez de complicar o software para extrair capacidade extra. Em muitos casos, adquirir hardware adicional é mais barato e simples do que o custo de extensas otimizações de software.

8.Pergunta

Qual é a relação entre eficiência e desempenho?

Resposta:Eficiência é definida como a relação entre desempenho e os recursos utilizados. Um sistema que atinge

Mais Livros Grátis
Escaneie para baixar
Ouça-o

uma certa taxa de transferência com menos recursos é considerado mais eficiente. Compreender essa relação ajuda a guiar decisões sobre os compromissos entre os recursos alocados e os resultados de desempenho esperados.

9.Pergunta

Como as decisões arquitetônicas em sistemas corporativos impactam diretamente as métricas de desempenho?

Resposta:As decisões arquitetônicas podem influenciar diferentes fatores de desempenho de maneiras distintas, como tempo de resposta, taxa de transferência e capacidades de gerenciamento de carga. Os projetistas devem considerar cuidadosamente esses impactos, já que escolhas que melhoram o desempenho em uma área podem afetar negativamente outras, sublinhando a importância de avaliações de desempenho holísticas.

Capítulo 8 | Padrões| Perguntas e respostas

1.Pergunta

O que define um padrão segundo Christopher Alexander, e qual é a sua relevância no design de software?

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Resposta:Segundo Christopher Alexander, "Cada padrão descreve um problema que ocorre repetidamente em nosso ambiente e, em seguida, descreve a essência da solução para esse problema de tal forma que você pode usar essa solução milhões de vezes, sem nunca fazê-la da mesma maneira duas vezes." Essa definição é relevante no design de software, pois enfatiza que os padrões podem oferecer soluções para questões comuns, permitindo que os desenvolvedores apliquem estratégias eficazes em vários contextos.

2.Pergunta

Como o processo de descoberta de padrões difere de meramente inventar novas ideias?

Resposta:Descobrir padrões envolve observar práticas existentes e identificar soluções comuns, enquanto inventar implica criar conceitos totalmente novos. Autores de padrões, como Martin Fowler, não afirmam inventar padrões; eles descobrem e documentam soluções que já se mostraram

Mais Livros Grátis
Escaneie para baixar
Ouça-o

eficazes no campo.

3.Pergunta

Por que a nomeação é importante no contexto dos padrões, e como isso melhora a comunicação entre os designers?

Resposta:A nomeação é crucial porque ajuda a estabelecer um vocabulário comum entre os designers, o que facilita uma comunicação mais clara. Por exemplo, se um desenvolvedor se refere a um design como 'Facade Remota', os colegas familiarizados com o termo entendem instantaneamente a arquitetura pretendida, possibilitando discussões eficientes e trabalho colaborativo.

4.Pergunta

Como alguém deve abordar o uso de padrões segundo a perspectiva de Fowler, especialmente em relação à adaptação?

Resposta:Fowler enfatiza que os padrões não podem ser aplicados de maneira cega. Em vez disso, eles devem ser vistos como pontos de partida que requerem adaptação para se ajustarem às circunstâncias específicas de um projeto. Isso

Mais Livros Grátis
Escaneie para baixar
Ouça-o

envolve ajustar as soluções para atender às necessidades particulares do projeto, o que é comparado a finalizar ideias 'meio-boleadas' no seu próprio contexto.

5.Pergunta

Qual é o papel dos exemplos na compreensão de padrões, e o que se deve ter em mente ao revisá-los?

Resposta:Os exemplos ilustram como os padrões podem ser implementados, mas não devem ser confundidos com o padrão em si. Eles são representações simplificadas destinadas a capturar a mensagem central do padrão. Os leitores devem lembrar que as implementações no mundo real vão variar e que eles precisam adaptar os padrões em vez de copiar exemplos diretamente.

6.Pergunta

Qual limitação Fowler reconhece sobre a coleção de padrões apresentada no livro?

Resposta:Fowler reconhece que a coleção de padrões não é abrangente nem um guia exaustivo para o desenvolvimento de aplicações corporativas. Ele destaca que os padrões

Mais Livros Grátis
Escaneie para baixar
Ouça-o

refletem sua compreensão e experiências atuais, e que o desenvolvimento de software está sempre evoluindo. Os padrões são vistos como pontos de partida, não como soluções definitivas.

7.Pergunta

Como os padrões contribuem para o conjunto de ferramentas de um arquiteto ou desenvolvedor?

Resposta:Os padrões servem como referência que os desenvolvedores podem consultar para resolver problemas recorrentes de forma eficaz. Eles encapsulam sabedoria comum e estratégias comprovadas que os designers podem aplicar e adaptar, facilitando assim melhores práticas de design e auxiliando na comunicação de ideias arquitetônicas.

Capítulo 9 | Camadas| Perguntas e respostas

1.Pergunta

Quais são os principais benefícios de usar camadas na arquitetura de software?

Resposta:A utilização de camadas ajuda a dividir sistemas complexos em subsistemas compreensíveis,

Mais Livros Grátis
Escaneie para baixar
Ouça-o

permitindo que os desenvolvedores foquem em uma camada sem precisar conhecer as outras. Isso minimiza dependências, facilita a padronização e aprimora a manutenibilidade. Por exemplo, um serviço FTP pode ser construído sobre diferentes camadas de transporte, como TCP ou UDP, sem modificação.

2.Pergunta

Como as camadas afetam o desempenho em aplicações de software?

Resposta:Embora a adição de camadas possa introduzir sobrecarga de desempenho devido a múltiplas transformações entre camadas, camadas bem projetadas também podem levar a otimizações de desempenho. Por exemplo, uma camada de controle de transações pode ser otimizada para velocidade, beneficiando toda a aplicação.

3.Pergunta

Quais desafios surgem com o uso de arquiteturas em camadas?

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Resposta:Um desafio é lidar com mudanças em cascata, onde modificações em uma camada podem exigir mudanças em várias camadas. Camadas extras também podem degradar o desempenho se não forem gerenciadas adequadamente. Além disso, escolher as camadas apropriadas e suas responsabilidades pode ser complexo.

4.Pergunta

Como a evolução da arquitetura de software influenciou a adoção de camadas?

Resposta:Inicialmente, os sistemas eram simples e não utilizavam camadas. Com o surgimento dos sistemas cliente-servidor nos anos 90, surgiram as arquiteturas de duas camadas. No entanto, à medida que a complexidade cresceu, especialmente com aplicações web, a arquitetura de três camadas (apresentação, domínio, fonte de dados) ganhou popularidade pela melhor separação de preocupações.

5.Pergunta

Qual a diferença entre uma camada e um nível na arquitetura de software?

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Resposta:Uma camada refere-se à organização lógica do código e das responsabilidades dentro de uma aplicação, enquanto um nível implica uma separação física, como executar diferentes camadas em máquinas separadas. Por exemplo, uma aplicação de três camadas pode executar todas as camadas em uma única máquina, mas ainda ter três camadas lógicas.

6.Pergunta

Ao separar as camadas da aplicação, qual é a prática recomendada em relação às dependências?

Resposta:As camadas de domínio e fonte de dados não devem depender da camada de apresentação para garantir que alterações na interface do usuário não impactem a lógica de negócios ou o acesso a dados. Essa estrutura promove uma refatoração mais fácil e a substituição de componentes sem perturbar a arquitetura geral.

7.Pergunta

Quais métodos podem ser empregados para gerenciar efetivamente a complexidade em aplicações em camadas?

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Resposta:Utilizar uma clara separação de preocupações, encapsular a lógica de domínio longe da apresentação e garantir que cada camada adira a responsabilidades únicas. À medida que a complexidade aumenta, considere dividir as camadas em módulos ou pacotes menores, promovendo modularidade enquanto mantém a separação.

8.Pergunta

Por que é crucial entender a diferença entre lógica de domínio e outros tipos de lógica?

Resposta:Identificar erroneamente a lógica de domínio pode levar à duplicação de código e desafios de manutenção. Um teste útil é determinar se uma funcionalidade exigiria duplicação ao adaptar a aplicação a uma interface diferente, o que sinaliza a infiltração da lógica de domínio na apresentação.

9.Pergunta

Quais considerações devem ser levadas em conta ao determinar onde executar as camadas da aplicação?

Resposta:A escolha entre execução no lado do cliente e no

Mais Livros Grátis
Escaneie para baixar
Ouça-o

lado do servidor depende de fatores como experiência do usuário (responsividade) e eficiência de implantação. Por exemplo, operações no lado do servidor simplificam a manutenção, mas podem introduzir latência nas interações do usuário, enquanto operações no lado do cliente podem melhorar a responsividade às custas de uma manutenção complexa.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Ad

Escanear para baixar

App Store
Escolha dos Editores

22k avaliações de 5 estrelas

Feedback Positivo

Afonso Silva

s cada resumo de livro não só
o, mas também tornam o
m divertido e envolvente. O
tou a leitura para mim.

Fantástico! Brígida Santos

Estou maravilhado com a variedade de livros e idiomas que o Bookey suporta. Não é apenas um aplicativo, é um portal para o conhecimento global. Além disso, ganhar pontos para caridade é um grande bônus!

Fi

O
só
o
O

na Oliveira

correr as
ém me dá
omprar a
ar!

Adoro! Duarte Costa

Usar o Bookey ajudou-me a cultivar um hábito de leitura sem sobrecarregar minha agenda. O design do aplicativo e suas funcionalidades são amigáveis, tornando o crescimento intelectual acessível a todos.

Economiza tempo!

O Bookey é o meu apli
crescimento intelectua
perspicazes e lindame
um mundo de conheci

Aplicativo incrível! Estevão Pereira

Eu amo audiolivros, mas nem sempre tenho tempo para ouvir o livro inteiro! O Bookey permite-me obter um resumo dos destaques do livro que me interessa!!! Que ótimo conceito!!! Altamente recomendado!

Aplicativo lindo

Este aplicativo é um salva-vidas para
de livros com agendas lotadas. Os re
precisos, e os mapas mentais ajudar
o que aprendi. Altamente recomend

Teste gratuito com Bookey

Capítulo 10 | Organizando a Lógica de Domínio| Perguntas e respostas

1.Pergunta

Quais são os três padrões principais para organizar a lógica de domínio segundo Martin Fowler?

Resposta:Os três padrões principais são Transaction Script, Domain Model e Table Module.

2.Pergunta

O que é um Transaction Script e quais são suas vantagens?

Resposta:Um Transaction Script é um procedimento simples que lida com ações do usuário processando entradas, aplicando validações, realizando cálculos, interagindo com bancos de dados e retornando dados. As vantagens incluem sua simplicidade, facilidade de entendimento pelos desenvolvedores e clareza na definição de limites de transação.

3.Pergunta

Quais são as desvantagens de usar um Transaction Script?

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Resposta:As desvantagens incluem código duplicado, já que tarefas semelhantes precisam ser repetidas em múltiplos scripts, tornando a aplicação complexa e potencialmente confusa sem uma estrutura clara.

4.Pergunta

Como um Domain Model difere de um Transaction Script?

Resposta:Um Domain Model organiza a lógica em torno dos substantivos do domínio, com cada objeto encapsulando o comportamento relevante, permitindo uma abordagem mais estruturada e gerenciável da lógica de domínio complexa, ao contrário de um Transaction Script que centraliza toda a lógica em uma única rotina.

5.Pergunta

Que desafios os desenvolvedores podem enfrentar ao passar a usar um Domain Model?

Resposta:Os desenvolvedores podem enfrentar frustrações devido à complexidade de navegar entre objetos e encontrar onde comportamentos específicos residem. Além disso,

Mais Livros Grátis
Escaneie para baixar
Ouça-o

entender como trabalhar com um Domain Model rico pode levar tempo e experiência.

6.Pergunta

Qual é o propósito de uma Service Layer na arquitetura de lógica de domínio?

Resposta:Uma Service Layer atua como uma API para a aplicação, facilitando a interação entre a lógica de apresentação e o modelo de domínio subjacente, e pode encapsular controle de transação e segurança.

7.Pergunta

Quais fatores influenciam a escolha entre usar Transaction Script, Domain Model ou Table Module?

Resposta:A escolha depende em grande parte da complexidade da lógica de domínio. Lógicas mais simples podem favorecer Transaction Script ou Table Module, enquanto lógicas mais complexas frequentemente necessitam de um Domain Model.

8.Pergunta

Como deve-se abordar a decisão de qual padrão de lógica de domínio adotar?

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Resposta:A decisão deve envolver uma análise da complexidade da aplicação e consideração da familiaridade da equipe de desenvolvimento com os diferentes padrões. Também é importante ser flexível e estar disposto a refatorar se as decisões iniciais se provarem inadequadas.

9.Pergunta

O que é um Table Module e como ele difere de um Domain Model?

Resposta:Um Table Module organiza a lógica de domínio em torno de tabelas (Conjuntos de Registros) em vez de objetos, mantendo uma única instância para múltiplos registros, enquanto um Domain Model possui uma instância por registro, oferecendo mais granularidade e encapsulamento de comportamento.

10.Pergunta

Em quais cenários você poderia combinar o uso de Transaction Script, Domain Model e Table Module?

Resposta:É comum usar Transaction Script para tarefas mais simples e um Domain Model ou Table Module para

Mais Livros Grátis
Escaneie para baixar
Ouça-o

funcionalidades mais complexas dentro da mesma aplicação. A flexibilidade é fundamental para gerenciar adequadamente a lógica de domínio.

Capítulo 11 | Mapeamento para Bancos de Dados Relacionais| Perguntas e respostas

1.Pergunta

Qual é o papel principal da camada de fonte de dados na arquitetura de aplicativos?

Resposta:O papel principal da camada de fonte de dados é gerenciar a comunicação com os vários componentes de infraestrutura que um aplicativo requer para funcionar, particularmente com bancos de dados relacionais, que são o foco principal de a maioria dos aplicativos contemporâneos.

2.Pergunta

Por que o SQL é considerado um fator dominante no sucesso dos bancos de dados relacionais?

Resposta:O SQL é prevalente porque oferece uma linguagem quase padrão para comunicação com bancos de dados, permitindo que os desenvolvedores realizem consultas e

Mais Livros Grátis
Escaneie para baixar
Ouça-o

comandos de forma eficaz, apesar de alguns desafios com as melhorias específicas de fornecedores.

3.Pergunta

Quais são os padrões arquitetônicos no contexto de Padrões de Arquitetura de Aplicações Corporativas?

Resposta:Os padrões arquitetônicos ditam como a lógica de domínio interage com a camada de banco de dados. Eles são escolhas de design cruciais que impactam significativamente a arquitetura do aplicativo e não são facilmente alteráveis posteriormente.

4.Pergunta

Quais são os dois principais padrões de Gateway descritos e como eles diferem?

Resposta:Os dois principais padrões de Gateway são o Row Data Gateway e o Table Data Gateway. O Row Data Gateway tem uma instância por linha retornada por uma consulta, alinhando-se bem à programação orientada a objetos. Em contraste, o Table Data Gateway tem uma instância por tabela e opera de maneira mais genérica,

Mais Livros Grátis
Escaneie para baixar
Ouça-o

retornando frequentemente um conjunto de registros.

5.Pergunta

O que é o padrão Active Record e quando é benéfico usá-lo?

Resposta:Active Record é um padrão onde os objetos de domínio são responsáveis por sua própria persistência e interações com o banco de dados. É benéfico quando há uma correspondência simples entre classes de domínio e tabelas de banco de dados, sem muita complexidade na lógica de domínio.

6.Pergunta

Quais desafios surgem quando a lógica de domínio se torna mais complexa e como isso afeta os mecanismos de persistência?

Resposta:À medida que a lógica de domínio se torna mais intrincada, combinar Active Record com um Modelo de Domínio rico pode levar a problemas de manutenibilidade e testes. Nesses casos, usar Data Mapper é aconselhável, pois permite uma melhor separação entre a lógica de domínio e a estrutura do banco de dados.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

7.Pergunta

O que é o padrão Unit of Work e qual problema ele resolve?

Resposta:O padrão Unit of Work rastreia todas as mudanças em objetos de domínio durante uma transação e assegura que as atualizações ao banco de dados sejam executadas corretamente. Esse padrão simplifica a gestão de múltiplas operações e ajuda a manter a consistência dos dados.

8.Pergunta

Como a otimização Lazy Load pode melhorar a interação com o banco de dados?

Resposta:A otimização Lazy Load permite adiar a recuperação de objetos relacionados até que sejam especificamente solicitados, minimizando a carga imediata sobre o banco de dados e melhorando o desempenho ao reduzir buscas de dados desnecessárias.

9.Pergunta

Qual é a importância do padrão Identity Map no contexto das interações com bancos de dados?

Resposta:O padrão Identity Map garante que cada linha de

Mais Livros Grátis
Escaneie para baixar
Ouça-o

dados seja representada por um único objeto na memória para evitar duplicações e inconsistências durante atualizações. Ele atua como um cache para gerenciar as identidades dos objetos recuperados do banco de dados.

10.Pergunta

Quais são os prós e contras de usar a Herança em uma Única Tabela no design de banco de dados?

Resposta:A Herança em uma Única Tabela simplifica a gestão de dados ao consolidar todos os dados hierárquicos em uma tabela, tornando mais fácil gerenciar e modificar. No entanto, pode levar a desperdícios de espaço com muitas colunas nulas e pode se tornar um gargalo para o acesso a dados.

11.Pergunta

Por que é crítico perfilar um aplicativo com seu ambiente de banco de dados específico?

Resposta:O perfilamento é essencial porque as regras gerais de otimização de desempenho podem variar significativamente com base no sistema de banco de dados

Mais Livros Grátis
Escaneie para baixar
Ouça-o

específico e nas consultas que estão sendo executadas. As percepções contextuais fornecidas a partir de métricas de desempenho reais ajudam os desenvolvedores a tomar decisões informadas.

12.Pergunta

Que vantagem a utilização do mapeamento de metadados oferece ao lidar com interações em bancos de dados?

Resposta:O mapeamento de metadados permite a definição de como os campos do banco de dados correspondem aos atributos dos objetos, o que pode simplificar tarefas de codificação repetitivas e permitir maior flexibilidade por meio de geração de código ou programação reflexiva.

13.Pergunta

Como um desenvolvedor pode garantir uma gestão adequada de conexões nas interações com o banco de dados?

Resposta:Para gerenciar conexões de forma eficiente, os desenvolvedores podem usar pooling de conexões para reduzir sobrecarga, amarrar conexões a transações para controle de escopo e usar comandos explícitos de

Mais Livros Grátis
Escaneie para baixar
Ouça-o

abertura/fechamento para evitar vazamentos de recursos.

14.Pergunta

Como diferentes tecnologias, como ferramentas de Mapeamento Objeto-Relacional (ORM), impactam o design de aplicativos relacionados a bancos de dados?

Resposta:As ferramentas ORM abstraem interações com o banco de dados, permitindo que os desenvolvedores trabalhem com objetos em vez de SQL bruto. Elas fornecem mapeamentos que podem reduzir significativamente as complexidades do manuseio de dados, embora tragam suas considerações de ajuste de desempenho.

15.Pergunta

Quais são algumas estratégias para gerenciar a persistência de objetos em um Modelo de Domínio rico?

Resposta:Ao gerenciar a persistência em um Modelo de Domínio rico, empregar estratégias como Data Mapper para lidar com a complexidade, utilizar Unit of Work para transações e Lazy Loading para entidades relacionadas ajuda a manter claras as fronteiras entre o acesso a dados e a lógica de domínio.

Mais Livros Grátis

Escaneie para baixar

Ouça-o

Capítulo 12 | Apresentação Web| Perguntas e respostas

1.Pergunta

Quais são as vantagens que interfaces de usuário baseadas em navegador trazem para aplicações corporativas?

Resposta:Interfaces de usuário baseadas em navegador oferecem vantagens como a eliminação da necessidade de instalação de software cliente, fornecendo uma abordagem comum de UI e permitindo fácil acesso universal.

2.Pergunta

Como um servidor Web decide como lidar com solicitações recebidas?

Resposta:Um servidor Web interpreta a URL de uma solicitação consultando seu arquivo de configuração, que vincula URLs específicas a programas ou scripts do servidor correspondentes projetados para tratá-las.

3.Pergunta

Quais são as duas principais formas de estruturação de

Mais Livros Grátis
Escaneie para baixar
Ouça-o

programas em um servidor Web e como elas diferem?
Resposta:As duas principais formas são scripts e páginas do servidor. Scripts (como scripts CGI ou servlets Java) lidam com chamadas HTTP através de funções ou métodos definidos, enquanto as páginas do servidor são estruturadas principalmente em torno do conteúdo HTML e incluem código embutido para execução dinâmica.

4.Pergunta

O que é o padrão Model-View-Controller (MVC) e sua importância em aplicações Web?
Resposta:O padrão MVC separa os modelos da aplicação (lógica de domínio) da apresentação (UI), facilitando modificações mais simples na UI e permitindo a integração de múltiplas apresentações posteriormente.

5.Pergunta

Por que a separação de modelos e visões é crucial no design de aplicações Web?
Resposta:Separar modelos de visões garante que mudanças na lógica de apresentação não afetem a lógica de domínio

Mais Livros Grátis
Escaneie para baixar
Ouça-o

subjacente, permitindo uma manutenção e teste mais fáceis de ambas as camadas.

6.Pergunta

Qual é o papel de um Controlador de Aplicação em aplicações Web?

Resposta:Um Controlador de Aplicação gerencia o fluxo de uma aplicação, controlando quais telas de interface do usuário aparecem em qual ordem, facilitando assim uma melhor navegação e uma experiência de usuário estruturada.

7.Pergunta

Quais são as vantagens e desvantagens de usar Template View em comparação com Transform View?

Resposta:Template View permite embutir lógica de programação diretamente no HTML, fornecendo flexibilidade, mas arriscando um código bagunçado. Transform View simplifica a apresentação ao transformar dados, tornando-se ideal para XML, mas menos intuitiva para necessidades complexas de UI.

8.Pergunta

Quais são os benefícios de usar uma Visualização em

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Duas Etapas no design de aplicações?
Resposta:Uma Visualização em Duas Etapas separa a criação lógica da tela da renderização HTML, facilitando a implementação de alterações globais nas estruturas HTML e acomodando variações nas exibições do front-end.

9.Pergunta

Qual é a diferença entre um Controlador de Página e um Front Controller?
Resposta:Um Controlador de Página gerencia a entrada e saída para páginas específicas, enquanto um Front Controller centraliza o tratamento de todas as solicitações em um único objeto, permitindo um processo de tratamento de solicitações mais simplificado e coeso.

10.Pergunta

O que caracteriza melhor o fluxo de tomada de decisão em sistemas com um Controlador de Aplicação?
Resposta:Se a aplicação dita o fluxo de telas, um Controlador de Aplicação é necessário; ao contrário, se os usuários podem navegar livremente entre as telas, sua implementação pode

Mais Livros Grátis
Escaneie para baixar
Ouça-o

ser excessiva.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Ad

Escanear para baixar

Ler, Compartilhar, Empoderar

Conclua Seu Desafio de Leitura, Doe Livros para Crianças Africanas.

O Conceito

BOOKS FOR AFRICA × ×

Esta atividade de doação de livros está sendo realizada em conjunto com a Books For Africa.Lançamos este projeto porque compartilhamos a mesma crença que a BFA: Para muitas crianças na África, o presente de livros é verdadeiramente um presente de esperança.

A Regra

Ganhe 100 pontos ----> **Resgate um livro** ----> **Doe para a África**

Seu aprendizado não traz apenas conhecimento, mas também permite que você ganhe pontos para causas beneficentes! Para cada 100 pontos ganhos, um livro será doado para a África.

Teste gratuito com Bookey

Capítulo 13 | Concorrência| Perguntas e respostas

1.Pergunta

Quais são os principais problemas de concorrência discutidos no capítulo?

Resposta:O capítulo destaca problemas de concorrência essenciais, como atualizações perdidas e leituras inconsistentes. Atualizações perdidas ocorrem quando dois processos tentam modificar os mesmos dados ao mesmo tempo, levando à sobrescrição de uma atualização. Leituras inconsistentes acontecem quando um processo lê dados que foram modificados por outro processo concorrente, resultando em informações desatualizadas ou incorretas.

2.Pergunta

Como as transações ajudam a gerenciar questões de concorrência em aplicações corporativas?

Resposta:As transações fornecem uma estrutura que ajuda a mitigar muitos problemas de concorrência ao agrupar

Mais Livros Grátis
Escaneie para baixar
Ouça-o

operações de manipulação de dados. Desde que as operações sejam realizadas dentro de uma transação, muitos problemas de concorrência podem ser evitados. No entanto, nem todas as interações podem se encaixar dentro de uma única transação, exigindo medidas adicionais de controle de concorrência.

3.Pergunta

Qual é a diferença entre controle de concorrência otimista e pessimista?

Resposta:O controle de concorrência otimista permite que vários processos editem dados simultaneamente, verificando conflitos apenas ao confirmar as alterações. Em contraste, o controle de concorrência pessimista impede que outros editem dados se um processo já o tiver solicitado, reduzindo assim potenciais conflitos à custa de uma concorrência reduzida.

4.Pergunta

Quais são as vantagens e desvantagens do uso do controle de concorrência otimista?

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Resposta:A vantagem do controle de concorrência otimista é uma melhor concorrência e eficiência, já que múltiplas atualizações podem ocorrer sem bloquear recursos. No entanto, a principal desvantagem é que os conflitos são detectados apenas no momento da confirmação, o que pode levar à frustração do usuário se ele tiver investido tempo para inserir dados que não podem ser salvos.

5.Pergunta

Como as transações garantem as propriedades ACID e quais são essas propriedades?

Resposta:As transações são regidas pelas propriedades ACID: Atomicidade (transações são todas ou nada), Consistência (as transações devem garantir que o estado permaneça válido), Isolamento (as transações não devem interferir umas nas outras) e Durabilidade (as mudanças de transações confirmadas devem persistir). Essas propriedades são essenciais para garantir processamento de dados confiável e integridade.

6.Pergunta

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Quais estratégias podem ser usadas para prevenir deadlocks no controle de concorrência?

Resposta:Para prevenir deadlocks, pode-se impor uma ordem rigorosa para a aquisição de bloqueios, garantindo que os processos não possam manter bloqueios enquanto solicitam outros. Outra estratégia envolve o uso de timeouts para solicitações de bloqueio, onde um processo abriria mão de seus bloqueios após esperar demais, permitindo que outros avancem.

7.Pergunta

Qual é a importância de entender os contextos de execução na gestão da concorrência?

Resposta:Compreender os contextos de execução é vital, pois determina como solicitações e sessões interagem dentro do sistema. Diferentes contextos afetama maneira como os problemas de concorrência se manifestam e como os recursos são bloqueados e compartilhados, influenciando o desempenho geral do sistema e a experiência do usuário.

8.Pergunta

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Quais recomendações são feitas para lidar com a concorrência em servidores de aplicação?

Resposta:O capítulo recomenda evitar bloqueios e sincronizações explícitas no código da aplicação. Ao invés disso, usar processos por sessão ou agrupar solicitações pode alcançar concorrência sem a complexidade de threads, tornando o sistema mais robusto e mais fácil de manter.

9.Pergunta

Como escolher entre transações longas e curtas em aplicações corporativas?

Resposta:Transações longas podem reduzir a complexidade em alguns casos, mas criam gargalos e problemas de escalabilidade. É aconselhável dividir transações de negócios em transações de sistema mais curtas sempre que possível, permitindo maior desempenho e reduzindo a contenção no banco de dados.

10.Pergunta

Qual o impacto do controle de concorrência na experiência do usuário?

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Resposta:O controle de concorrência impacta diretamente a experiência do usuário ao determinar quão suavemente várias transações podem ocorrer. A escolha entre controles otimistas e pessimistas afeta a capacidade de resposta e a probabilidade de frustração do usuário devido a falhas ou atrasos nas transações.

11.Pergunta

Qual é a abordagem recomendada ao lidar com problemas de concorrência em aplicações corporativas?

Resposta:A abordagem preferida é confiar no sistema de transações para lidar com problemas de concorrência sempre que possível. Utilizar padrões de transação, como bloqueios otimistas e pessimistas, pode simplificar a complexidade de gerenciar a consistência dos dados, mas é essencial entender os trade-offs envolvidos.

Capítulo 14 | Estado da Sessão| Perguntas e respostas

1.Pergunta

Qual é a questão fundamental por trás do debate sobre sessões sem estado versus sessões com estado em

Mais Livros Grátis
Escaneie para baixar
Ouça-o

aplicações corporativas?

Resposta:A questão fundamental está em reconhecer que algumas sessões são inerentemente com estado e, em seguida, determinar como gerenciar esse estado. Essa percepção impacta como os dados de transações de negócios são armazenados antes de serem enviados ao banco de dados.

2.Pergunta

O que significa um servidor ser 'sem estado'?

Resposta:Um servidor sem estado não retém nenhum estado entre as requisições. Cada requisição é processada de forma independente, o que significa que o servidor não mantém nenhuma informação sobre interações anteriores. Isso permite um uso eficiente dos recursos, pois múltiplas requisições podem ser tratadas simultaneamente.

3.Pergunta

Você pode dar um exemplo de um objeto de servidor sem estado?

Resposta:Um exemplo de um objeto de servidor sem estado é

Mais Livros Grátis
Escaneie para baixar
Ouça-o

uma página da web que fornece informações sobre um livro com base em um número ISBN fornecido na URL. Cada requisição é tratada como única e, uma vez que o servidor responde, ele não retém nenhuma informação sobre essa requisição.

4.Pergunta

Quais desafios surgem com o uso de servidores com estado?

Resposta:Servidores com estado requerem a manutenção do estado para clientes individuais, o que pode levar a ineficiências de recursos. Por exemplo, se vários usuários estiverem fazendo requisições, um servidor com estado pode ocupar recursos com objetos de servidor ociosos à espera de interações dos usuários.

5.Pergunta

Como o conceito de estado de sessão se aplica a transações de negócios?

Resposta:Estado de sessão refere-se a dados temporários que são relevantes apenas para uma interação específica do

Mais Livros Grátis
Escaneie para baixar
Ouça-o

usuário dentro de uma transação de negócios. Esse estado precisa ser gerenciado e armazenado entre requisições até que a transação seja concluída.

6.Pergunta

Quais são alguns métodos para armazenar estado de sessão?

Resposta:O estado de sessão pode ser armazenado de várias maneiras: Estado de Sessão no Cliente, que mantém dados no lado do cliente (como cookies ou campos ocultos); Estado de Sessão no Servidor, que mantém dados na memória ou em armazenamento durável no servidor; e Estado de Sessão em Banco de Dados, que armazena dados em um banco de dados usando tabelas e campos.

7.Pergunta

Quais são as vantagens e desvantagens entre Estado de Sessão no Cliente e Estado de Sessão no Servidor?

Resposta:O Estado de Sessão no Cliente reduz a carga do servidor ao transferir alguns dados para o cliente, mas pode levantar preocupações de segurança e aumentar o uso de

Mais Livros Grátis
Escaneie para baixar
Ouça-o

largura de banda. O Estado de Sessão no Servidor, embora potencialmente mais eficiente em termos de recursos, requer gestão cuidadosa para garantir que persista com precisão entre as requisições sem desperdiçar recursos.

8.Pergunta

Por que a isolação é importante na gestão do estado de sessão?

Resposta:A isolação é crucial para garantir que a sessão de um usuário não interfira na de outro; isso assegura a integridade e segurança dos dados, especialmente em aplicações como sistemas de reservas, onde vários usuários podem estar modificando dados simultaneamente.

9.Pergunta

Quando o Estado de Sessão em Banco de Dados pode ser a opção preferida?

Resposta:O Estado de Sessão em Banco de Dados pode ser preferido em cenários onde a recuperação e o agrupamento são críticos, e é necessário manter a estabilidade e integridade dos dados entre vários servidores e transações.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

10.Pergunta

Qual é uma consideração importante ao optar por implementar sessões em aplicações corporativas?

Resposta:Uma consideração importante é o equilíbrio entre as necessidades de desempenho e a gestão de recursos, bem como como a experiência do usuário será afetada se as sessões forem canceladas ou interrompidas.

Capítulo 15 | Estratégias de Distribuição| Perguntas e respostas

1.Pergunta

Quais são os principais inconvenientes dos sistemas de objetos distribuídos?

Resposta:Os sistemas de objetos distribuídos costumam levar a problemas de desempenho devido à maior latência das chamadas remotas em comparação com chamadas de procedimento local. Distribuir objetos aumenta a complexidade, torna os sistemas mais difíceis de manter e normalmente exige o uso de interfaces de granulação mais grossa, o que diminui a flexibilidade e a elegância do design

Mais Livros Grátis
Escaneie para baixar
Ouça-o

orientado a objetos.

2.Pergunta

Por que é importante usar clustering em vez de distribuição ao projetar uma aplicação?

Resposta:O clustering permite que todas as classes operem dentro de um único processo, possibilitando chamadas locais rápidas e um melhor desempenho. Ao rodar várias cópias desse processo em diferentes nós, a aplicação pode equilibrar eficientemente a carga, melhorar a manutenção e simplificar a programação sem incorrer na sobrecarga de chamadas remotas.

3.Pergunta

Qual é o princípio por trás do uso de interfaces de granulação grossa nas fronteiras de distribuição?

Resposta:Interfaces de granulação grossa são usadas nas fronteiras de distribuição para minimizar o número de chamadas remotas. Quando as chamadas remotas são lentas, combinar várias operações em uma única chamada reduz a penalidade de desempenho associada à latência da rede,

Mais Livros Grátis
Escaneie para baixar
Ouça-o

tornando a interface mais eficiente para a comunicação distribuída.

4.Pergunta

Qual é a Primeira Lei do Design de Objetos Distribuídos segundo Martin Fowler?

Resposta:A Primeira Lei do Design de Objetos Distribuídos é: 'Não distribua seus objetos!' Isso enfatiza a importância de evitar distribuições desnecessárias para reduzir a complexidade e melhorar o desempenho.

5.Pergunta

Como deve-se lidar com as fronteiras de distribuição ao projetar software?

Resposta:Para gerenciar efetivamente as fronteiras de distribuição, limite-as o máximo possível e use objetos de granulação grossa apenas para acesso remoto, enquanto mantém o funcionamento interno da aplicação focado em objetos de granulação fina para preservar a simplicidade e a eficiência.

6.Pergunta

O que é uma Fachada Remota e como ela auxilia no

Mais Livros Grátis
Escaneie para baixar
Ouça-o

design de objetos distribuídos?

Resposta:Uma Fachada Remota é um padrão arquitetural que utiliza objetos de granulação grossa para agir como uma fachada para objetos de granulação fina, minimizando assim a complexidade introduzida por chamadas remotas. Permite que os desenvolvedores usem operações de granulação fina dentro do mesmo processo, expondo apenas os métodos de granulação grossa necessários para o acesso remoto.

7.Pergunta

Por que um Objeto de Transferência de Dados (DTO) é usado em sistemas distribuídos?

Resposta:Um Objeto de Transferência de Dados é utilizado para agrupar dados para transferência eficiente pela rede, assegurando que apenas as informações necessárias sejam enviadas em uma única chamada, em vez de realizar múltiplas chamadas para diferentes pedaços de dados.

8.Pergunta

O que Martin Fowler sugere sobre o uso de interfaces baseadas em XML versus chamadas remotas diretas?

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Resposta:Fowler recomenda o uso de interfaces baseadas em XML principalmente quando diferentes plataformas precisam se comunicar entre si. Para sistemas construídos na mesma plataforma, chamadas remotas diretas são preferidas para melhor desempenho. Combinações de ambas podem adicionar complexidade, mas podem ser úteis se requisitos específicos exigirem.

9.Pergunta

Que tipo de abordagem Martin Fowler favorece para gerenciar sistemas distribuídos em geral?

Resposta:Martin Fowler cada vez mais prefere uma abordagem baseada em mensagens, inerentemente assíncrona, para lidar com sistemas distribuídos, pois promove o desacoplamento e potencialmente melhor escalabilidade em comparação com chamadas de procedimento remoto síncronas.

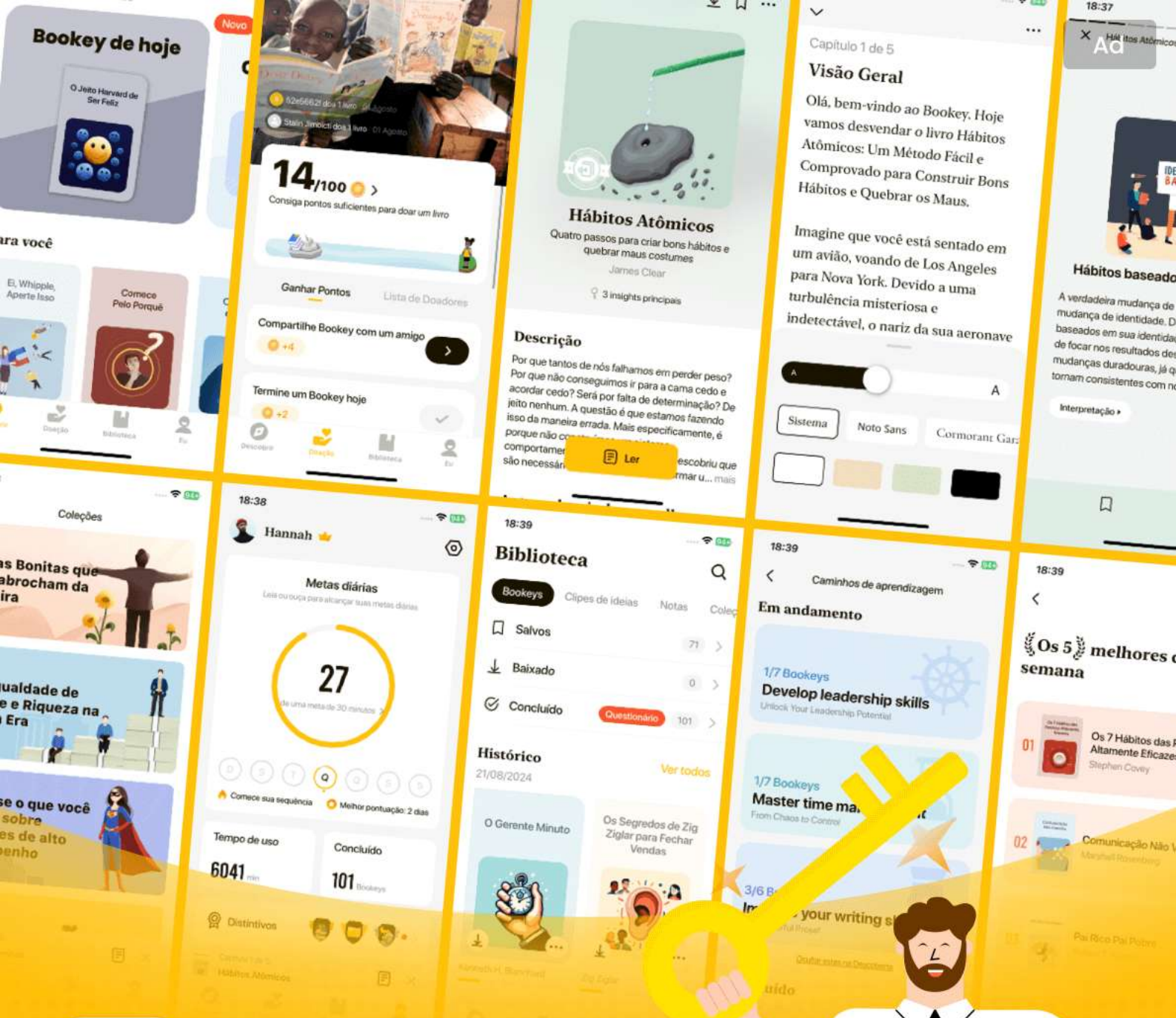
10.Pergunta

Qual é a importância de entender a diferença entre interfaces locais e remotas?

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Resposta:Entender a diferença entre interfaces locais e remotas é crucial porque impacta como você projeta as interações entre objetos. Enquanto as chamadas locais são rápidas e de granulação fina, as chamadas remotas são mais lentas e necessitam de uma estrutura de granulação grossa, influenciando assim o desempenho e a manutenibilidade geral do sistema.

Mais Livros Grátis
Escaneie para baixar
Ouça-o



As melhores ideias do mundo desbloqueiam seu potencial

Essai gratuit avec Bookey

Descarregar na App Store

OBTER NO Google Play

Escanear para baixar

Capítulo 16 | Unindo Tudo| Perguntas e respostas

1.Pergunta

Qual é o principal conselho oferecido neste capítulo sobre arquitetura de aplicações corporativas?

Resposta:O capítulo enfatiza a importância de reunir os padrões discutidos anteriormente para guiar o design das aplicações corporativas. Destaca que, embora o autor não possua um conhecimento onisciente, o conselho dado deve servir para estimular o pensamento crítico e a tomada de decisão pessoal, em vez de substituir o julgamento de um arquiteto.

2.Pergunta

Como um arquiteto deve abordar o processo de tomada de decisão ao projetar uma aplicação corporativa?

Resposta:Os arquitetos devem considerar cuidadosamente o contexto específico de seu projeto antes de tomar decisões. Eles devem utilizar os padrões discutidos neste livro como uma estrutura de pensamento, mas depender de sua própria

Mais Livros Grátis
Escaneie para baixar
Ouça-o

compreensão e expertise sobre o projeto em questão.

3.Pergunta

Quais são os três principais padrões de lógica de domínio mencionados, e quais são suas respectivas forças e fraquezas?

Resposta:Os três principais padrões de lógica de domínio são Script de Transação, Módulo de Tabela e Modelo de Domínio. O Script de Transação é simples e eficaz para lógica direta, mas tem dificuldades com regras de negócios complexas. O Módulo de Tabela equilibra complexidade e se encaixa bem em bancos de dados relacionais, enquanto o Modelo de Domínio é poderoso para lógica intrincada, mas pode ser complicado de aprender e integrar com bancos de dados relacionais.

4.Pergunta

O que é integração contínua, e por que é importante para decisões arquitetônicas?

Resposta:Integração contínua é uma prática de desenvolvimento onde as mudanças de código são testadas automaticamente e integradas em uma base de código

Mais Livros Grátis
Escaneie para baixar
Ouça-o

compartilhada. É vital para decisões arquitetônicas, pois facilita modificações e ajustes, ajudando as equipes a permanecerem ágeis e responsivas às mudanças.

5.Pergunta

Por que um arquiteto pode optar por um Módulo de Tabela em vez de outros padrões em um ambiente .NET?

Resposta:O Módulo de Tabela é favorecido em .NET porque seu design se alinha bem com as interações comuns de banco de dados e as ferramentas disponíveis na plataforma, tornando-se uma escolha pragmática para desenvolvedores que trabalham dentro desse ecossistema.

6.Pergunta

Como os padrões de acesso a banco de dados diferem com base na camada de domínio escolhida?

Resposta:A escolha da camada de domínio influencia significativamente a seleção dos padrões de acesso a banco de dados. Por exemplo, se um Script de Transação estiver sendo usado, padrões como Gateway de Dados por Linha ou Gateway de Dados de Tabela podem ser apropriados,

Mais Livros Grátis
Escaneie para baixar
Ouça-o

enquanto um Modelo de Domínio pode exigir soluções mais sofisticadas como Mapeador de Dados.

7.Pergunta

Que consideração deve ser dada ao escolher a camada de apresentação de uma aplicação?

Resposta:Ao escolher a camada de apresentação, é necessário decidir entre uma interface de cliente rico e uma interface de navegador HTML. A última é frequentemente mais simples e acessível, enquanto clientes ricos podem oferecer uma melhor experiência ao usuário, mas envolvem mais esforço de programação.

8.Pergunta

Por que a refatoração arquitetônica é importante em aplicações corporativas?

Resposta:A refatoração arquitetônica permite a modificação e melhoria da estrutura da aplicação sem uma reforma completa, permitindo que os desenvolvedores se adaptem a novas exigências ou insights compreendidos durante o ciclo de vida da aplicação.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

9.Pergunta

Qual é o equívoco comum sobre o uso de serviços Web na arquitetura de aplicações?

Resposta:O equívoco comum é que se deve decompor aplicações únicas em vários serviços Web. Em vez disso, é mais eficiente construir aplicações abrangentes e expor funcionalidades como serviços Web quando necessário.

10.Pergunta

O que se deve lembrar ao projetar aplicações sobre o uso de procedimentos armazenados?

Resposta:Procedimentos armazenados podem aprimorar o desempenho devido à sua execução dentro do mesmo processo que o banco de dados, mas podem levar a um acoplamento forte com fornecedores específicos de banco de dados, portanto, deve-se ter cuidado com seu uso para manter a modularidade e a facilidade de portabilidade.

Capítulo 17 | Padrões de Lógica de Domínio| Perguntas e respostas

1.Pergunta

Qual é o objetivo principal do padrão Transaction Script,

Mais Livros Grátis
Escaneie para baixar
Ouça-o

segundo Martin Fowler?

Resposta:O objetivo principal do padrão Transaction Script é organizar a lógica de negócios de forma que cada procedimento trate um único pedido de transação, seja consultando ou alterando dados. É uma abordagem simples e direta que atua como um modelo procedural para gerenciar interações com o sistema.

2.Pergunta

Como o padrão Transaction Script suporta aplicações de negócios?

Resposta:O padrão Transaction Script apoia as aplicações de negócios ao estruturar a lógica em torno das transações que a aplicação realiza, como reservar um quarto de hotel ou processar uma venda, permitindo que cada transação seja encapsulada em seu próprio script.

3.Pergunta

Quais são algumas práticas recomendadas para estruturar Transaction Scripts?

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Resposta:As práticas recomendadas incluem separar os Transaction Scripts em sub-rotinas ou classes distintas, mantê-los livres de qualquer lógica de apresentação direta e organizá-los por área temática para aumentar a manutenibilidade e a legibilidade.

4.Pergunta

Quando um desenvolvedor deve considerar usar um Modelo de Domínio em vez de Transaction Scripts?

Resposta:Um desenvolvedor deve considerar usar um Modelo de Domínio quando a lógica de negócios se torna complexa e envolve muitas regras ou relacionamentos, a fim de evitar duplicação e melhorar a manutenibilidade da aplicação.

5.Pergunta

Você pode dar um exemplo específico de como Transaction Scripts podem tratar o reconhecimento de receita em sistemas de negócios?

Resposta:Para o reconhecimento de receita, um Transaction Script poderia calcular os valores devidos com base no tipo de contrato, por exemplo, reconhecendo diferentes partes de

Mais Livros Grátis
Escaneie para baixar
Ouça-o

uma taxa ao implementar regras para cenários de faturamento complexos, como reconhecer um terço de um pagamento agora e dois terços depois.

6.Pergunta

Qual é uma possível desvantagem de usar Transaction Scripts para lógicas de negócios mais complexas?

Resposta:Uma desvantagem potencial de usar Transaction Scripts é o risco de duplicação de código e a dificuldade em gerenciar mudanças devido à natureza procedural dos scripts, especialmente quando várias transações compartilham lógica comum.

7.Pergunta

O que diferencia um Table Module de um Modelo de Domínio conforme explicado no texto?

Resposta:Um Table Module lida com a lógica de negócios geral para todas as linhas em uma tabela ou vista de banco de dados, focando em uma única instância que gerencia múltiplos registros, enquanto um Modelo de Domínio representa objetos individuais no sistema, permitindo um

Mais Livros Grátis
Escaneie para baixar
Ouça-o

design orientado a objetos mais rico e complexo.

8.Pergunta

Quais benefícios o padrão de Camada de Serviço oferece?

Resposta:O padrão de Camada de Serviço oferece benefícios como fornecer uma interface comum para diferentes clientes de uma aplicação, gerenciar transações, coordenar respostas entre várias operações e encapsular a lógica de negócios para manter uma separação mais limpa de responsabilidades.

9.Pergunta

Como se determina quando implementar uma Camada de Serviço?

Resposta:Uma Camada de Serviço deve ser implementada quando uma aplicação requer múltiplas interações do cliente lidando com respostas ou transações complexas, especialmente em casos onde há necessidade de redundância entre diferentes interfaces de usuário ou sistemas de backend.

10.Pergunta

Qual é a importância de encapsular a lógica da aplicação em uma camada de serviço?

Resposta:Encapsular a lógica da aplicação em uma camada

Mais Livros Grátis
Escaneie para baixar
Ouça-o

de serviço é importante porque reduz o acoplamento entre a interface do usuário e as regras de negócios, tornando o sistema mais fácil de manter e adaptar à medida que os requisitos de negócios mudam.

Capítulo 18 | Padrões de Arquitetura de Fontes de Dados| Perguntas e respostas

1.Pergunta

O que é um Gateway de Dados de Tabela e como ele funciona?

Resposta:Um Gateway de Dados de Tabela é um objeto que atua como um gateway para uma tabela de banco de dados, gerenciando todas as interações para essa tabela, incluindo seleções, inserções, atualizações e exclusões. Sua interface geralmente consiste em vários métodos de busca para recuperar dados e métodos para modificá-los, tudo isso enquanto mapeia os parâmetros de entrada para chamadas SQL. Ao manter o SQL separado da lógica da aplicação, simplifica as interações com o banco de dados e melhora a manutenibilidade.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

2.Pergunta

Por que é problemático misturar SQL com a lógica da aplicação?

Resposta:Misturar SQL com a lógica da aplicação pode levar a diversos problemas, incluindo dificuldade em gerenciar e otimizar o banco de dados, além de potenciais erros de SQL mal escrito. Muitos desenvolvedores podem não se sentir confortáveis com a sintaxe SQL, levando a bugs e ineficiências no acesso ao banco de dados.

3.Pergunta

Quais são as vantagens que o padrão Gateway de Dados de Tabela oferece?

Resposta:O padrão Gateway de Dados de Tabela simplifica a interação com o banco de dados com sua interface limpa, permitindo uma encapsulação natural da lógica de acesso. Isso torna mais fácil evoluir a estrutura do banco de dados de forma independente. Além disso, ele pode funcionar harmoniosamente com outros padrões, como Módulo de Tabela e Modelo de Domínio, o que reduz o acoplamento

Mais Livros Grátis
Escaneie para baixar
Ouça-o

entre o código e o banco de dados.

4.Pergunta

Quando você deve usar um Gateway de Dados de Tabela em vez de um Mapeador de Dados?

Resposta:Um Gateway de Dados de Tabela deve ser utilizado quando a simplicidade é uma prioridade e quando o design está intimamente alinhado à estrutura do banco de dados. Se a lógica da aplicação é direta e se concentra principalmente em operações CRUD, o Gateway de Dados de Tabela é a opção mais fácil. No entanto, para uma lógica mais complexa que requer desacoplamento do banco de dados da lógica de negócios, um Mapeador de Dados é preferido.

5.Pergunta

Quais são os possíveis pontos negativos do uso do padrão Active Record?

Resposta:O padrão Active Record acopla rigidamente a lógica de domínio ao acesso ao banco de dados, tornando-o menos flexível em termos de refatoração ou evolução de qualquer um dos designs. Ele pode simplificar operações

Mais Livros Grátis
Escaneie para baixar
Ouça-o

CRUD, mas se torna complicado com uma lógica de negócios mais complexa devido aos desafios de gerenciar relacionamentos diretos e garantir manutenibilidade.

6.Pergunta

Qual é o papel do padrão Mapeador de Dados na arquitetura de aplicações?

Resposta:O padrão Mapeador de Dados fornece uma camada que transfere dados entre objetos e um banco de dados, permitindo que eles permaneçam independentes entre si. Essa separação de preocupações permite que a lógica de negócios complexa seja melhor organizada e evita as armadilhas do acoplamento direto entre modelos de dados e a estrutura do banco de dados.

7.Pergunta

Como o Lazy Load pode ser benéfico no contexto de mapeadores de dados?

Resposta:O Lazy Load otimiza o desempenho ao carregar dados apenas quando necessário, reduzindo o tempo de carregamento inicial e minimizando o uso de memória. Ele

Mais Livros Grátis
Escaneie para baixar
Ouça-o

permite que objetos de domínio sejam buscados sob demanda, melhorando a eficiência em aplicações com estruturas de dados complexas.

8.Pergunta

Por que um desenvolvedor poderia escolher um Mapeador de Dados simples em vez de abordagens de mapeamento mais complexas?

Resposta:Um Mapeador de Dados simples atende às necessidades básicas sem a sobrecarga de estratégias de mapeamento complicadas. Ele é útil em casos onde existem relacionamentos diretos entre objetos e tabelas de banco de dados, e se torna uma opção atraente porque pode lidar com operações CRUD sem complexidade desnecessária.

9.Pergunta

Quais considerações devem ser feitas ao decidir implementar um Gateway de Dados de Linha em vez de um Gateway de Dados de Tabela?

Resposta:A escolha entre Gateway de Dados de Linha e Gateway de Dados de Tabela muitas vezes envolve avaliar se deve-se lidar com os dados em um nível de registro único

Mais Livros Grátis
Escaneie para baixar
Ouça-o

(Gateway de Dados de Linha) ou em um nível de tabela (Gateway de Dados de Tabela). Gateways de Dados de Linha podem resultar em duplicação de código se não forem usados com cuidado, enquanto Gateways de Dados de Tabela podem ser mais adequados para interações simples com tabelas. A decisão deve levar em conta a complexidade da aplicação, a manutenibilidade e a probabilidade de necessidade de escalabilidade.

10.Pergunta

Por que alguém poderia preferir usar mapeamento de metadados ao implementar Mapeadores de Dados?

Resposta:O mapeamento de metadados aumenta a flexibilidade, pois permite alterações na estratégia de mapeamento sem alterar o código-fonte. Esta abordagem pode se adaptar dinamicamente a diferentes esquemas de objetos e banco de dados, simplificando o processo de mapeamento e reduzindo a necessidade de extensas refatorações de código à medida que os requisitos evoluem.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Ad

Escanear para baixar

Experimente o aplicativo Bookey para ler mais de 1000 resumos dos melhores livros do mundo

Desbloqueie 1000+ títulos, 80+ tópicos

Novos títulos adicionados toda semana

duct & Brand | Liderança & Colaboração | Gerenciamento de Tempo | Relacionamento & Comunica

Estratégia de Negócios | Criatividade | Memórias | Conheça a Si Mesmo | Psicologia Po

mpreendedorismo | História Mundial | Comunicação entre Pais e Filhos | Autocuidado | Mi

Visões dos melhores livros do mundo

mento pos

Os 7 Hábitos das Pessoas Altamente Eficazes

Mini Hábitos

Hábitos Atômicos

O Clube das 5 da Manhã

Como Fazer Amigos e Influenciar Pessoas

Com Não

Teste gratuito com Bookey

Capítulo 19 | Padrões Comportamentais de Objeto-Relacional| Perguntas e respostas

1.Pergunta

Qual é o propósito do padrão Unit of Work?

Resposta:O padrão Unit of Work é projetado para manter uma lista de objetos afetados por uma transação de negócios e coordenar a gravação das alterações no banco de dados, garantindo que atualizações, inserções e exclusões sejam gerenciadas de forma eficaz, sem chamadas excessivas ao banco de dados.

2.Pergunta

Por que é importante rastrear as mudanças no Unit of Work?

Resposta:Rastrear mudanças é essencial para garantir que quaisquer modificações feitas em objetos durante uma transação de negócios sejam corretamente refletidas no banco de dados ao ser feito o commit, evitando inconsistências e operações desnecessárias no banco de dados.

3.Pergunta

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Quais são as duas principais técnicas para registrar objetos dentro de um Unit of Work?

Resposta:As duas principais técnicas são 'registro por chamada', onde o usuário deve se lembrar de registrar objetos com o Unit of Work, e 'registro de objeto', onde o próprio objeto informa ao Unit of Work sobre suas mudanças de estado.

4.Pergunta

O que é um Mapa de Identidade e por que é usado em conjunto com o Unit of Work?

Resposta:Um Mapa de Identidade é usado para acompanhar todos os objetos que foram lidos do banco de dados dentro de uma única transação de negócios, evitando o carregamento do mesmo registro do banco de dados várias vezes, o que melhora o desempenho e mantém a consistência dos objetos.

5.Pergunta

O que é Lazy Load e como isso beneficia o desempenho da aplicação?

Resposta:Lazy Load é um padrão de design que adia a

Mais Livros Grátis
Escaneie para baixar
Ouça-o

inicialização de um objeto até o ponto em que ele é necessário. Isso pode melhorar o desempenho, reduzindo o tempo de carregamento inicial e a utilização de memória da aplicação ao não buscar todos os dados relacionados imediatamente.

6.Pergunta

Você pode explicar as diferentes implementações de Lazy Load?

Resposta:Lazy Load pode ser implementado por meio de inicialização preguiçosa, proxies virtuais, mantenedores de valores ou fantasmas para controlar quando os dados relacionados em um objeto são efetivamente carregados do banco de dados, permitindo um uso otimizado dos recursos.

7.Pergunta

Quais armadilhas comuns devem ser evitadas ao usar Lazy Load?

Resposta:Armadilhas comuns incluem carregamento em cascata, onde acessar uma coleção de objetos Lazy Loaded causa múltiplas consultas ao banco de dados, e potenciais

Mais Livros Grátis
Escaneie para baixar
Ouça-o

problemas de identidade com proxies virtuais que levam a comparações de igualdade incorretas.

8.Pergunta

Como o Unit of Work ajuda em cenários de transação complexos?

Resposta:O Unit of Work é fundamental para lidar com transações complexas que podem abranger múltiplas transações do sistema usando técnicas como bloqueio otimista e pessimista offline, garantindo a integridade dos dados e o gerenciamento adequado das mudanças durante todo o processo.

9.Pergunta

Em quais cenários o Lazy Load deve ser utilizado com cautela?

Resposta:Lazy Load deve ser empregado quando acessar dados relacionados envolve chamadas adicionais ao banco de dados que não são necessárias para o contexto imediato, para evitar impactos de desempenho desnecessários, mas deve ser evitado para atributos armazenados junto aos dados

Mais Livros Grátis
Escaneie para baixar
Ouça-o

principais do objeto.

10.Pergunta

Como você pode implementar um Mapa de Identidade em Java e quais são alguns benefícios?

Resposta:Em Java, um Mapa de Identidade pode ser implementado com um mapa hash para armazenar objetos carregados com base em sua chave primária. Os benefícios incluem evitar leituras inconsistentes do banco de dados e melhorar os tempos de acesso por meio de cache.

11.Pergunta

Qual é o papel do padrão Active Record nas interações com o banco de dados?

Resposta:O padrão Active Record combina a lógica de manipulação de dados com a interação com o banco de dados, permitindo que objetos gerenciem sua persistência, simplificando assim o código necessário para gerenciar operações de banco de dados.

12.Pergunta

Que estratégia poderia ser empregada para reduzir a complexidade do Lazy Loading em uma aplicação?

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Resposta:Uma estratégia é utilizar programação orientada a aspectos para separar a lógica de Lazy Load da lógica de negócios, permitindo que os desenvolvedores se concentrem em seus modelos de domínio sem sobrecarregá-los com preocupações de persistência.

13.Pergunta

Por que é crucial gerenciar o processo de commit em um Unit of Work?

Resposta:Gerenciar o processo de commit é crucial porque garante que todas as mudanças sejam aplicadas em uma única transação, reduzindo o risco de inconsistências, reforçando a integridade referencial e gerenciando quaisquer verificações de concorrência necessárias.

14.Pergunta

Que funcionalidade precisa ser incluída em mapeadores de dados ao lidar com objetos fantasmas?

Resposta:Os mapeadores de dados devem implementar funcionalidade para carregar objetos fantasmas adequadamente, garantindo que acessadores acionem o

Mais Livros Grátis
Escaneie para baixar
Ouça-o

processo de carregamento e que os dados carregados sejam corretamente atribuídos aos atributos do objeto fantasma.

15.Pergunta

Como o Mapa de Identidade ajuda no desempenho ao trabalhar com objetos?

Resposta:Ao manter uma única referência a objetos já carregados, o Mapa de Identidade reduz a frequência de chamadas ao banco de dados para dados frequentemente acessados, atuando efetivamente como um cache e mitigando estrangulamentos de desempenho em aplicações.

16.Pergunta

Quais são as desvantagens da inicialização preguiçosa e como podem ser mitigadas?

Resposta:A inicialização preguiçosa pode introduzir dependências entre objetos e sua fonte de dados, complicando testes unitários. Isso pode ser mitigado garantindo que todos os campos sejam devidamente encapsulados e gerenciando a lógica de carregamento separadamente, como por meio de proxies virtuais.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

17.Pergunta

Como uma abordagem híbrida pode melhorar a eficiência do padrão Unit of Work?

Resposta:Uma abordagem híbrida pode envolver uma combinação de registro por chamada e registro de objeto, permitindo flexibilidade na gestão de mudanças transitórias enquanto garante que estados de dados críticos sejam rastreados e atualizados com precisão.

Capítulo 20 | Padrões Estruturais Objetos-Relacionais| Perguntas e respostas

1.Pergunta

O que é o padrão Identity Field e por que é importante no mapeamento objeto-relacional?

Resposta:O padrão Identity Field armazena um campo de ID do banco de dados em um objeto para manter a identidade entre um objeto em memória e uma linha no banco de dados. É crucial porque permite a sincronização contínua da identidade do objeto com a linha correspondente no banco de dados, possibilitando atividades como leitura e

Mais Livros Grátis
Escaneie para baixar
Ouça-o

escrita de dados sem confusão de identidade.

2.Pergunta

Quais são as considerações ao escolher chaves para o mapeamento de Identity Field?

Resposta:Ao escolher chaves, considere se deve usar chaves significativas ou sem significado. Chaves significativas, como números de Seguro Social, podem ser suscetíveis a erros dos usuários e devem ser geralmente evitadas. Em vez disso, chaves sem significado, como inteiros gerados, fornecem a unicidade e imutabilidade necessárias para um mapeamento de identidade eficaz.

3.Pergunta

Como funcionam as chaves compostas e quando elas devem ser usadas?

Resposta:Chaves compostas usam mais de um campo do banco de dados para identificar exclusivamente um registro. Elas são benéficas quando a relação dos dados de uma tabela para outra exige múltiplos campos para contexto. No entanto, elas complicam o mapeamento e devem ser evitadas em

Mais Livros Grátis
Escaneie para baixar
Ouça-o

favor de chaves simples sempre que possível.

4.Pergunta

O que é o padrão Foreign Key Mapping e como ele se aplica no mapeamento objeto-relacional?

Resposta:O Foreign Key Mapping mapeia associações entre objetos para referências de chave estrangeira em tabelas de banco de dados, permitindo a representação de relacionamentos como álbum para artista ou pedido para itens da linha em memória, enquanto os liga corretamente no banco de dados.

5.Pergunta

Quais desafios estão associados às atualizações ao usar Foreign Key Mapping?

Resposta:Atualizar coleções referenciadas por chaves estrangeiras pode ser desafiador, pois é necessário gerenciar adições, atualizações e exclusões de forma eficaz dentro do objeto pai sem perder a integridade das associações.

6.Pergunta

Qual é a diferença entre Embedded Value e Serialized LOB?

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Resposta:Embedded Value mapeia os campos dos dados de um objeto nos campos da tabela do proprietário, permitindo consultas eficientes. Serialized LOB, por outro lado, serializa todo um gráfico de objetos em um único grande objeto armazenado no banco de dados e é melhor para relacionamentos complexos de objetos, embora não tenha a capacidade de consultar os dados incorporados diretamente.

7.Pergunta

O que é Single Table Inheritance e quais são suas vantagens e desvantagens?

Resposta:Single Table Inheritance mapeia uma hierarquia de herança em uma única tabela do banco de dados contendo todos os campos para todas as classes na hierarquia. As vantagens incluem menos joins e refatoração mais simples. As desvantagens incluem espaço desperdiçado com colunas vazias e potenciais problemas de desempenho se a tabela se tornar muito grande.

8.Pergunta

Como a Class Table Inheritance difere da Concrete Table Inheritance?

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Resposta:Class Table Inheritance possui uma tabela para cada classe na hierarquia, enquanto Concrete Table Inheritance cria uma tabela por subclasse concreta. A Class Table fornece maior clareza às custas do desempenho devido a múltiplos joins, enquanto a Concrete Table minimiza joins, mas pode levar a dificuldades na manutenção da integridade referencial.

9.Pergunta

Quando deve-se considerar o uso de Dependent Mapping?

Resposta:Use Dependent Mapping quando um objeto filho estiver intimamente acoplado a um objeto pai e não precisar existir de forma independente. Este padrão simplifica a gestão de persistência, pois salvar o pai gerencia automaticamente os objetos dependentes.

10.Pergunta

Qual é o papel dos Inheritance Mappers no design objeto-relacional?

Resposta:Inheritance Mappers fornecem uma estrutura para

Mais Livros Grátis
Escaneie para baixar
Ouça-o

organizar mapeadores de banco de dados que lidam com hierarquias de herança, minimizando a duplicação de código e permitindo uma gestão mais fácil de carregamento e salvamento de dados tanto em classes abstratas quanto concretas.

Capítulo 21 | Padrões de Mapeamento de Metadados Objetos-Relacionais| Perguntas e respostas

1.Pergunta

Qual é a principal vantagem de usar Mapeamento de Metadados para mapeamento objeto-relacional?

Resposta:O Mapeamento de Metadados simplifica o processo de gerenciamento do mapeamento objeto-relacional, permitindo que os desenvolvedores definam mapeamentos em um formato tabular, que pode ser processado por código genérico. Isso reduz o código tedioso e repetitivo, diminuindo a complexidade nas interações com o banco de dados.

2.Pergunta

Como a geração de código e a programação reflexiva

Mais Livros Grátis
Escaneie para baixar
Ouça-o

diferem em sua abordagem ao Mapeamento de Metadados?

Resposta:A geração de código envolve criar código-fonte com base em metadados durante o processo de construção, resultando em classes que parecem ser escritas à mão, mas são, na verdade, geradas automaticamente. A programação reflexiva, por outro lado, utiliza metadados para invocar métodos dinamicamente em tempo de execução, permitindo mais flexibilidade, mas potencialmente introduzindo desvantagens de desempenho devido à sua execução mais lenta em comparação com o código gerado estaticamente.

3.Pergunta

Em quais cenários pode ser vantajoso usar programação reflexiva, apesar de suas desvantagens?

Resposta:A programação reflexiva é benéfica em situações onde os mapeamentos de banco de dados devem mudar com frequência, pois permite atualizações nos dados de mapeamento sem a necessidade de recompilar o código. Essa flexibilidade é útil durante o desenvolvimento quando é

Mais Livros Grátis
Escaneie para baixar
Ouça-o

necessário se adaptar rapidamente a requisitos em mudança.

4.Pergunta

Quais desafios o Mapeamento de Metadados enfrenta quanto à manutenção da simplicidade?

Resposta:Embora um esquema de metadados simples geralmente cubra 90% dos casos de forma eficaz, situações anômalas podem complicar significativamente as coisas, exigindo uma complexidade adicional na estrutura de metadados. Assim, casos especiais podem resultar na necessidade de substituir o código de mapeamento genérico, muitas vezes exigindo soluções intricadas.

5.Pergunta

Como a mudança no nome de um campo privado no código pode afetar aplicações que usam Mapeamento de Metadados?

Resposta:Renomear um campo privado pode causar quebras inesperadas em aplicações que usam Mapeamento de Metadados, pois ferramentas automatizadas podem não detectar nomes de campos armazenados em arquivos de metadados externos, como XML. Assim, atualizações

Mais Livros Grátis
Escaneie para baixar
Ouça-o

manuais nos metadados são necessárias após tal refatoração.

6.Pergunta

Qual é o propósito do padrão Query Object e quando ele deve ser utilizado?

Resposta:O padrão Query Object permite encapsular consultas de banco de dados de uma forma mais orientada a objetos, permitindo que os desenvolvedores criem consultas com base em objetos de domínio em vez da sintaxe SQL. Esse padrão deve ser utilizado quando há necessidade de interações complexas com bancos de dados ou quando a aplicação requer flexibilidade na consulta em diferentes estruturas de banco de dados.

7.Pergunta

Como o padrão Repository pode ser usado para melhorar a legibilidade e a manutenibilidade do código?

Resposta:O padrão Repository abstrai a lógica de acesso a dados em um nível de separação, permitindo que os clientes interajam com objetos de domínio de forma simples e declarativa, sem precisar se preocupar com a lógica

Mais Livros Grátis
Escaneie para baixar
Ouça-o

subjacente do banco de dados. Isso torna o código mais limpo e mais fácil de manter, especialmente em sistemas com muitos tipos de domínio.

8.Pergunta

Por que é aconselhável evoluir o design do seu Mapeamento de Metadados ao longo do tempo?

Resposta:À medida que as necessidades da aplicação crescem e mudam, evoluir o design do Mapeamento de Metadados permite uma melhor adaptação a novos requisitos sem complicar excessivamente a configuração inicial. Complexidade crescente pode ser adicionada gradualmente para atender às necessidades de novos recursos à medida que surgem.

9.Pergunta

Quais aspectos fundamentais os desenvolvedores devem considerar ao decidir como armazenar Metadados?

Resposta:Os desenvolvedores devem considerar fatores como facilidade de acesso e análise versus flexibilidade e facilidade de edição ao escolher como armazenar Metadados, seja em

Mais Livros Grátis
Escaneie para baixar
Ouça-o

arquivos XML, diretamente no código-fonte ou como parte do próprio banco de dados.

10.Pergunta

Como o uso do padrão Repository facilita os testes unitários?

Resposta:Nos testes unitários, utilizar o padrão Repository permite o uso de coleções em memória para objetos de domínio, acelerando a execução dos testes e simplificando as configurações de teste, evitando assim a sobrecarga associada ao acesso ao banco de dados.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Ad

Escanear para baixar

Por que o Bookey é um aplicativo indispensável para amantes de livros

Conteúdo de 30min
Quanto mais profunda e clara for a interpretação que fornecemos, melhor será sua compreensão de cada título.

Clipes de Ideias de 3min
Impulsione seu progresso.

Questionário
Verifique se você dominou o que acabou de aprender.

E mais
Várias fontes, Caminhos em andamento, Coleções...

Teste gratuito com Bookey

Capítulo 22 | Padrões de Apresentação Web| Perguntas e respostas

1.Pergunta

Qual é o objetivo principal do padrão Model-View-Controller (MVC)?

Resposta:O objetivo principal do padrão MVC é separar uma aplicação em três papéis distintos: Model, que lida com os dados e a lógica de negócio; View, que exibe as informações; e Controller, que gerencia a entrada do usuário e atualiza o Model e a View conforme necessário. Essa separação promove um código organizado e melhora a manutenibilidade e escalabilidade.

2.Pergunta

Por que a separação da camada de apresentação do modelo é considerada um princípio de design fundamental?

Resposta:Separar a apresentação do modelo permite que os desenvolvedores personalizem como os usuários experienciam a aplicação sem alterar a lógica de negócio

Mais Livros Grátis
Escaneie para baixar
Ouça-o

subjacente. Isso possibilita múltiplas vistas para representar os mesmos dados em diferentes formatos (como interfaces web, APIs ou aplicações desktop) e simplifica os testes, uma vez que elementos não visuais (o modelo) podem ser testados independentemente dos elementos visuais.

3.Pergunta

Como o padrão Observer se relaciona ao MVC, especialmente com atualizações dinâmicas da view?

Resposta:No MVC, o padrão Observer é frequentemente implementado para permitir que a View reaja a mudanças no Model. Quando o Model atualiza seus dados, ele pode notificar todas as Views registradas (observadores) para que possam se atualizar, garantindo que a interface do usuário reflita os dados mais atuais sem criar dependências entre o Model e as Views.

4.Pergunta

Quais são os benefícios de usar um Page Controller em uma aplicação web?

Resposta:Um Page Controller permite que cada página web

Mais Livros Grátis
Escaneie para baixar
Ouça-o

gerencie seu próprio fluxo, lidando com solicitações e respostas de forma direta. Esse padrão é simples e intuitivo para os desenvolvedores, tornando fácil de entender e implementar, especialmente para aplicações onde a maior parte da lógica do controlador é direta.

5.Pergunta

Qual é a distinção entre um Page Controller e um Front Controller?

Resposta:O Page Controller lida com solicitações em nível de página, permitindo que cada página tenha sua própria lógica de processamento, enquanto o Front Controller centraliza o tratamento de solicitações, roteando todas as solicitações através de um único controlador. Isso pode simplificar a configuração e permitir comportamento compartilhado (como autenticação) em todas as solicitações.

6.Pergunta

Quais vantagens o Transform View oferece em comparação ao Template View?

Resposta:O Transform View permite a separação da

Mais Livros Grátis
Escaneie para baixar
Ouça-o

transformação de dados da lógica de apresentação, focando em converter dados de domínio em HTML. Ele possibilita testes mais fáceis e simplifica a manutenção do código, pois a lógica de transformação é contida separadamente, muitas vezes utilizando XSLT para melhor organização das regras de renderização, ao contrário do Template View, que tende a entrelaçar a lógica de apresentação com a estrutura HTML.

7.Pergunta

Por que é considerado uma desvantagem embutir muita lógica nas views, como scripts do lado do servidor ou scriptlets?

Resposta:Embutir muita lógica nas views complica a manutenção e reduz a clareza da estrutura da aplicação. Isso pode resultar em um código difícil de testar e entender, borrando a linha entre lógica de negócio e apresentação, o que a arquitetura MVC busca prevenir.

8.Pergunta

Quando se deve considerar usar o Transform View em vez do Template View em uma aplicação?

Resposta:Considere usar o Transform View ao trabalhar com

Mais Livros Grátis
Escaneie para baixar
Ouça-o

dados XML ou ambientes que suportam XSLT, onde transformações estruturadas podem proporcionar uma separação clara de preocupações e aumentar a portabilidade entre diferentes plataformas. Essa abordagem é benéfica para aplicações que necessitam de adaptabilidade na apresentação sem depender fortemente de implementações específicas de views.

9.Pergunta

Quais implicações o padrão Observer tem ao implementar atualizações dinâmicas de conteúdo em uma UI?

Resposta:O padrão Observer facilita garantir que todas as partes da UI estejam consistentes com os dados do Model. Quando ocorrem mudanças no Model, todas as Views registradas podem ser automaticamente atualizadas sem um acoplamento rígido, permitindo uma experiência de usuário mais responsiva e modularidade no design.

10.Pergunta

Quais devem ser os princípios de design orientadores ao implementar MVC em uma aplicação de software?

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Resposta:Os principais princípios de design para implementar MVC incluem promover uma clara separação de preocupações, garantir que os modelos não dependam da lógica de apresentação, criar Views independentes que possam renderizar o mesmo Model de maneiras diferentes e buscar alta testabilidade ao isolar a lógica de negócio dos elementos de UI.

11.Pergunta

Como a implementação do MVC pode levar a uma arquitetura de software mais manutenível?

Resposta:Implementar o MVC leva à manutenibilidade ao permitir que diferentes equipes de desenvolvimento trabalhem nos Models, Views e Controllers de forma independente. Essa abordagem modular simplifica atualizações e mudanças, além de tornar a depuração e os testes mais fáceis, isolando problemas em componentes específicos.

Capítulo 23 | Padrões de Distribuição| Perguntas e respostas

1.Pergunta

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Quais são as principais vantagens de usar um Facade Remoto em uma aplicação corporativa?

Resposta:As principais vantagens de usar um Facade Remoto incluem a minimização do número de chamadas remotas entre processos, o que melhora a eficiência e o desempenho. Ele fornece uma interface de granularidade mais grossa que permite que os clientes acessem objetos complexos de granularidade fina com uma única chamada, reduzindo assim a sobrecarga de múltiplas invocações de método. Além disso, ajuda na implementação de controles de segurança e transacionais de forma mais eficaz.

2.Pergunta

Como um Facade Remoto afeta a estrutura dos objetos em uma aplicação?

Resposta:Um Facade Remoto exige uma mudança de objetos de granularidade fina para uma interface de granularidade mais grossa. Isso significa que, em vez de trabalhar com

Mais Livros Grátis
Escaneie para baixar
Ouça-o

muitos pequenos métodos e objetos, os desenvolvedores devem consolidar chamadas e atualizar múltiplos objetos relacionados em uma única chamada. Embora isso possa levar a uma redução na clareza da intenção e no controle, é uma troca necessária para melhorar a eficiência da rede.

3.Pergunta

Qual é a importância dos Objetos de Transferência de Dados (DTOs) no contexto de Facades Remotos?

Resposta:Os Objetos de Transferência de Dados facilitam a transferência de múltiplos itens de dados entre processos distribuídos em uma única chamada de método, o que é crucial para minimizar a cara comunicação inter-processos. Eles encapsulam todos os dados necessários para uma operação, garantindo que estruturas de objetos complexas sejam simplificadas em formatos de fácil transferência, além de permitir a serialização para atravessar fronteiras de rede.

4.Pergunta

Os Facades Remotos podem conter lógica de domínio? Por quê?

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Resposta:Os Facades Remotos não devem conter lógica de domínio; eles são projetados para serem interfaces finas que delegam para objetos de domínio subjacentes. Encapsular a lógica de negócios dentro de um facade compromete a separação de preocupações que melhora a manutenibilidade e a modularidade. É melhor colocar a lógica de domínio nos objetos de granularidade fina ou em scripts de transação separados.

5.Pergunta

Quais considerações devem ser levadas em conta ao projetar um Facade Remoto?

Resposta:Ao projetar um Facade Remoto, deve-se considerar sua granularidade—equilibrando entre o número de métodos para uma API rica e as implicações de desempenho das chamadas de rede. Além disso, deve-se ter cuidado para garantir a separação da lógica de domínio do facade, aplicar segurança no nível do facade e determinar se o facade deve manter estado ou ser sem estado, pois isso impacta a gestão de recursos e a eficiência.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

6.Pergunta

Quais são as diferenças entre Facades Remotos e Facades de Sessão?

Resposta:Os Facades Remotos se concentram em fornecer uma interface fina para modelos de objetos de granularidade fina projetados para chamadas remotas, enquanto os Facades de Sessão geralmente englobam mais lógica de coordenação e fluxos de trabalho, frequentemente contendo lógica de domínio. A principal diferença reside em seu propósito pretendido e no volume de lógica que encapsulam.

7.Pergunta

Quando um Objeto de Transferência de Dados deve ser utilizado em uma aplicação?

Resposta:Use um Objeto de Transferência de Dados sempre que for necessário transferir múltiplos itens de dados entre processos, particularmente em chamadas remotas. Eles simplificam a estrutura de dados para comunicação, ajudando a gerenciar efetivamente os custos de rede e problemas de latência.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

8.Pergunta

Quais formatos de serialização são comumente usados com Objetos de Transferência de Dados e por quê?

Resposta:Os formatos de serialização comuns para Objetos de Transferência de Dados incluem binário e XML, sendo a serialização binária eficiente para desempenho de rede devido à menor sobrecarga de dados, enquanto a serialização XML é preferida por sua facilidade de leitura e compatibilidade entre sistemas. A escolha do método de serialização muitas vezes depende da flexibilidade necessária para mudanças na estrutura de dados.

9.Pergunta

Quais desafios podem surgir do uso de Objetos de Transferência de Dados, particularmente em relação à sincronização?

Resposta:Os desafios com Objetos de Transferência de Dados incluem garantir a sincronização entre as definições do servidor e do cliente, o que, se não for gerenciado, pode levar a falhas de comunicação quando o cliente não corresponde às expectativas do servidor. Isso requer uma

Mais Livros Grátis
Escaneie para baixar
Ouça-o

cuidadosa gestão de versões e, potencialmente, mecanismos de serialização tolerantes para evitar erros decorrentes de problemas de desserialização.

10.Pergunta

O que o capítulo sugere sobre a relação entre o design de façade e as necessidades da interface do usuário?

Resposta:O capítulo sugere que, ao projetar um Facade Remoto, ele deve ser adaptado às necessidades específicas da interface do usuário, com métodos projetados para facilitar interações do usuário de forma eficaz. Isso significa que o facade pode precisar englobar vários métodos para lidar com operações de dados em massa que se alinham com as atividades típicas do usuário na aplicação.

Capítulo 24 | Padrões de Concurrency Offline| Perguntas e respostas

1.Pergunta

Qual é o princípio fundamental por trás do padrão de bloqueio offline otimista?

Resposta:O princípio fundamental do padrão de bloqueio offline otimista é validar alterações em um

Mais Livros Grátis
Escaneie para baixar
Ouça-o

registro, garantindo que nenhuma outra sessão o modificou desde que foi carregado. Quando uma sessão tenta confirmar as alterações, ela verifica a versão do registro em relação à versão armazenada em seu estado de sessão, permitindo que múltiplos usuários trabalhem nos mesmos dados simultaneamente, enquanto previne conflitos. Se outra sessão atualizou o registro, a transação é revertida para manter a integridade dos dados.

2.Pergunta

Como a abordagem de bloqueio offline pessimista difere do bloqueio offline otimista?

Resposta:A abordagem de bloqueio offline pessimista presume que os conflitos são prováveis e os previne ao adquirir um bloqueio no registro de dados antes que qualquer modificação seja feita. Isso limita a concorrência, pois apenas uma sessão pode acessar os dados bloqueados de cada vez, garantindo que uma transação comercial possa prosseguir sem interrupções. Em contraste, o bloqueio offline otimista

Mais Livros Grátis
Escaneie para baixar
Ouça-o

permite que várias transações tentem alterações, concentrando-se na detecção de conflitos em vez da prevenção.

3.Pergunta

Por que é essencial utilizar números de versão na implementação do bloqueio offline otimista?

Resposta:Os números de versão são essenciais na implementação do bloqueio offline otimista, pois ajudam a rastrear alterações nos registros. Cada vez que um registro é atualizado, seu número de versão é incrementado. Quando uma sessão tenta confirmar suas alterações, deve verificar se o número de versão no banco de dados corresponde ao que está armazenado na sessão. Se não corresponder, a transação falha, protegendo contra atualizações perdidas e garantindo a consistência dos dados.

4.Pergunta

Quais desafios podem surgir ao implementar transações concorrentes em uma aplicação empresarial?

Resposta:Os desafios ao implementar transações

Mais Livros Grátis
Escaneie para baixar
Ouça-o

concorrentes incluem lidar com problemas de integridade de dados, como atualizações perdidas e leituras inconsistentes, garantir que as sessões não interfiram umas nas outras e gerenciar as expectativas dos usuários quanto ao tratamento de erros quando conflitos ocorrem. Complexidades também surgem quando múltiplas sessões tentam ler ou modificar dados compartilhados, necessitando de estratégias para detectar e resolver conflitos.

5.Pergunta

Como os bloqueios de granularidade grossa ajudam a resolver alguns problemas de concorrência?

Resposta:Os bloqueios de granularidade grossa simplificam o gerenciamento do acesso concorrente ao tratar um grupo de objetos relacionados como uma única entidade bloqueável. Em vez de adquirir bloqueios individuais para cada objeto em um grupo, o que pode ser complicado e reduzir o desempenho, um único bloqueio é usado para todo o conjunto. Isso reduz a contenção e facilita o gerenciamento do acesso concorrente, permitindo que operações em objetos

Mais Livros Grátis
Escaneie para baixar
Ouça-o

relacionados ocorram simultaneamente sem interferências.

6.Pergunta

Em que cenários você preferiria usar o bloqueio offline pessimista em vez do bloqueio offline otimista?

Resposta:O bloqueio offline pessimista deve ser preferido em cenários onde a probabilidade de modificações concorrentes é alta ou o custo de um conflito (como corrupção ou perda de dados) é significativo. Isso inclui situações onde a consistência dos dados é crítica e a experiência do usuário deve ser priorizada para evitar a perda de entradas do usuário ou a necessidade de retrabalhos frequentes devido a conflitos.

7.Pergunta

Quais implicações o uso de bloqueio implícito tem em aplicações empresariais?

Resposta:O uso de bloqueio implícito pode reduzir significativamente a carga sobre os desenvolvedores, gerenciando automaticamente a mecânica de bloqueio dentro da estrutura da aplicação. No entanto, requer que a estrutura garanta que todos os bloqueios necessários sejam adquiridos

Mais Livros Grátis
Escaneie para baixar
Ouça-o

e liberados adequadamente. Se algum bloqueio for esquecido ou mal gerido, isso pode levar a problemas críticos de consistência e erros não detectados, tornando essencial que os desenvolvedores compreendam e confiem no mecanismo de bloqueio implícito.

8.Pergunta

Quais são os potenciais desafios de depender apenas de bloqueios offline otimistas em um ambiente de alta concorrência?

Resposta:Em um ambiente de alta concorrência, depender apenas de bloqueios offline otimistas pode levar a falhas frequentes nas transações, já que múltiplas sessões competem para confirmar alterações. Isso pode resultar em tempo de processamento desperdiçado para os usuários, diminuição da satisfação e possíveis inconsistências nos dados se não for gerido adequadamente. Além disso, sem feedback adequado para o usuário e um tratamento de erros eficiente, os usuários podem se frustrar com o desempenho do sistema.

9.Pergunta

Descreva um cenário onde verificações de versão podem

Mais Livros Grátis
Escaneie para baixar
Ouça-o

ajudar a detectar leituras inconsistentes. Como você implementaria isso?

Resposta:Em um cenário de sistema de cobrança, onde uma sessão gera uma cobrança com base em dados de clientes, se outra sessão atualizar o endereço do cliente durante esse processo, o cálculo da cobrança pode se tornar inválido. Para implementar verificações de versão, a sessão de geração da cobrança também deveria incluir uma verificação de versão nos dados de endereço do cliente antes de confirmar a cobrança. Isso pode ser feito adicionando o registro de endereço ao conjunto de alterações e verificando sua versão junto aos detalhes da cobrança antes de finalizar a transação.

10.Pergunta

Por que a prevenção de deadlocks deve ser uma consideração ao usar bloqueios offline pessimistas?

Resposta:A prevenção de deadlocks é crucial com bloqueios offline pessimistas, pois bloquear recursos pode criar situações em que duas ou mais transações esperam indefinidamente umas pelas outras liberarem os bloqueios,

Mais Livros Grátis
Escaneie para baixar
Ouça-o

levando a operações paralisadas. Implementar uma estratégia para lidar com potenciais deadlocks evita que o sistema se torne não responsivo, mantendo a experiência do usuário e o desempenho da aplicação. Métodos como evitar transações longas, usar mecanismos de timeout ou implementar algoritmos de detecção de deadlocks podem ajudar a mitigar esse risco.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Ad

Escanear para baixar

App Store
Escolha dos Editores

22k avaliações de 5 estrelas

Feedback Positivo

Afonso Silva

s cada resumo de livro não só
o, mas também tornam o
m divertido e envolvente. O
tou a leitura para mim.

Fantástico! Brígida Santos

Estou maravilhado com a variedade de livros e idiomas que o Bookey suporta. Não é apenas um aplicativo, é um portal para o conhecimento global. Além disso, ganhar pontos para caridade é um grande bônus!

Fi

O
só
o
O

na Oliveira

correr as
ém me dá
omprar a
ar!

Adoro! Duarte Costa

Usar o Bookey ajudou-me a cultivar um hábito de leitura sem sobrecarregar minha agenda. O design do aplicativo e suas funcionalidades são amigáveis, tornando o crescimento intelectual acessível a todos.

Economiza tempo!

O Bookey é o meu apli
crescimento intelectua
perspicazes e lindame
um mundo de conheci

Aplicativo incrível! Estevão Pereira

Eu amo audiolivros, mas nem sempre tenho tempo para ouvir o livro inteiro! O Bookey permite-me obter um resumo dos destaques do livro que me interessa!!! Que ótimo conceito!!! Altamente recomendado!

Aplicativo lindo

Este aplicativo é um salva-vidas para
de livros com agendas lotadas. Os re
precisos, e os mapas mentais ajudar
o que aprendi. Altamente recomend

Teste gratuito com Bookey

Capítulo 25 | Padrões de Estado de Sessão| Perguntas e respostas

1.Pergunta

Qual é a importância do Estado de Sessão do Cliente em um ambiente de servidor sem estado?

Resposta:O Estado de Sessão do Cliente permite que o servidor mantenha a ausência de estado, armazenando os dados da sessão do lado do cliente. Isso é essencial para aplicações onde a redução da carga do servidor e a manutenção de alta disponibilidade são críticas, permitindo recursos como clustering e failover.

2.Pergunta

Descreva as três formas comuns de implementar o Estado de Sessão do Cliente.

Resposta:As três formas comuns de implementar o Estado de Sessão do Cliente são: 1) **Parâmetros de URL** - Adequados para pequenas quantidades de dados, mas limitados pelo comprimento da URL. 2) **Campos Ocultos** - Os dados são armazenados em um elemento de

Mais Livros Grátis
Escaneie para baixar
Ouça-o

entrada oculto no HTML, mas permanecem visíveis para os usuários no código da página. 3) **Cookies** - Enviados automaticamente com as requisições, mas têm limitações de tamanho e podem ser desativados pelos usuários.

3.Pergunta

Quais são as vantagens e desvantagens de usar cookies para o Estado de Sessão do Cliente?

Resposta:As vantagens dos cookies incluem gerenciamento automático de estado e facilidade de uso no gerenciamento de dados da sessão. As desvantagens incluem limitações de tamanho, resistência potencial dos usuários (já que eles podem desativar os cookies) e riscos de segurança, pois informações sensíveis podem ser expostas, a menos que sejam criptografadas.

4.Pergunta

Como o Estado de Sessão do Servidor difere do Estado de Sessão do Cliente?

Resposta:O Estado de Sessão do Servidor armazena todos os dados da sessão no servidor em vez de no cliente, o que pode

Mais Livros Grátis
Escaneie para baixar
Ouça-o

simplificar o gerenciamento de dados, mas pode introduzir problemas de escalabilidade. Em contraste, o Estado de Sessão do Cliente descarrega o armazenamento para o cliente, mantendo o servidor sem estado à custa de potenciais preocupações com segurança e aumento na transferência de dados.

5.Pergunta

Quais são os casos de uso recomendados para o Estado de Sessão em Banco de Dados?

Resposta:O Estado de Sessão em Banco de Dados é ideal para aplicações que requerem estado persistente entre instâncias de servidor, frequentemente utilizadas em ambientes agrupados onde os dados da sessão precisam ser acessíveis por múltiplos servidores. Esse método é eficaz para aplicações que não dependem fortemente de tempos de resposta imediatos, já que interações com o banco de dados podem introduzir latência.

6.Pergunta

Quais preocupações de segurança devem ser abordadas ao usar o Estado de Sessão do Cliente?

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Resposta:Ao usar o Estado de Sessão do Cliente, uma preocupação principal é a vulnerabilidade dos dados da sessão à interceptação e manipulação. É crucial criptografar informações sensíveis e validar qualquer dado recebido do cliente para evitar sequestro de sessão e manipulação de dados.

7.Pergunta

Por que uma aplicação pode preferir usar o Estado de Sessão do Servidor em vez do Estado de Sessão do Cliente?

Resposta:Uma aplicação pode preferir o Estado de Sessão do Servidor ao lidar com grandes quantidades de dados de sessão ou quando agentes do usuário (como navegadores) não podem armazenar dados suficientes devido a limitações em cookies ou comprimento de URL. O Estado de Sessão do Servidor simplifica o gerenciamento de dados e oferece melhor controle sobre a gestão de recursos.

8.Pergunta

Explique as implicações do uso de beans de sessão com estado versus beans de sessão sem estado no

Mais Livros Grátis
Escaneie para baixar
Ouça-o

gerenciamento de sessões.

Resposta:Beans de sessão com estado mantêm informações específicas do cliente em várias chamadas, simplificando o gerenciamento de sessões para aplicações que precisam de estado persistente. No entanto, eles podem introduzir gargalos de desempenho em comparação com beans sem estado, que são mais eficientes para lidar com múltiplos clientes sem reter informações de estado.

9.Pergunta

Quais desafios podem surgir ao usar o Estado de Sessão em Banco de Dados em termos de gerenciamento de transações?

Resposta:Usar o Estado de Sessão em Banco de Dados pode complicar o gerenciamento de transações porque os dados da sessão precisam ser separados entre estados ativos e pendentes. Isso pode levar a problemas de integridade de dados e concorrência, especialmente quando múltiplas transações estão acessando os mesmos dados da sessão. Gerenciar o ciclo de vida dos dados da sessão e garantir que

Mais Livros Grátis
Escaneie para baixar
Ouça-o

não conflitem com outras requisições ativas torna-se crítico.

10.Pergunta

Como os desenvolvedores podem mitigar os impactos de desempenho do Estado de Sessão em Banco de Dados?

Resposta:Os desenvolvedores podem mitigar os impactos de desempenho implementando caching para os dados da sessão, reduzindo leituras e gravações frequentes no banco de dados. Além disso, otimizar consultas ao banco de dados e garantir indexação adequada nos dados relacionados à sessão pode melhorar significativamente o desempenho.

11.Pergunta

O que o capítulo recomenda em relação ao gerenciamento de IDs de sessão para minimizar riscos de segurança?

Resposta:O capítulo recomenda usar IDs de sessão aleatórios gerados pela plataforma e aproveitar práticas seguras, como hashing, para minimizar os riscos de roubo de sessão. Sempre valide os IDs de sessão recebidos dos clientes para garantir que correspondam a sessões ativas.

Capítulo 26 | Padrões Básicos| Perguntas e respostas

Mais Livros Grátis
Escaneie para baixar
Ouça-o

1.Pergunta

Qual é o papel de um Gateway na arquitetura de aplicações corporativas?

Resposta:Um Gateway é um objeto que encapsula o acesso a um sistema ou recurso externo. Ele simplifica a interação com APIs complexas ou complicadas ao fornecer uma API mais direta que se alinha com as necessidades da aplicação, permitindo que outros objetos a utilizem sem se preocupar com as complexidades do recurso subjacente.

2.Pergunta

Quando você deve considerar o uso de um Gateway?

Resposta:Você deve considerar o uso de um Gateway sempre que tiver uma interface complicada para um recurso externo. Essa abordagem evita que a complexidade se espalhe pelo sistema e torna o código mais fácil de ler e gerenciar.

3.Pergunta

Como um Gateway facilita os testes em aplicações de software?

Resposta:Um Gateway fornece um ponto claro para

Mais Livros Grátis
Escaneie para baixar
Ouça-o

implantar Stubs de Serviço, que podem simular interações externas durante os testes. Ao ter um Gateway, você pode trocar o serviço real por um stub para executar testes sem depender do sistema externo real, aumentando assim a confiabilidade e a velocidade dos seus testes.

4.Pergunta

Quais são os benefícios de manter a implementação de um Gateway minimalista?

Resposta:Um Gateway minimalista foca em seu papel essencial de adaptar o serviço externo, evitando complexidade desnecessária. Esse design garante que qualquer lógica complexa permaneça dentro dos clientes do Gateway, resultando em um código mais simples que é mais fácil de manter e menos propenso a erros.

5.Pergunta

Você pode explicar o conceito de um Mapper e como ele difere de um Gateway?

Resposta:Um Mapper estabelece comunicação entre dois sistemas independentes, permitindo que eles interajam sem

Mais Livros Grátis
Escaneie para baixar
Ouça-o

precisar conhecer um ao outro. Normalmente, envolve tarefas de mapeamento mais complicadas, enquanto um Gateway simplesmente encapsula o acesso a recursos externos. Assim, Mappers são mais complexos e geralmente usados quando subsistemas precisam permanecer completamente desacoplados.

6.Pergunta

O que é um Objeto de Valor e por que é importante no design de software?

Resposta:Um Objeto de Valor é um pequeno objeto imutável que representa um aspecto descritivo do domínio, como dinheiro ou uma data. Sua igualdade é baseada nos valores reais dos campos em vez da identidade do objeto. Objetos de Valor encapsulam atributos que são essenciais para a lógica de negócios sem as complicações de referências a objetos, permitindo um código mais claro e confiável.

7.Pergunta

Como a classe Money ilustra o conceito de um Objeto de Valor?

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Resposta:A classe Money atua como um Objeto de Valor ao encapsular um valor e uma moeda. Ela garante que os cálculos sejam feitos com precisão, lida com problemas de arredondamento e mantém a imutabilidade, prevenindo mudanças inesperadas quando valores monetários são compartilhados entre diferentes partes de uma aplicação.

8.Pergunta

Qual é o propósito de uma subclasse Special Case no design de aplicações?

Resposta:Uma subclasse Special Case é implementada para lidar com condições específicas que, de outra forma, resultariam em valores nulos. Ao fornecer um objeto padrão que se comporta de maneira controlada (por exemplo, um objeto nulo), os desenvolvedores podem evitar verificações nulas espalhadas pelo código, simplificando a lógica e reduzindo a redundância.

9.Pergunta

Em quais cenários o padrão Plugin é utilizado e como ele melhora a flexibilidade do software?

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Resposta:O padrão Plugin é aplicado quando comportamentos necessitam de implementações diferentes com base em condições de tempo de execução. Ele permite configurações centralizadas e facilmente gerenciáveis, permitindo que os desenvolvedores troquem implementações sem modificar o código. Essa flexibilidade é particularmente útil em implantações em múltiplos ambientes.

10.Pergunta

Quais são as principais preocupações ao desenvolver um Registro em uma aplicação?

Resposta:Ao desenvolver um Registro, as principais preocupações incluem o escopo dos dados (processo, thread ou sessão), garantir fácil acesso aos métodos sem expor as estruturas de dados subjacentes, e gerenciar o estado global com cuidado para evitar conflitos, especialmente em ambientes multi-thread.

11.Pergunta

Como interfaces explícitas podem melhorar a usabilidade de estruturas de dados como Conjuntos de Registro?

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Resposta:Interfaces explícitas fornecem métodos e propriedades bem definidos para interagir com estruturas de dados como Conjuntos de Registro, melhorando a segurança de tipos e facilitando o acesso dos desenvolvedores aos atributos dos dados. Elas ajudam a evitar confusões que podem surgir de interfaces implícitas, onde o significado dos métodos de acesso é menos claro.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Ad

Escanear para baixar

Ler, Compartilhar, Empoderar

Conclua Seu Desafio de Leitura, Doe Livros para Crianças Africanas.

O Conceito

BOOKS FOR AFRICA × ×

Esta atividade de doação de livros está sendo realizada em conjunto com a Books For Africa.Lançamos este projeto porque compartilhamos a mesma crença que a BFA: Para muitas crianças na África, o presente de livros é verdadeiramente um presente de esperança.

A Regra

Ganhe 100 pontos ----> **Resgate um livro** ----> **Doe para a África**

Seu aprendizado não traz apenas conhecimento, mas também permite que você ganhe pontos para causas beneficentes! Para cada 100 pontos ganhos, um livro será doado para a África.

Teste gratuito com Bookey

Padrões de Arquitetura de Aplicações Corporativas Quiz e teste

Ver a resposta correta no site do Bookey

Capítulo 1 | Para Quem Este Livro É| Quiz e teste

1. O livro 'Padrões de Arquitetura de Aplicações Corporativas' é destinado a programadores, designers e arquitetos que trabalham em aplicações corporativas.
2. O livro apresenta conceitos completamente novos no campo da arquitetura de aplicações corporativas que nunca foram discutidos antes.
3. Os padrões no livro podem ajudar a melhorar a comunicação entre designers, fornecendo um vocabulário comum.

Capítulo 2 | Agradecimentos| Quiz e teste

1. Martin Fowler reconhece David Rice por seu papel significativo no desenvolvimento do livro.
2. Kyle Brown e Rachel Reinitz são mencionados como colaboradores de uma empresa diferente da ThoughtWorks.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

3. Fowler expressa gratidão à sua esposa, Cindy, por seu apoio durante o processo de escrita.

Capítulo 3 | Colofão| Quiz e teste

1. Martin Fowler escreveu o livro usando XML e tecnologias relacionadas.
2. O livro foi publicado em novembro de 2000.
3. A Ponte Leonard P. Zakim Bunker Hill é a ponte estaiada mais larga do mundo.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Escanear para baixar

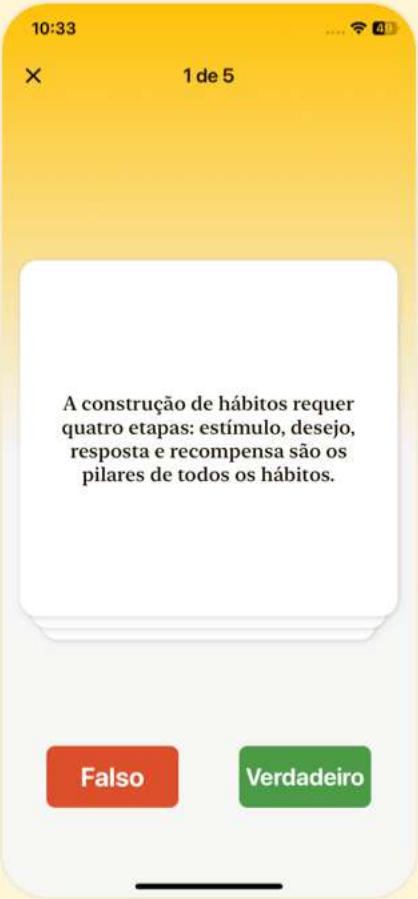
Baixe o app Bookey para desfrutar

Mais de 1000 resumos de livros com quizzes

Teste grátis disponível!

Descarregar na App Store

OBTER NO Google Play



Capítulo 4 | Arquitetura| Quiz e teste

1. A arquitetura na indústria de software é universalmente aceita e não gera confusão.
2. Decisões feitas no início do processo de desenvolvimento de um sistema são frequentemente vistas como mais difíceis de mudar posteriormente.
3. A arquitetura em camadas é o único estilo arquitetural que pode ser usado em aplicações corporativas segundo Martin Fowler.

Capítulo 5 | Aplicações Corporativas| Quiz e teste

1. Aplicações corporativas são exclusivamente sistemas grandes e não podem incluir aplicações menores.
2. Dados persistentes em aplicações corporativas devem ser acessíveis em várias execuções de programa e muitas vezes se estendem por anos.
3. As aplicações corporativas geralmente dependem de arquivos planos em vez de bancos de dados relacionais para o gerenciamento de dados.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Capítulo 6 | Tipos de Aplicações Corporativas| Quiz e teste

1. Os Varejistas Online B2C requerem uma lógica de negócios complicada devido ao alto volume de usuários.
2. Um Sistema de Processamento de Locação geralmente lida com menos usuários, mas possui uma lógica de negócios mais complexa em comparação a um Varejista Online B2C.
3. A escolha da arquitetura para aplicações corporativas pode ser uniforme entre diferentes tipos de aplicações.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Escanear para baixar

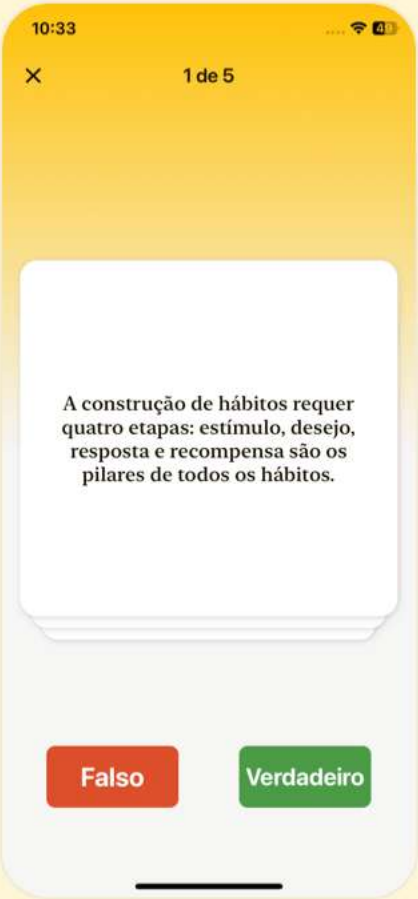
Baixe o app Bookey para desfrutar

Mais de 1000 resumos de livros com quizzes

Teste grátis disponível!

Descarregar na App Store

OBTER NO Google Play



Capítulo 7 | Pensando em Performance| Quiz e teste

1. O desempenho é um aspecto irrelevante nas decisões arquitetônicas e pode ser ignorado durante o processo de desenvolvimento.
2. A escalabilidade vertical refere-se a adicionar mais servidores para melhorar o desempenho de um sistema.
3. É importante validar as recomendações de desempenho com base na configuração específica da aplicação antes da implementação.

Capítulo 8 | Padrões| Quiz e teste

1. Os Padrões fornecem soluções exaustivas para todos os desafios de design em aplicações corporativas.
2. Compreender os Padrões estabelece um vocabulário compartilhado entre os designers, facilitando uma comunicação melhor.
3. Os Padrões podem ser aplicados de forma rígida, sem a necessidade de ajustes para se adequarem a situações específicas.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Capítulo 9 | Camadas| Quiz e teste

1. A organização em camadas na arquitetura de software envolve a divisão de sistemas complexos em subsistemas gerenciáveis dispostos em uma estrutura de 'bolo de camadas'.
2. A camada de domínio de uma aplicação é responsável por exibir informações e gerenciar interações do usuário.
3. Manter uma separação clara entre as camadas em uma aplicação pode levar a uma melhor manutenibilidade e flexibilidade.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Escanear para baixar

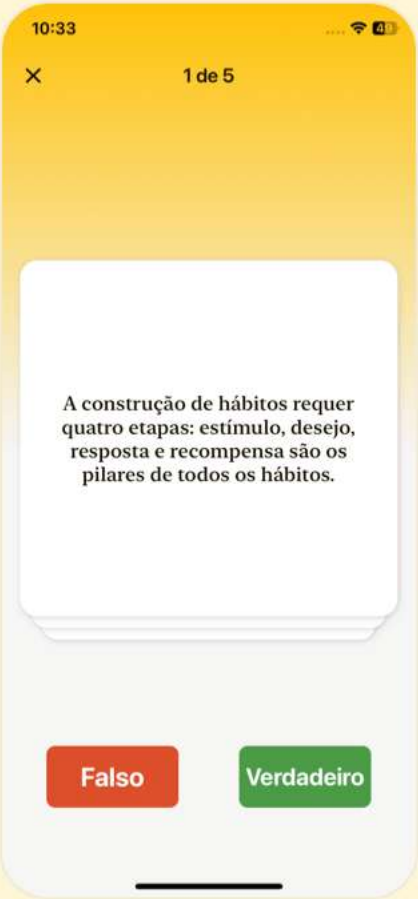
Baixe o app Bookey para desfrutar

Mais de 1000 resumos de livros com quizzes

Teste grátis disponível!

Descarregar na App Store

OBTER NO Google Play



Capítulo 10 | Organizando a Lógica de Domínio| Quiz e teste

1. O Transaction Script é o padrão mais simples e é fácil para os desenvolvedores entenderem e implementarem.
2. O Domain Model organiza a lógica de domínio em torno dos verbos do domínio.
3. Uma Service Layer pode fornecer uma API simplificada e gerenciar o controle de transações entre a lógica de apresentação e o modelo de domínio.

Capítulo 11 | Mapeamento para Bancos de Dados Relacionais| Quiz e teste

1. A camada de fonte de dados foca principalmente em bancos de dados NoSQL nos designs de aplicação.
2. Active Record combina lógica de domínio com acesso a dados, o que pode exigir uma transição para padrões como Data Mapper à medida que a complexidade cresce.
3. O uso de pool de conexões com o banco de dados é desnecessário, uma vez que as conexões podem ser abertas

Mais Livros Grátis
Escaneie para baixar
Ouça-o

e fechadas conforme necessário, sem preocupações com eficiência.

Capítulo 12 | Apresentação Web| Quiz e teste

1. O surgimento de interfaces de usuário baseadas em navegadores web tornou desnecessária a instalação de software cliente.
2. O padrão MVC é projetado para agrupar o manuseio de entrada, lógica de negócios e apresentação para uma aplicação web.
3. Todas as aplicações web se beneficiam do uso de um Controlador de Aplicação para gerenciar o fluxo de apresentação, independentemente de sua complexidade.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Escanear para baixar

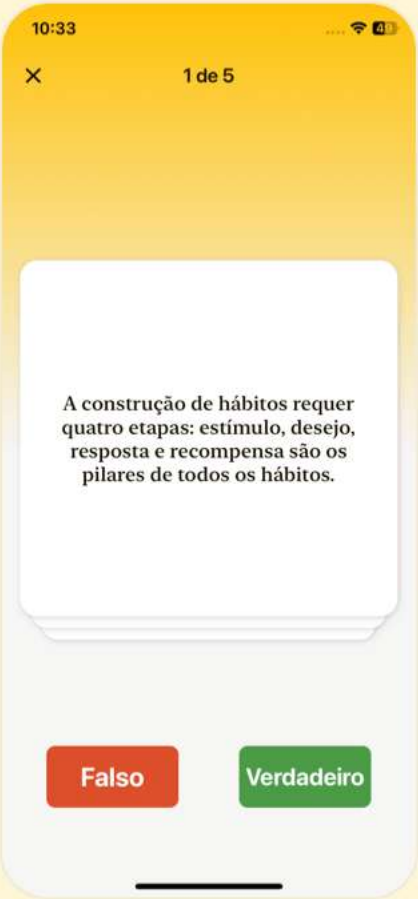
Baixe o app Bookey para desfrutar

Mais de 1000 resumos de livros com quizzes

Teste grátis disponível!

Descarregar na App Store

OBTER NO Google Play



Capítulo 13 | Concorrência| Quiz e teste

1. Os problemas de concorrência podem levar a atualizações perdidas, onde as alterações de um usuário podem sobrepor as alterações de outro, causando perda de dados.
2. A concorrência online é mais complexa do que a concorrência offline devido à menor necessidade de gerenciamento de transações em aplicações corporativas.
3. O Controle de Concorrência Otimista permite múltiplas edições e verifica conflitos ao atualizar, enquanto o Controle de Concorrência Pessimista restringe a edição até que um usuário tenha terminado.

Capítulo 14 | Estado da Sessão| Quiz e teste

1. Transações de negócios e transações de sistema são fundamentalmente as mesmas e não impactam nas estratégias de concorrência.
2. Servidores sem estado ajudam na gestão de recursos ao não reter estado entre as requisições.
3. O estado de sessão do servidor envolve um esforço de

Mais Livros Grátis
Escaneie para baixar
Ouça-o

desenvolvimento mais extenso em comparação com os estados de sessão do cliente e do banco de dados.

Capítulo 15 | Estratégias de Distribuição| Quiz e teste

1. Distribuir objetos em aplicações corporativas sempre leva a uma melhoria de desempenho.
2. Fowler recomenda o uso de interfaces de granulação fina para objetos remotos a fim de minimizar o número de chamadas através da fronteira de distribuição.
3. A comunicação assíncrona baseada em mensagens é preferida em relação a métodos RPC síncronos em sistemas distribuídos, segundo Fowler.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Escanear para baixar

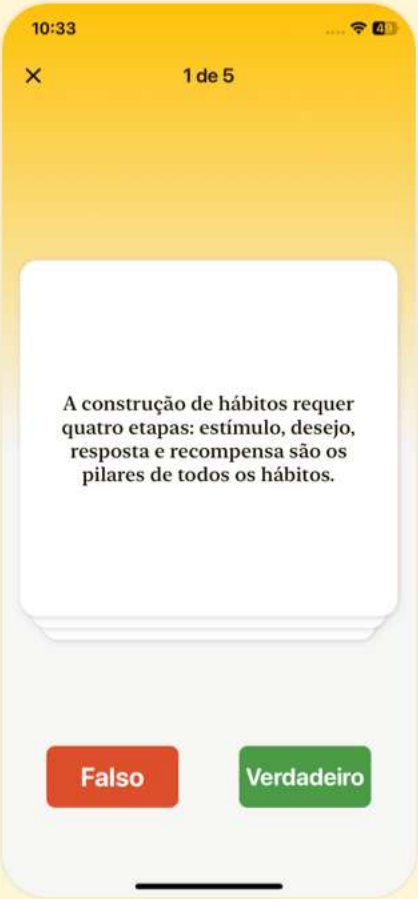
Baixe o app Bookey para desfrutar

Mais de 1000 resumos de livros com quizzes

Teste grátis disponível!

Descarregar na App Store

OBTER NO Google Play



Capítulo 16 | Unindo Tudo| Quiz e teste

1. O autor sugere que as decisões arquitetônicas na arquitetura de aplicações corporativas devem ser tomadas estritamente com base nas orientações fornecidas no capítulo.
2. O Modelo de Domínio é a melhor escolha para aplicações com lógica de negócio complexa, de acordo com o resumo do capítulo.
3. Procedimentos armazenados são sempre recomendados para otimizar o desempenho sem nenhuma desvantagem, segundo a discussão do capítulo.

Capítulo 17 | Padrões de Lógica de Domínio| Quiz e teste

1. O padrão Transaction Script é adequado para aplicações com lógica de negócios complexa que entrelaça múltiplas regras.
2. Um Modelo de Domínio oferece maior flexibilidade e reduz a duplicação de código em comparação com um Transaction Script.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

3. Camadas de Serviço são aplicáveis apenas em aplicações com um único tipo de cliente e requisitos de transação simples.

Capítulo 18 | Padrões de Arquitetura de Fontes de Dados| Quiz e teste

1. O Gateway de Dados de Tabela é projetado para lidar com lógica de negócios complexa em aplicações.
2. O Active Record combina a lógica de acesso a dados com a lógica de domínio em objetos únicos.
3. O Data Mapper é ideal para modelos de domínio simples que se alinham de perto com o design do banco de dados.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Escanear para baixar

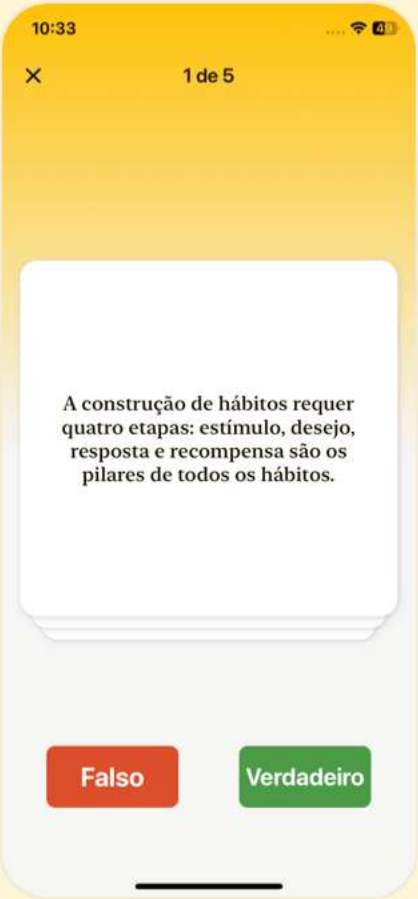
Baixe o app Bookey para desfrutar

Mais de 1000 resumos de livros com quizzes

Teste grátis disponível!

Descarregar na App Store

OBTER NO Google Play



Capítulo 19 | Padrões Comportamentais de Objeto-Relacional| Quiz e teste

1. O padrão Unit of Work gerencia uma lista de objetos afetados por uma transação de negócios.
2. O padrão Identity Map permite múltiplas instâncias do mesmo registro de banco de dados na memória para otimização de desempenho.
3. O padrão Lazy Load recupera todos os dados relacionados de uma vez para otimizar o uso de recursos.

Capítulo 20 | Padrões Estruturais Objetos-Relacionais| Quiz e teste

1. O padrão de Campo de Identidade permite que um objeto mantenha um identificador único que vincula a representação em memória à linha correspondente no banco de dados.
2. Chaves significativas, como Números de Segurança Social, são preferíveis em relação a chaves sem significado, como IDs gerados, pois reduzem o risco de erro humano.
3. O método de Herança de Tabela Única para representar hierarquias de classes inclui todos os campos de todas as

Mais Livros Grátis
Escaneie para baixar
Ouça-o

subclasses em uma única tabela, tornando a recuperação de dados simples.

Capítulo 21 | Padrões de Mapeamento de Metadados Objetos-Relacionais| Quiz e teste

1. O Mapeamento de Metadados ajuda a eliminar a codificação repetitiva frequentemente exigida para a correspondência entre banco de dados e objetos em memória.
2. A Programação Reflexiva oferece melhor desempenho do que a Geração de Código devido ao seu acesso dinâmico aos métodos de objeto.
3. Usar Objetos de Consulta é apropriado em sistemas que utilizam Modelos de Domínio e Mapeadores de Dados quando consultas complexas se tornam necessárias.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Escanear para baixar

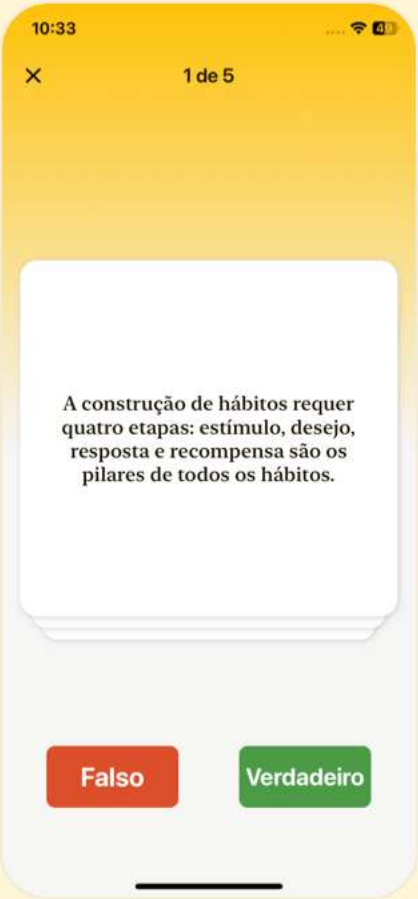
Baixe o app Bookey para desfrutar

Mais de 1000 resumos de livros com quizzes

Teste grátis disponível!

Descarregar na App Store

OBTER NO Google Play



Capítulo 22 | Padrões de Apresentação Web| Quiz e teste

1. O MVC separa a interação com a interface do usuário em três papéis distintos: Modelo, Visão e Controlador.
2. O Controlador de Página gerencia várias páginas em uma aplicação web simultaneamente.
3. As Visões de Template são adequadas para aplicações complexas que requerem modificações técnicas por não-programadores.

Capítulo 23 | Padrões de Distribuição| Quiz e teste

1. O padrão Remote Facade melhora a eficiência da rede ao oferecer uma interface de granularidade grossa que reduz o número de chamadas de método necessárias em interações remotas.
2. As Facades Remotas contêm lógica de domínio e podem misturá-la com suas operações, tornando-as versáteis para várias considerações de design.
3. Os Objetos de Transferência de Dados (DTOs) são

Mais Livros Grátis
Escaneie para baixar
Ouça-o

projetados para transportar dados entre processos de forma eficiente, e devem ser estruturados para garantir compatibilidade em ambos os lados de qualquer barreira de distribuição.

Capítulo 24 | Padrões de Concurrency Offline| Quiz e teste

1. O bloqueio otimista offline pressupõe que conflitos são comuns e os evita bloqueando dados antes das modificações.
2. O bloqueio pessimista offline é necessário quando a probabilidade de conflitos é alta e os custos de conflito são substanciais.
3. O bloqueio de granularidade grossa trata objetos individuais como entidades bloqueáveis separadas para evitar a complexidade na gestão de bloqueios.

Mais Livros Grátis
Escaneie para baixar
Ouça-o

Escanear para baixar

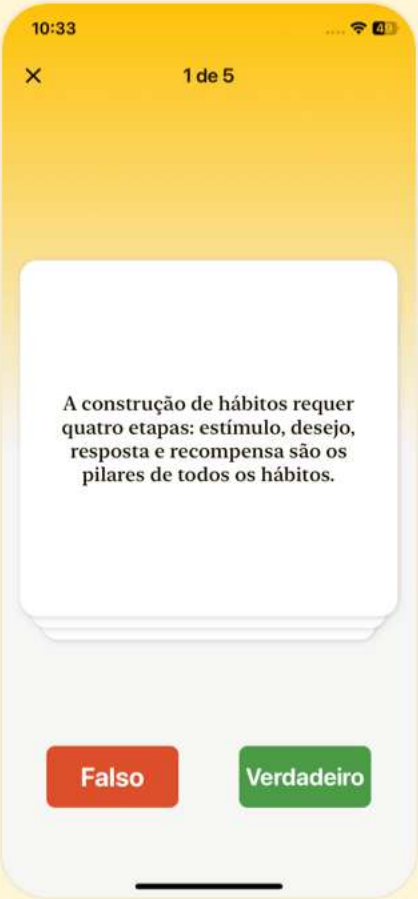
Baixe o app Bookey para desfrutar

Mais de 1000 resumos de livros com quizzes

Teste grátis disponível!

Descarregar na App Store

OBTER NO Google Play



Capítulo 25 | Padrões de Estado de Sessão| Quiz e teste

1. O Estado de Sessão do Cliente envolve armazenar dados da sessão no lado do servidor.
2. O Estado de Sessão do Servidor requer que o servidor acompanhe as informações da sessão do usuário usando um ID de sessão.
3. O Estado de Sessão do Banco de Dados é menos complexo que o Estado de Sessão do Cliente ou do Servidor porque elimina a necessidade de IDs de sessão.

Capítulo 26 | Padrões Básicos| Quiz e teste

1. Um Gateway é um objeto que simplifica APIs complexas de recursos externos, encapsulando-as em uma classe que imita uma interface de objeto regular.
2. Um Mapper permite que subsistemas permaneçam totalmente cientes uns dos outros para manter um baixo acoplamento.
3. O padrão de Interface Separada envolve a implementação

Mais Livros Grátis
Escaneie para baixar
Ouça-o

de uma interface no mesmo pacote onde ela é definida.

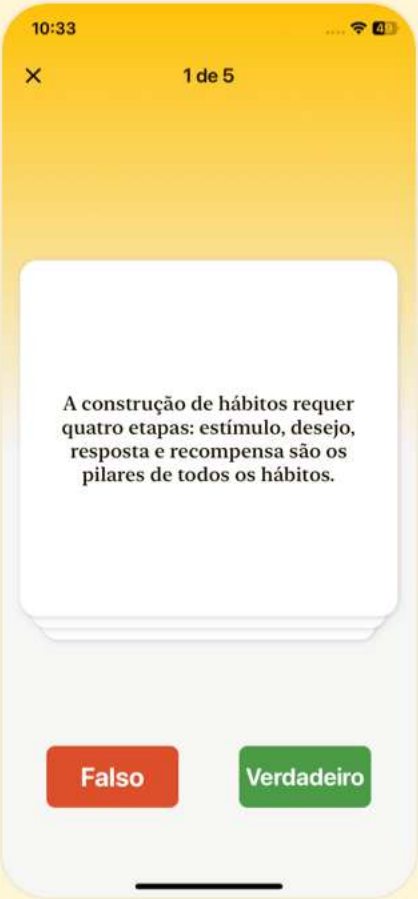
Mais Livros Grátis
Escaneie para baixar
Ouça-o

Escanear para baixar

Baixe o app Bookey para desfrutar

Mais de 1000 resumos de livros com quizzes

Teste grátis disponível!

Descarregar na App Store

OBTER NO Google Play